

ระบบจัดการโรงแรมรับฝากสัตว์เลี้ยง (PetsHotel)

PetsHotel: Pet Boarding Hotel Management System

จัดทำโดย

นายกิตติธัช คำโท รหัสนักศึกษา 673380029-0

นายณัช พนมเรียงไชย รหัสนักศึกษา 673380041-0

นางสาวรติมา สวัสดิ์นที รหัสนักศึกษา 673380055-9

นางสาวกิติญาดา กองคำ รหัสนักศึกษา 673380509-6

นางสาวนาเดีย คัดอ่าน รหัสนักศึกษา 673380513-5

เสนอ

รศ. ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and
Development (Section 1)

วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

ภาคเรียนที่ 1 ปีการศึกษา 2569

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาระบบจัดการโรงแรมรับฝากสัตว์เลี้ยง(PetsHotel)ในรูปแบบเว็บแอปพลิเคชันเพื่อแก้ปัญหการการจองผ่านช่องทางแชทหรือการจดบันทึกด้วยมือซึ่งทำให้ตรวจสอบห้องว่างได้ยากเกิดการจองซ้อนและเจ้าของสัตว์เลี้ยงไม่ทราบความเป็นอยู่ของสัตว์ระหว่างการฝากระบบรองรับผู้ใช้3บทบาทได้แก่ลูกค้าพนักงานและผู้ดูแลระบบลูกค้าสามารถค้นหาห้องว่างตามช่วงวันที่จองห้องพร้อมเลือกบริการเสริมและโปรโมชั่นและดูรายงานการดูแลรายวันพนักงานสามารถยืนยันการจอง เช็ก อิน เช็ก เอาต์ ยืนยัน รับชำระเงิน และบันทึก รายงาน การดูแล ส่วนผู้ดูแลระบบจัดการห้องพัก โปรโมชั่น บริการเสริม สมาชิก และดู Dashboard รายได้

ระบบพัฒนาด้วย Java 26 และ Spring Boot 4.1.1 ใช้ Thymeleafสำหรับส่วนแสดงผล SpringSecurity สำหรับการยืนยันตัวตนและกำหนดสิทธิ์PostgreSQLเป็นฐานข้อมูลโดยจัดการโครงสร้างด้วยFlywayและเปิดให้เรียกใช้ผ่าน REST API ที่มีเอกสาร Swagger การออกแบบใช้สถาปัตยกรรมแบบLayeredArchitectureร่วมกับDesign Pattern3รูปแบบได้แก่StateสำหรับสถานะการจองStrategyสำหรับการคำนวณราคาและObserverสำหรับการแจ้งเตือนทางอีเมล และวิเคราะห์ตามหลัก SOLID ผลการทดสอบด้วย JUnit 5 และ Mockito จำนวน 458 กรณี ผ่านทั้งหมด มี Line coverage 84.7% และ Branch coverage 70.9% ระบบติดตั้งใช้งานจริงด้วย Docker บน Render ร่วมกับฐานข้อมูล Neon PostgreSQL และมี GitHub Actions ทดสอบและ deploy อัตโนมัติ

คำสำคัญ: ระบบจองห้องพัก, โรงแรมสัตว์เลี้ยง, Spring Boot, Design Pattern, SOLID, REST API

Abstract

This project presents PetsHotel, a web-based management system for a pet boarding hotel. The system supports three roles. Customers can search for available rooms, book stays with extra services and promotions, and view daily care reports for their pets. Staff can confirm bookings, check pets in and out, confirm payments, and record daily care reports. Administrators can manage rooms, promotions, extra services and member accounts, and view a revenue dashboard. The system is built with Java 26, Spring Boot 4.1.1, Thymeleaf, Spring Security, PostgreSQL and Flyway. It follows a layered architecture and applies the State, Strategy and Observer design patterns, supported by an analysis based on the SOLID principles. All 458 automated tests pass, with 84.7% line coverage and 70.9% branch coverage. The system is deployed with Docker on Render with a Neon PostgreSQL database, and a GitHub Actions CI/CD pipeline builds, tests and deploys it automatically.

Keywords: booking system, pet hotel, Spring Boot, design patterns, SOLID, REST API

กิตติกรรมประกาศ

รายงานโครงงานฉบับนี้สำเร็จลุล่วงได้ด้วยความกรุณาจาก รศ. ดร.ปัญญาพล หอระตะที่ให้คำแนะนำเกี่ยวกับการออกแบบซอฟต์แวร์ การประยุกต์ใช้ Design Pattern และหลัก SOLI ตลอดจนตรวจสอบแก้ไขข้อบกพร่องต่าง ๆ ของโครงงาน คณะผู้จัดทำขอขอบพระคุณเป็นอย่างสูง

ขอขอบคุณสมาชิกทุกคนในกลุ่ม ที่ร่วมกันวิเคราะห์ ออกแบบ พัฒนา ทดสอบ และตรวจทานงานของกันและกัน จนระบบสำเร็จลุล่วง

คณะผู้จัดทำ

ตุลาคม 2569

สารบัญ

บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ.....	1
1.2 วัตถุประสงค์.....	1
1.3 ขอบเขตของโครงการ.....	2
1.3.1 ผู้ใช้งานระบบ	2
1.3.2 ขอบเขตด้านฟังก์ชัน	2
1.3.3 ขอบเขตด้านเทคโนโลยี	3
1.3.4 สิ่งอยู่นอกขอบเขต	3
1.4 ประโยชน์ที่คาดว่าจะได้รับ.....	3
1.5 นิยามศัพท์เฉพาะ	3
1.6 สมาชิกและการแบ่งหน้าที่	4
1.7 แผนการดำเนินงาน.....	4
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	6
2.1 สถาปัตยกรรมซอฟต์แวร์แบบชั้น (Layered Architecture).....	6
2.2 รูปแบบ Model-View-Controller (MVC).....	6
2.3 Design Patterns	6
2.4 หลักการออกแบบ SOLID.....	7
2.5 ภาษาการจำลองแบบรวม (UML)	7
2.6 เทคโนโลยีที่ใช้พัฒนา.....	7

2.7 REST API และ HTTP Status Code.....	8
2.8 การทดสอบซอฟต์แวร์	9
2.9 Container และ CI/CD.....	9
2.10 ระบบหรืองานที่เกี่ยวข้อง	9
บทที่ 3 การวิเคราะห์และออกแบบระบบ	11
3.1 การวิเคราะห์ความต้องการ	11
3.1.1 ความต้องการด้านฟังก์ชัน (Functional Requirements)	11
3.1.2 ความต้องการที่ไม่ใช่ฟังก์ชัน (Non-functional Requirements).....	11
3.2 แผนภาพ Use Case.....	12
รายการ Use Case	13
3.3 แบบจำลองโดเมน (Domain Model)	14
3.4 สถาปัตยกรรมระบบ	15
3.5 แผนภาพคลาส (Class Diagram).....	17
3.6 แผนภาพลำดับ (Sequence Diagram).....	18
3.7 แผนภาพกิจกรรมและแผนภาพสถานะ	22
3.8 กฎทางธุรกิจที่สำคัญ.....	24
3.8.1 การตรวจห้องว่าง	24
3.8.2 การคำนวณราคา	24
3.9 การออกแบบฐานข้อมูล	24
3.10 การออกแบบด้วย Design Pattern.....	26

สรุป.....	26
State Pattern.....	27
Observer Pattern.....	31
Strategy Pattern	34
Enterprise Patterns	37
3.11 การวิเคราะห์ตามหลัก SOLID	41
กิตติธัช: Room, User และ Notification.....	41
ธนัช: Availability, Room Search และ Exception Handling.....	43
รติมา: Pet และ Daily Care Report	44
กิติญาดา: Booking และ State Pattern.....	46
นาเดีย: Pricing และ Promotion.....	50
3.12 การออกแบบ REST API.....	52
3.12.1 การจัดการข้อผิดพลาด.....	53
3.13 การออกแบบความปลอดภัย.....	54
3.14 การออกแบบการติดตั้งใช้งาน.....	54
บทที่ 4 ผลการดำเนินงาน	56
4.1 สภาพแวดล้อมการพัฒนาและการติดตั้ง	56
4.2 ผลการพัฒนาส่วนต่าง ๆ ของระบบ	56
4.2.1 ส่วนของผู้เยี่ยมชมและลูกค้า	56
4.2.2 ส่วนของพนักงาน	62

4.2.3 ส่วนของผู้ดูแลระบบ.....	64
4.3 ผลการพัฒนา REST API.....	66
4.4 ผลการทดสอบระบบ.....	68
สรุปผล.....	68
ประเภทการทดสอบ.....	68
4.4.1 ตัวอย่างกรณีทดสอบที่สำคัญ.....	69
4.5 ผลการติดตั้งใช้งานและ CI/CD	70
4.6 การทำงานเป็นทีมด้วย Git.....	71
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ.....	73
5.1 สรุปผลการดำเนินงาน	73
5.2 อภิปรายผล.....	73
5.3 ปัญหาและแนวทางแก้ไข	74
5.4 ข้อจำกัดของระบบ.....	75
5.5 ข้อเสนอแนะและแนวทางพัฒนาต่อ.....	75
บรรณานุกรม.....	76
ภาคผนวก ก คำอธิบาย Use Case (Use Case Description)	78
ภาคผนวก ข พจนานุกรมข้อมูล (Data Dictionary)	100
ภาคผนวก ค คู่มือการติดตั้งและบัญชีทดสอบ	129
ภาคผนวก ง ประวัติผู้จัดทำ.....	132

สารบัญตาราง

ตารางที่ 1.1 บทบาทผู้ใช้และขอบเขตความสามารถ	2
ตารางที่ 1.2 นิยามศัพท์เฉพาะ	3
ตารางที่ 1.3 สมาชิกและหน้าที่รับผิดชอบ	4
ตารางที่ 1.4 แผนการดำเนินงาน	4
ตารางที่ 2.1 Design Pattern ที่ใช้ในระบบ	6
ตารางที่ 2.2 หลักการ SOLID (Martin, 2003).....	7
ตารางที่ 2.3 เทคโนโลยีที่ใช้ในการพัฒนา	7
ตารางที่ 2.4 เปรียบเทียบระบบที่เกี่ยวข้อง.....	9
ตารางที่ 3.1 ความต้องการด้านฟังก์ชัน	11
ตารางที่ 3.2 โครงสร้าง package ของระบบ.....	15
ตารางที่ 3.3 สูตรการคำนวณราคา.....	24
ตารางที่ 3.4 ตารางในฐานข้อมูล	25
ตารางที่ 3.5 รายการ REST API.....	52
ตารางที่ 3.6 การแปลง Exception เป็น HTTP Status.....	53
ตารางที่ 4.1 สภาพแวดล้อมการพัฒนา	56
ตารางที่ 4.2 ตัวอย่างกรณีทดสอบ.....	69
ตารางที่ 4.3 Branch และผลงานของสมาชิก	71
ตารางที่ 5.1 สรุปผลตามวัตถุประสงค์	73
ตารางที่ 5.2 ปัญหาที่พบและแนวทางแก้ไข	74

สารบัญรูป

รูปที่ 3.1 Use Case Diagram ของระบบ PetsHotel	13
รูปที่ 3.2 Domain Model	14
รูปที่ 3.3 Component Diagram	15
รูปที่ 3.4 Class Diagram	17
รูปที่ 3.5 Sequence Diagram: ค้นหาห้องว่าง (UC-03)	18
รูปที่ 3.6 Sequence Diagram: สร้างการจอง (UC-05).....	19
รูปที่ 3.7 Sequence Diagram: เช็กเอาท์ (UC-08)	20
รูปที่ 3.8 Sequence Diagram: บันทึกการรายงานการดูแล (UC-10).....	21
รูปที่ 3.9 Activity Diagram: กระบวนการจอง.....	22
รูปที่ 3.10 State Diagram: สถานะการจอง	23
รูปที่ 3.11 ER Diagram	25
รูปที่ 3.12 Class Diagram ของ Observer Pattern.....	33
รูปที่ 3.13 Deployment Diagram	54
รูปที่ 4.1 หน้าเข้าสู่ระบบ (/login).....	57
รูปที่ 4.2 หน้าสมัครสมาชิก (/register)	57
รูปที่ 4.3 หน้าแรกและรายการห้องพัก (/ , /rooms).....	58
รูปที่ 4.4 หน้าค้นหาห้องว่าง (/rooms/available)	59
รูปที่ 4.5 หน้าสัตว์เลี้ยงของฉัน (/pets).....	59
รูปที่ 4.6 หน้าจองห้องพัก (/bookings/new)	60

รูปที่ 4.7 หน้าการจองของฉันและรายละเอียดการจอง.....	61
รูปที่ 4.8 หน้ารายงานการดูแล (/reports).....	61
รูปที่ 4.9 หน้าบัญชีของฉัน (/profile).....	62
รูปที่ 4.10 หน้าจัดการการจอง (/admin/bookings).....	62
รูปที่ 4.11 หน้ารายงานการดูแล (/admin/reports).....	63
รูปที่ 4.12 อีเมลยืนยันการจองใน Mailpit.....	63
รูปที่ 4.13 หน้า Dashboard รายได้ (/admin).....	64
รูปที่ 4.14 หน้าจัดการห้องพัก.....	65
รูปที่ 4.15 หน้าจัดการโปรโมชั่น.....	65
รูปที่ 4.16 หน้าจัดการบริการเสริม.....	66
รูปที่ 4.17 หน้าจัดการสมาชิก.....	66
รูปที่ 4.18 หน้า Swagger UI แสดงรายการ API.....	67
รูปที่ 4.19 ตัวอย่างผลลัพธ์ JSON ของ GET /api/rooms/search และตัวอย่าง ErrorResponse (400/404/409).....	68
รูปที่ 4.20 สรุปผลการทดสอบ.....	69
รูปที่ 4.21 สรุป Code coverage จาก JaCoCo.....	69
รูปที่ 4.22 ผลการรัน GitHub Actions ที่ผ่านทั้ง build-test และ deploy.....	70
รูปที่ 4.23 ระบบที่ deploy แล้วบน Render (petshotel.onrender.com).....	71

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

ปัจจุบันผู้เลี้ยงสัตว์จำนวนมากมีความจำเป็นต้องฝากสัตว์เลี้ยงไว้กับโรงแรมสัตว์เลี้ยงเมื่อต้องเดินทางหรือไม่สามารถดูแลได้ แต่โรงแรมสัตว์เลี้ยงขนาดเล็กถึงกลางส่วนใหญ่ยังรับจองผ่านแชท โทรศัพท์ หรือจดบันทึกด้วยมือ ทำให้เกิดปัญหาหลายประการ ได้แก่

1. ลูกค้าต้องการฝากสัตว์เลี้ยงในวันที่กำหนด แต่ไม่ทราบว่ายังมีห้องที่เหมาะสมว่างหรือไม่ ต้องสอบถามและรอคำตอบจากพนักงาน
2. การตรวจสอบห้องว่างด้วยมือทำได้ยากและผิดพลาดง่าย อาจเกิดการจองห้องซ้อนช่วงวันเดียวกัน
3. เจ้าของฝากสัตว์เลี้ยงแล้วไม่ทราบว่าสัตว์เลี้ยงเป็นอย่างไร กินอาหาร ทำกิจกรรม หรือมีอาการผิดปกติหรือไม่
4. พนักงานต้องดูแลสัตว์หลายตัวซึ่งแต่ละตัวต้องการการดูแลต่างกัน การบันทึกข้อมูลแบบกระดาษทำให้ติดตามย้อนหลังได้ยาก
5. การคำนวณราคาที่มีทั้งค่าห้อง บริการเสริม ค่าธรรมเนียมวันหยุด และส่วนลด ทำด้วยมือแล้วผิดพลาดได้ง่าย

คณะผู้จัดทำจึงพัฒนาระบบ PetsHotel

ซึ่งเป็นเว็บแอปพลิเคชันสำหรับจัดการโรงแรมรับฝากสัตว์เลี้ยงตั้งแต่การค้นหาห้องว่าง การจอง การยืนยันและเช็คอิน-เช็คเอาท์ การรับชำระเงิน ไปจนถึงรายงานการดูแลรายวัน โดยเน้นการออกแบบซอฟต์แวร์ที่ติดตามเนื้อหารายวิชา ได้แก่ สถาปัตยกรรมแบบชั้น การใช้ Design Pattern หลัก SOLID การทดสอบอัตโนมัติ และการติดตั้งใช้งานจริงบนคลาวด์

1.2 วัตถุประสงค์

1. เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับจัดการการจองห้องพักของโรงแรมรับฝากสัตว์เลี้ยง ที่ป้องกันการจองห้องซ้อนช่วงวันเดียวกันได้
2. เพื่อให้ลูกค้าค้นหาห้องว่าง จองห้อง และติดตามรายงานการดูแลสัตว์เลี้ยงรายวันได้ด้วยตนเอง
3. เพื่อให้พนักงานและผู้ดูแลระบบจัดการการจอง ห้องพัก โปรโมชั่น บริการเสริม สมาชิก และดูสรุปรายได้ได้จากทีเดียว

4. เพื่อประยุกต์ใช้สถาปัตยกรรม Layered Architecture, Design Pattern (State, Strategy, Observer) และหลัก SOLID ในการพัฒนาซอฟต์แวร์จริง
5. เพื่อฝึกกระบวนการพัฒนาซอฟต์แวร์เป็นทีม ได้แก่ การใช้ Git, การทดสอบอัตโนมัติ, CI/CD และการ deploy ระบบขึ้นคลาวด์

1.3 ขอบเขตของโครงการ

1.3.1 ผู้ใช้งานระบบ

ตารางที่ 1.1 บทบาทผู้ใช้และขอบเขตความสามารถ

บทบาท	ความสามารถหลัก
ผู้เยี่ยมชม (Guest)	ดูห้องพัก ค้นหาห้องพักตามวันที่และจำนวนสัตว์ สมัครงาน
ลูกค้า (CUSTOMER)	จัดการข้อมูลสัตว์เลี้ยง จองห้องพักพร้อมบริการเสริมและโปรโมชั่น ยกเลิกการจอง ดูใบเสร็จ ดูรายงานการดูแล ดูข้อมูลบัญชีของตน
พนักงาน (STAFF)	ยืนยันการจอง เช็กอิน เช็กเอาท์ ยืนยันรับชำระเงิน ยกเลิกการจอง บันทึกและแก้ไขรายงานการดูแลรายวัน
ผู้ดูแลระบบ (ADMIN)	ทำได้ทุกอย่างของพนักงาน และจัดการห้องพัก โปรโมชั่น บริการเสริม สมัครงาน (ระงับ/เปิดใช้บัญชี) และดู Dashboard รายได้

1.3.2 ขอบเขตด้านฟังก์ชัน

- ระบบสมัครงาน: สมัครงาน เข้าสู่ระบบ จัดการเข้าสู่ระบบ (Remember me) 30 วัน
พาไปหน้าตามบทบาทหลังเข้าสู่ระบบ และแจ้งเตือนเมื่อบัญชีถูกระงับ
- ระบบค้นหาห้องพัก: ค้นหาตามวันเช็กอิน วันเช็กเอาท์ และจำนวนสัตว์
ตรวจสอบความจุและการจองทับช่วงวัน และดูปฏิทินห้องพักว่างรายห้อง
- ระบบจองห้องพัก: เลือกสัตว์เลี้ยง บริการเสริม โปรโมชั่น คำนวณราคาก่อนยืนยัน
และจัดการสถานะการจองด้วย State Pattern
- ระบบคำนวณราคา: ค่าห้องพักต่อสัตว์ต่อคืน ค่าบริการเสริม ค่าธรรมเนียมวันหยุด 30%
ส่วนลดพักยาว 10% และส่วนลดโปรโมชั่น
- ระบบรับชำระเงินและใบเสร็จ: พนักงานยืนยันรับชำระเงิน และลูกค้าดูใบเสร็จได้
- ระบบรายงานการดูแลรายวัน: พนักงานบันทึกการกิน กิจกรรม การดูแลความสะอาด
อารมณ์ และสุขภาพ ลูกค้าเปิดดูรายงานของสัตว์ตนเอง
- ระบบแจ้งเตือน: ส่งอีเมลยืนยันเมื่อพนักงานยืนยันการจอง ด้วย Observer Pattern
- ระบบผู้ดูแล: จัดการห้องพัก โปรโมชั่น บริการเสริม สมัครงาน และ Dashboard รายได้

- REST API พร้อมเอกสาร Swagger สำหรับห้อง การค้นหา การจอง การชำระเงิน สัตว์เลี้ยง รายงานการดูแล โปรโมชั่น และบริการเสริม

1.3.3 ขอบเขตด้านเทคโนโลยี

- ภาษา Java 26 และ Spring Boot 4.1.1 (Web MVC, Data JPA, Security, Validation, Mail)
- ส่วนแสดงผล Thymeleaf, ฐานข้อมูล PostgreSQL, จัดการ schema ด้วย Flyway
- ทดสอบด้วย JUnit 5, Mockito, Spring Boot Test และวัด coverage ด้วย JaCoCo
- ติดตั้งใช้งานด้วย Docker บน Render และฐานข้อมูล Neon PostgreSQL พร้อม CI/CD ด้วย GitHub Actions

1.3.4 สิ่งที่อยู่นอกขอบเขต

- การชำระเงินออนไลน์ผ่าน Payment Gateway (ระบบให้พนักงานเป็นผู้ยืนยันรับชำระเงิน)
- การลิ้มรสผ่าน / รีเซิร์ฟผ่านทางอีเมล
- แอปพลิเคชันบนมือถือแบบ Native

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ลูกค้าค้นหาห้องว่างและจองได้ตลอด 24 ชั่วโมง โดยไม่ต้องรอฟังก์ชันตอบ
2. ลดปัญหาการจองซ้อน เพราะระบบตรวจการจองทับช่วงวันก่อนบันทึกทุกครั้ง
3. เจ้าของสัตว์เลี้ยงติดตามความเป็นอยู่ของสัตว์ผ่านรายงานการดูแลรายวันได้
4. พนักงานและผู้ดูแลระบบทำงานได้เป็นระบบ ราคาคำนวณถูกต้องสม่ำเสมอ และดูสรุปรายได้ได้ทันที
5. คณะผู้จัดทำได้ฝึกประยุกต์ใช้หลักการออกแบบซอฟต์แวร์ การทดสอบ และการทำงานเป็นทีมกับโครงการจริง

1.5 นิยามศัพท์เฉพาะ

ตารางที่ 1.2 นิยามศัพท์เฉพาะ

คำศัพท์	ความหมาย
การจองทับช่วงวัน (Overlapping booking)	การจองห้องเดียวกันที่ช่วงวันซ้อนกัน คือ checkInเดิม < checkOutใหม่ และ checkOutเดิม > checkInใหม่ โดยนับเฉพาะสถานะ PENDING, CONFIRMED, CHECKED_IN

คำศัพท์	ความหมาย
สถานะการจอง (Booking status)	PENDING (รอยืนยัน), CONFIRMED (ยืนยันแล้ว), CHECKED_IN (เข้าพัก), CHECKED_OUT (สิ้นสุดการเข้าพัก), CANCELLED (ยกเลิก)
บริการเสริม (Extra service)	บริการที่เพิ่มจากห้องพัก เช่น อาบน้ำ ตัดขน คิตราค่าต่อหน่วย x จำนวน
รายงานการดูแล (Daily care report)	บันทึกการดูแลสัตว์เลี้ยงแต่ละตัวในแต่ละวันระหว่างเข้าพัก
Design Pattern	รูปแบบการแก้ปัญหาการออกแบบซอฟต์แวร์ที่เกิดขึ้นซ้ำ ๆ
SOLID	หลักการออกแบบเชิงวัตถุ 5 ข้อ: SRP, OCP, LSP, ISP, DIP
CI/CD	Continuous Integration / Continuous Deployment การทดสอบและติดตั้งใช้งานอัตโนมัติเมื่อมีการเปลี่ยนแปลงโค้ด
DTO	Data Transfer Object วัตถุสำหรับรับ-ส่งข้อมูลระหว่างชั้น แยกจาก Entity ของฐานข้อมูล

1.6 สมาชิกและการแบ่งหน้าที่

ตารางที่ 1.3 สมาชิกและหน้าที่รับผิดชอบ

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา	หน้าที่รับผิดชอบ
1	นายกิตติธัช คำโท	673380029-0	Room & Admin, User, Payment, Notification (Observer), Security, Flyway, Docker / Deploy
2	นายณัช พนมเรียงไชย	673380041-0	Availability & Room Search, REST API Error Handling, Login / Profile, CI
3	นางสาวรติมา สวัสดิ์นที	673380055-9	Pet & Daily Care Report, Test Report
4	นางสาวกิตติญาดา กองคำ	673380509-6	Booking & State Pattern
5	นางสาวนาเดีย คัดอ่าน	673380513-5	Pricing (Strategy), Promotion, Extra Service, Dashboard

1.7 แผนการดำเนินงาน

ตารางที่ 1.4 แผนการดำเนินงาน

ขั้นตอน	รายละเอียด	ช่วงเวลา
1.	กำหนดผู้ใช้ ฟังก์ชัน และขอบเขต	10-14 ก.ย. 2569

ขั้นตอน	รายละเอียด	ช่วงเวลา
ศึกษาปัญหาและรวบรวมความต้องการ		
2. วิเคราะห์และออกแบบระบบ	Use Case, Domain Model, Class Diagram, ER Diagram, Sequence/Activity/State Diagram	14–20 ก.ย. 2569 และปรับ diagram ให้ตรงกับโค้ด 3–8 ต.ค. 2569
3. พัฒนาระบบ	แบ่ง module ตามสมาชิก พัฒนabn branch ของตนและรวมผ่าน Pull Request เข้า develop	14 ก.ย. – 9 ต.ค. 2569
4. ทดสอบระบบ	Unit / Controller / Repository test, วัต coverage	22 ก.ย. – 9 ต.ค. 2569 (ทดสอบควบคู่กับการพัฒนา)
5. Deploy และ CI/CD	Docker, Render, Neon, GitHub Actions	3–5 ต.ค. 2569
6. จัดทำเอกสารและนำเสนอ	รายงาน, README, สไลด์, สาธิตระบบ	3–9 ต.ค. 2569

หมายเหตุ: ช่วงเวลาอ้างอิงจากวันที่ของ commit ใน repository ซึ่งเริ่มวันที่ 10 กันยายน 2569 และ merge เข้า main ครั้งล่าสุดวันที่ 9 ตุลาคม 2569

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

บทนี้อธิบายทฤษฎี หลักการออกแบบ และเทคโนโลยีที่ใช้ในการพัฒนาระบบ PetsHotel

2.1 สถาปัตยกรรมซอฟต์แวร์แบบชั้น (Layered Architecture)

Layered Architecture แบ่งระบบออกเป็นชั้นตามหน้าที่ โดยชั้นบนเรียกใช้ชั้นล่างเท่านั้น ระบบนี้แบ่งเป็น Presentation (Controller และ View), Service (Business logic) และ Repository (การเข้าถึงข้อมูล) ข้อดีคือแยกความรับผิดชอบชัดเจน เปลี่ยนแปลงชั้นหนึ่งได้โดยกระทบชั้นอื่นน้อย และทดสอบแต่ละชั้นแยกกันได้ ในโครงงานนี้กำหนดกติกาว่า Controller ต้องไม่เรียก Repository โดยตรง (Fowler, 2002)

2.2 รูปแบบ Model–View–Controller (MVC)

MVC แยกข้อมูล (Model) การแสดงผล (View) และการควบคุมการทำงาน (Controller) ออกจากกัน Spring Web MVC ใช้ DispatcherServlet รับคำขอ ส่งต่อให้ Controller ซึ่งเตรียม Model แล้วเลือก View (Thymeleaf template) สำหรับแสดงผล หรือส่ง JSON กลับในกรณีของ REST Controller

2.3 Design Patterns

Design Pattern คือแนวทางแก้ปัญหาการออกแบบที่พบบ่อยซึ่งได้รับการพิสูจน์แล้ว ตามที่ Gamma และคณะ (1994) จัดกลุ่มเป็น Creational, Structural และ Behavioral ระบบนี้ใช้ Behavioral Pattern 3 รูปแบบ

ตารางที่ 2.1 Design Pattern ที่ใช้ในระบบ

Pattern	แนวคิด	การใช้ในระบบ
State	ให้วัตถุเปลี่ยนพฤติกรรมตามสถานะภายใน โดยแยกแต่ละสถานะเป็นคลาส	สถานะการจอง PENDING → CONFIRMED → CHECKED_IN → CHECKED_OUT / CANCELLED
Strategy	ห่อหุ้มอัลกอริทึมแต่ละแบบเป็นคลาสที่สลับใช้แทนกันได้ผ่าน interface เดียวกัน	การคำนวณราคา 5 กลยุทธ์ผ่าน PricingStrategy
Observer	เมื่อวัตถุหนึ่งเปลี่ยนแปลง ผู้สังเกตที่ลงทะเบียนไว้จะได้รับแจ้งโดยไม่ผูก	BookingConfirmedEvent → BookingConfirmedListener

Pattern	แนวคิด	การใช้ในระบบ
	ติดกัน	ส่งอีเมลหลัง commit

นอกจากนี้ยังใช้ Enterprise Pattern ได้แก่ Repository, Service Layer, DTO + Mapper และ Dependency Injection (Fowler, 2002)

2.4 หลักการออกแบบ SOLID

ตารางที่ 2.2 หลักการ SOLID (Martin, 2003)

หลักการ	ความหมาย
S – Single Responsibility	คลาสหนึ่งควรมีเหตุผลให้เปลี่ยนแปลงเพียงเหตุผลเดียว
O – Open/Closed	เปิดให้ขยาย แต่ปิดไม่ให้แก้ไขโค้ดเดิม
L – Liskov Substitution	คลาสลูกต้องใช้แทนคลาสแม่ได้โดยพฤติกรรมไม่ผิดไปจากที่ผู้เรียกคาดหวัง
I – Interface Segregation	ไม่บังคับให้ผู้ใช้พึ่งพาเมธอดที่ไม่ได้ใช้ แยก interface ให้เล็กและเฉพาะทาง
D – Dependency Inversion	โมดูลระดับสูงและระดับต่ำพึ่งพา abstraction ไม่พึ่งพาคลาสจริงโดยตรง

2.5 ภาษาการจำลองแบบรวม (UML)

UML เป็นภาษาภาพมาตรฐานสำหรับอธิบายการออกแบบซอฟต์แวร์ โครงการนี้ใช้ Use Case Diagram, Domain Model, Class Diagram, Sequence Diagram, Activity Diagram, State Diagram, Component Diagram และ Deployment Diagram โดยเขียนด้วย PlantUML เพื่อเก็บไว้ใน Git ร่วมกับโค้ด

2.6 เทคโนโลยีที่ใช้พัฒนา

ตารางที่ 2.3 เทคโนโลยีที่ใช้ในการพัฒนา

ส่วน	เทคโนโลยี	หน้าที่ในระบบ
Language	Java 26	ภาษาหลักของระบบ
Framework	Spring Boot 4.1.1	Web MVC, Data JPA, Security, Validation, Mail
View	Thymeleaf + Thymeleaf Extras	สร้างหน้าเว็บฝั่งเซิร์ฟเวอร์

ส่วน	เทคโนโลยี	หน้าที่ในระบบ
	Spring Security	และซ่อน/แสดงเมนูตามบทบาท (sec:authorize)
Security	Spring Security	เข้าสู่ระบบแบบ session, BCrypt, CSRF (CookieCsrfTokenRepository), Remember-me, กำหนดสิทธิ์ตามบทบาท
ORM	Spring Data JPA / Hibernate	แมป Entity กับตาราง และสร้าง Repository จาก interface
Database	PostgreSQL	เก็บข้อมูลของระบบ
Migration	Flyway	ควบคุมเวอร์ชันโครงสร้างตาราง (V1-V3) โดย Hibernate ตั้ง ddl-auto=validate
API Docs	springdoc-openapi (Swagger UI)	สร้างเอกสาร REST API และทดลองเรียกได้จากหน้าเว็บ
Testing	JUnit 5, Mockito, Spring Boot Test, JaCoCo	ทดสอบอัตโนมัติและวัด code coverage
Build	Maven (Maven Wrapper)	จัดการ dependency และ build
Container	Docker, Docker Compose	สร้างอิมเมจแอป และรันพร้อม PostgreSQL และ Mailpit บนเครื่อง
Deployment	Render + Neon PostgreSQL	รันเว็บแอปบนคลาวด์ และให้บริการฐานข้อมูล
CI/CD	GitHub Actions	รัน test และ docker build ทุก push/PR และ deploy เมื่อ push เข้า main
Version control	Git / GitHub	แต่ละคนพัฒนาบน branch ของตน รวมผ่าน Pull Request เข้า develop และ main

2.7 REST API และ HTTP Status Code

REST (Representational State Transfer) เป็นรูปแบบการออกแบบ Web API ที่มองข้อมูลเป็นทรัพยากร (resource) ระบุด้วย URL และใช้ HTTP method ตามความหมาย ได้แก่ GET อ่าน, POST สร้าง, PUT แก้ไขทั้งหมด, PATCH แก้บางส่วน และ DELETE ลบ พร้อมตอบ HTTP status code ที่สื่อผลลัพธ์ เช่น 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found และ 409 Conflict (Fielding, 2000) เอกสาร API อธิบายด้วยมาตรฐาน OpenAPI

2.8 การทดสอบซอฟต์แวร์

การทดสอบหน่วย (Unit test) ตรวจสอบการทำงานของคลาสหนึ่งโดยแทน dependency ด้วย mock (Mockito) ส่วนการทดสอบ Controller ใช้ MockMvc จำลองคำขอ HTTP และการทดสอบ Repository ใช้ฐานข้อมูล PostgreSQL จริง Code coverage วัดสัดส่วนบรรทัด (Line) และเงื่อนไข (Branch) ที่ถูกทดสอบด้วย JaCoCo

2.9 Container และ CI/CD

Docker บรรจุแอปพลิเคชันพร้อมสภาพแวดล้อมเป็นอิมเมจ ทำให้รันได้เหมือนกันทุกเครื่อง Continuous Integration คือการ build และทดสอบอัตโนมัติทุกครั้งที่มีการเปลี่ยนแปลงโค้ด และ Continuous Deployment คือการนำเวอร์ชันที่ผ่านการทดสอบขึ้นระบบจริงโดยอัตโนมัติ

2.10 ระบบหรืองานที่เกี่ยวข้อง

ระบบจัดการโรงแรมสัตว์เลี้ยงเชิงพาณิชย์ที่ใช้งานจริง มีหลายระบบ คณะผู้จัดทำเลือกศึกษา 2 ระบบ ได้แก่ Gingr และ Revelation Pets โดย Gingr เป็นระบบบนคลาวด์ สำหรับธุรกิจรับฝากและดูแลสัตว์เลี้ยง มีปฏิทินแสดงห้องว่าง และห้องที่มีสัตว์เข้าพัก ให้ลูกค้าส่งคำจอง ผ่าน Client Portal เพื่อให้พนักงานอนุมัติ ส่ง Report Card การดูแล ให้เจ้าของทางอีเมลหรือ SMS และมี API สำหรับเชื่อมต่อระบบอื่น (Gingr, n.d.) ส่วน Revelation Pets เป็นระบบบนเว็บที่ให้ลูกค้าตรวจห้องว่าง และจองออนไลน์ได้ตลอด 24 ชั่วโมง มี Activity Page สำหรับบันทึกกิจกรรม การกิน และการพักผ่อน ของสัตว์รายวัน กำหนดส่วนลดได้ และมีรายงานการจอง การเข้าพัก และยอดขาย (Revelation Pets, n.d.) ทั้งสองระบบมีความสามารถครบกว่า ในด้านการชำระเงินออนไลน์ และการแจ้งเตือนหลายช่องทาง แต่เป็นบริการที่ต้องเสียค่าใช้จ่าย PetsHotel จึงมุ่งพัฒนาความสามารถหลักที่จำเป็นสำหรับโรงแรมขนาดเล็ก ได้แก่ การป้องกันการจองซ้อน การคำนวณราคาตามกฎหมายที่กำหนดเอง และรายงานการดูแลรายวัน ในรูปแบบที่ขยายต่อได้ตามหลักการออกแบบที่ดี

ตารางที่ 2.4 เปรียบเทียบระบบที่เกี่ยวข้อง

ความสามารถ	Gingr	Revelation Pets	PetsHotel
ค้นหาห้องว่างตามช่วงวัน	✓	✓	✓
จองออนไลน์พร้อมคำนวณราคา	✓	✓	✓

ความสามารถ	Gingr	Revelation Pets	PetsHotel
	(พนักงานอนุมัติคำ ขออนุญาต)		
รายงานการดูแลสัตว์รายวัน	✓	✓	✓
จัดการโปรโมชั่น / บริการเสริม	ไม่พบข้อมูล	✓ (ส่วนลด)	✓
Dashboard รายได้	✓	✓	✓
REST API	✓	✓ (ผ่านการเชื่อมต่อ ระบบ)	✓

บทที่ 3

การวิเคราะห์และออกแบบระบบ

3.1 การวิเคราะห์ความต้องการ

3.1.1 ความต้องการด้านฟังก์ชัน (Functional Requirements)

ตารางที่ 3.1 ความต้องการด้านฟังก์ชัน

รหัส	ความต้องการ
FR-01	ผู้เยี่ยมชมสมัครสมาชิกได้ โดยอีเมลต้องไม่ซ้ำ และรหัสผ่านถูกเข้ารหัสด้วย BCrypt
FR-02	ผู้ใช้เข้าสู่ระบบได้ ระบบพาไปหน้าตามบทบาท (ADMIN → /admin, STAFF → /admin/bookings, CUSTOMER → หน้าแรก) หรือกลับไปหน้าที่ขอไว้ก่อนเข้าสู่ระบบ
FR-03	ผู้ใช้ค้นหาห้องพักตามวันเช็คอิน วันเช็คเอาท์ และจำนวนสัตว์ได้ โดยไม่ต้องเข้าสู่ระบบ
FR-04	ลูกค้าเพิ่ม แก้ไข และปิดใช้งานข้อมูลสัตว์เลี้ยงได้
FR-05	ลูกค้าจองห้องพัก เลือกสัตว์ บริการเสริม และโปรโมชั่น และเห็นราคาก่อนยืนยัน
FR-06	ระบบป้องกันการจองห้องพักช่วงวัน และสัตว์ตัวเดียวกันถูกจองซ้อนช่วงวัน
FR-07	พนักงานยืนยัน เช็คอิน เช็คเอาท์ และยกเลิกการจองตามลำดับสถานะที่ถูกต้อง
FR-08	ระบบส่งอีเมลยืนยันถึงลูกค้าเมื่อการจองได้รับการยืนยัน
FR-09	พนักงานยืนยันรับชำระเงิน และลูกค้าดูใบเสร็จได้
FR-10	พนักงานบันทึกรายงานการดูแลรายวัน และลูกค้าดูรายงานของสัตว์ตนเองได้
FR-11	ผู้ดูแลระบบจัดการห้องพัก โปรโมชั่น บริการเสริม และสมาชิกได้
FR-12	ผู้ดูแลระบบดู Dashboard รายได้ได้
FR-13	ระบบมี REST API พร้อมเอกสาร Swagger ตอบ status code และรูปแบบ error ที่สม่ำเสมอ

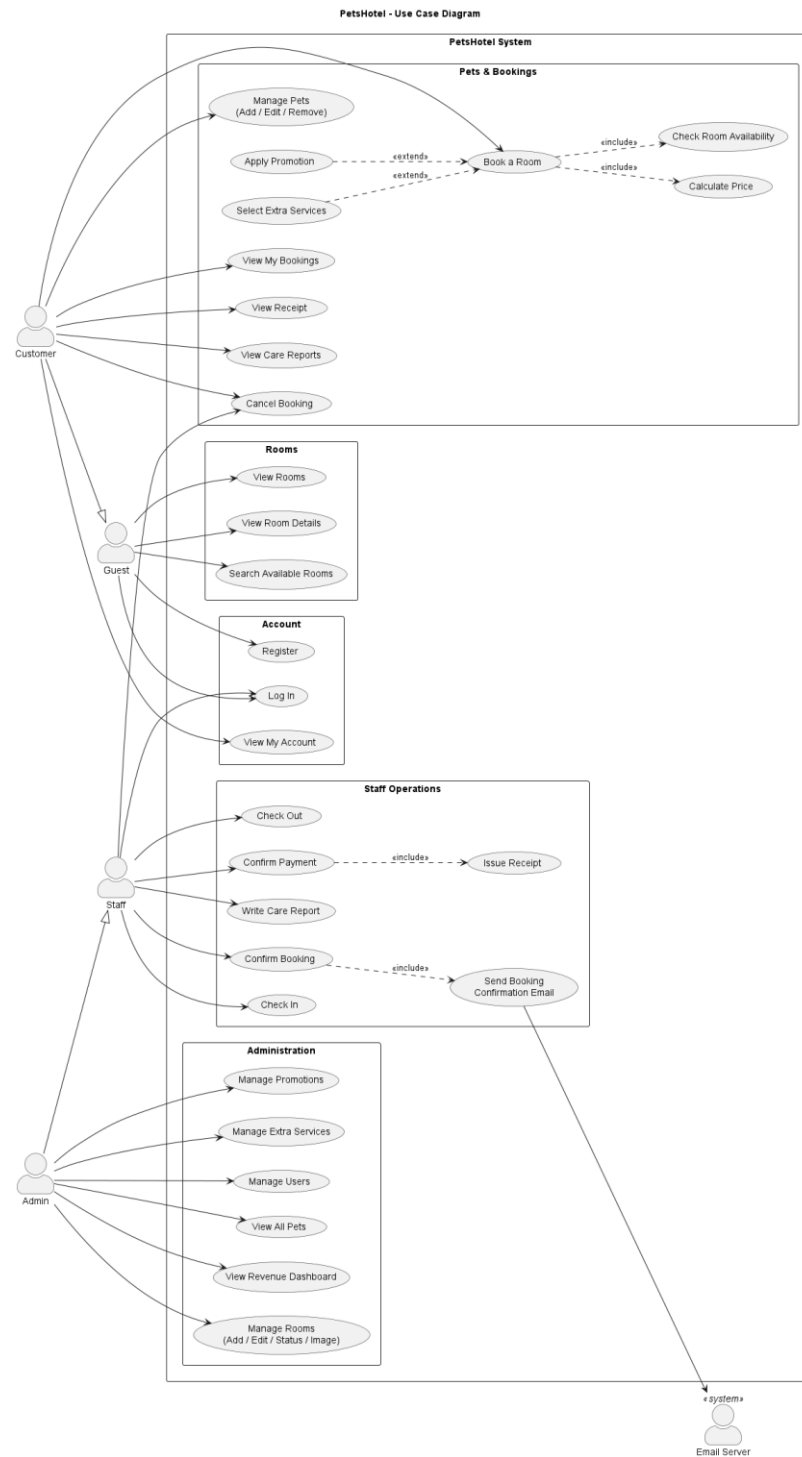
3.1.2 ความต้องการที่ไม่ใช่ฟังก์ชัน (Non-functional Requirements)

- ความปลอดภัย: รหัสผ่านเข้ารหัส BCrypt, ป้องกัน CSRF, ตรวจสอบสิทธิ์ตามบทบาททั้งหน้าเว็บและ API, ผู้ใช้เข้าถึงได้เฉพาะข้อมูลของตน
- ความถูกต้องของข้อมูล: ตรวจสอบความถูกต้องด้วย Bean Validation และตรวจกฎธุรกิจใน Service, ใช้ transaction และล็อกข้อมูลการจองเมื่อเปลี่ยนสถานะ
- การบำรุงรักษา: แบ่งชั้นชัดเจน ใช้ interface และ Dependency Injection มีเอกสาร diagram และ data dictionary
- การทดสอบ: มี test อัตโนมัติครอบคลุม Line coverage ไม่ต่ำกว่า 80%

- การใช้งาน: หน้าเว็บภาษาไทย แจกข้อผิดพลาดที่เข้าใจง่าย รองรับการใช้งานผ่าน browser
- การติดตั้ง: รันได้ด้วยคำสั่งเดียวผ่าน Docker Compose และ deploy อัตโนมัติผ่าน CI/CD
- ความลับ: รหัสผ่านฐานข้อมูลและคีย์ต่าง ๆ เก็บใน environment variable ไม่เก็บใน repository

3.2 แผนภาพ Use Case

ระบบมีผู้กระทำ (Actor) ได้แก่ ผู้เยี่ยมชม ลูกค้า พนักงาน และผู้ดูแลระบบ มี Use Case ทั้งหมด 16 รายการ ดังรูปที่ 3.1 และตารางรายการ Use Case ด้านล่างรายละเอียดของแต่ละ Use Case แสดงในภาคผนวก ก



รูปที่ 3.1 Use Case Diagram ของระบบ PetsHotel

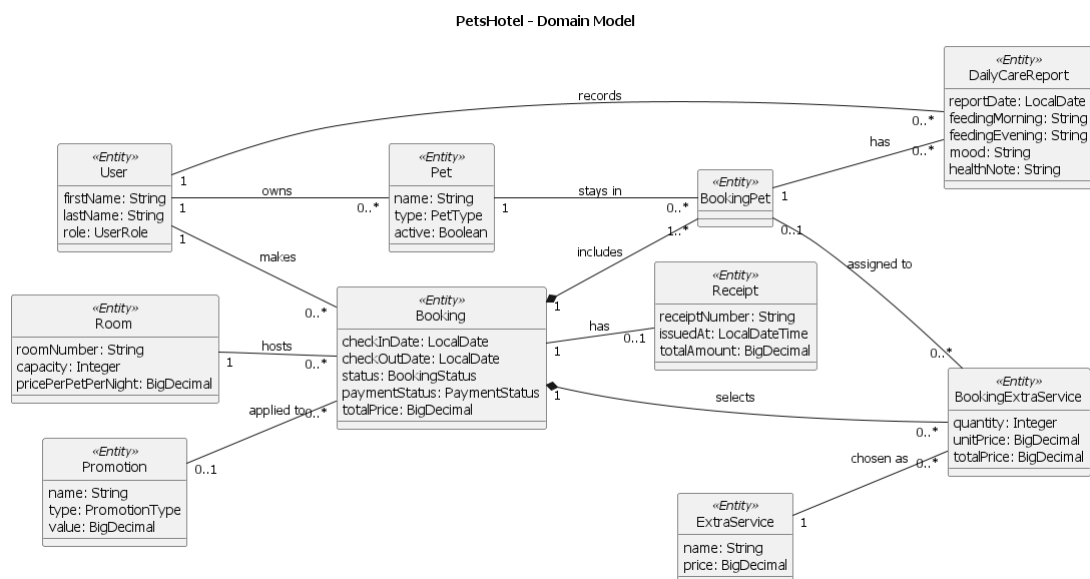
รายการ Use Case

รหัส	Use Case	Actor หลัก	ผู้เขียน
UC-01	สมัครสมาชิก	ผู้เยี่ยมชม	กิตติธัช
UC-02	เข้าสู่ระบบ	ทุก role	กิตติธัช

รหัส	Use Case	Actor หลัก	ผู้เขียน
UC-03	ค้นหาห้องพัก	ผู้เยี่ยมชม, ลูกค้า	ธนัช
UC-04	จัดการสัตว์เลี้ยง	ลูกค้า	รติมา
UC-05	จองห้องพัก	ลูกค้า	กิติญาดา
UC-06	ยกเลิกการจอง	ลูกค้า, พนักงาน	กิติญาดา
UC-07	ยืนยันการจอง	พนักงาน	กิตติธัช
UC-08	เช็คอิน / เช็คเอาท์	พนักงาน	กิติญาดา
UC-09	ยืนยันรับชำระเงิน	พนักงาน	กิตติธัช
UC-10	บันทึกรายงานการดูแล	พนักงาน	รติมา
UC-11	ดูรายงานการดูแล	ลูกค้า	รติมา
UC-12	จัดการห้องพัก	ผู้ดูแลระบบ	กิตติธัช
UC-13	จัดการโปรโมชั่น	ผู้ดูแลระบบ	นาเดีย
UC-14	จัดการบริการเสริม	ผู้ดูแลระบบ	นาเดีย
UC-15	จัดการสมาชิก	ผู้ดูแลระบบ	กิตติธัช
UC-16	ดู Dashboard รายได้	ผู้ดูแลระบบ	นาเดีย

3.3 แบบจำลองโดเมน (Domain Model)

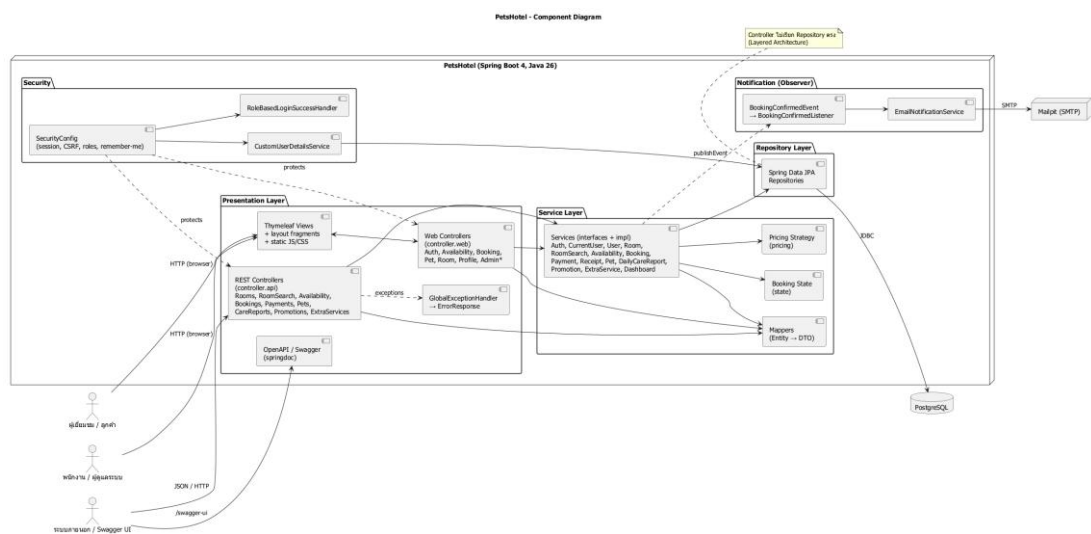
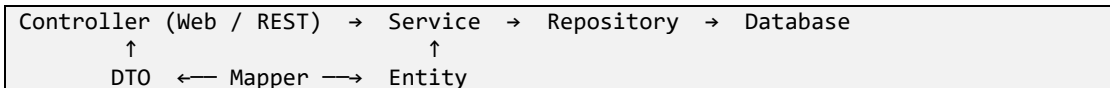
แบบจำลองโดเมนแสดงแนวคิดหลักของระบบและความสัมพันธ์ ได้แก่ ผู้ใช้ สัตว์เลี้ยง หอพัก การจอง สัตว์เลี้ยงในการจอง บริการเสริม โปรโมชั่น ใบเสร็จ และรายงานการดูแล



รูปที่ 3.2 Domain Model

3.4 สถาปัตยกรรมระบบ

ระบบใช้ Layered Architecture โดย Controller แยกเป็น controller/web สำหรับหน้าเว็บ Thymeleaf และ controller/api สำหรับ REST API ทั้งสองชั้นเรียกใช้ Service ผ่าน interface ชุดเดียวกัน Service เรียก Repository ของ Spring Data JPA และใช้ Mapper แปลง Entity เป็น DTO



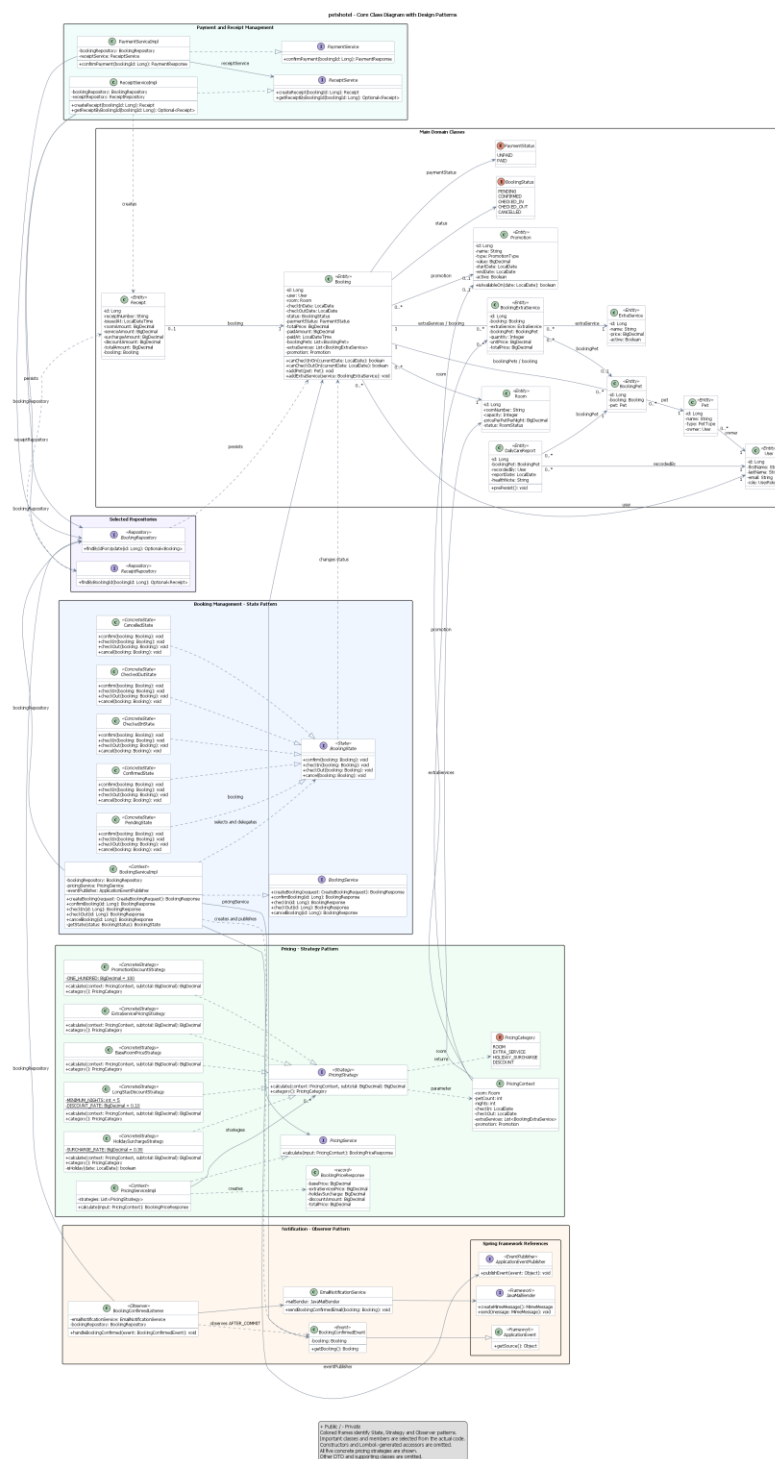
รูปที่ 3.3 Component Diagram

ตารางที่ 3.2 โครงสร้าง package ของระบบ

Package	หน้าที่
config	Security, Swagger (OpenAPI), Web config, RoleBasedLoginSuccessHandler
controller/api	REST controllers ตอบ JSON
controller/web	Web controllers ส่งข้อมูลให้ Thymeleaf
domain	Entity และ Enum
dto	Request / Response DTO
exception	Custom exceptions และ GlobalExceptionHandler
mapper	แปลง Entity ↔ DTO
notification	Observer: event, listener, email
pricing	Strategy: การคำนวณราคา
repository	Spring Data JPA repositories

Package	หน้าที่
service	Business logic (interface + impl)
state	State: สถานะการจอง

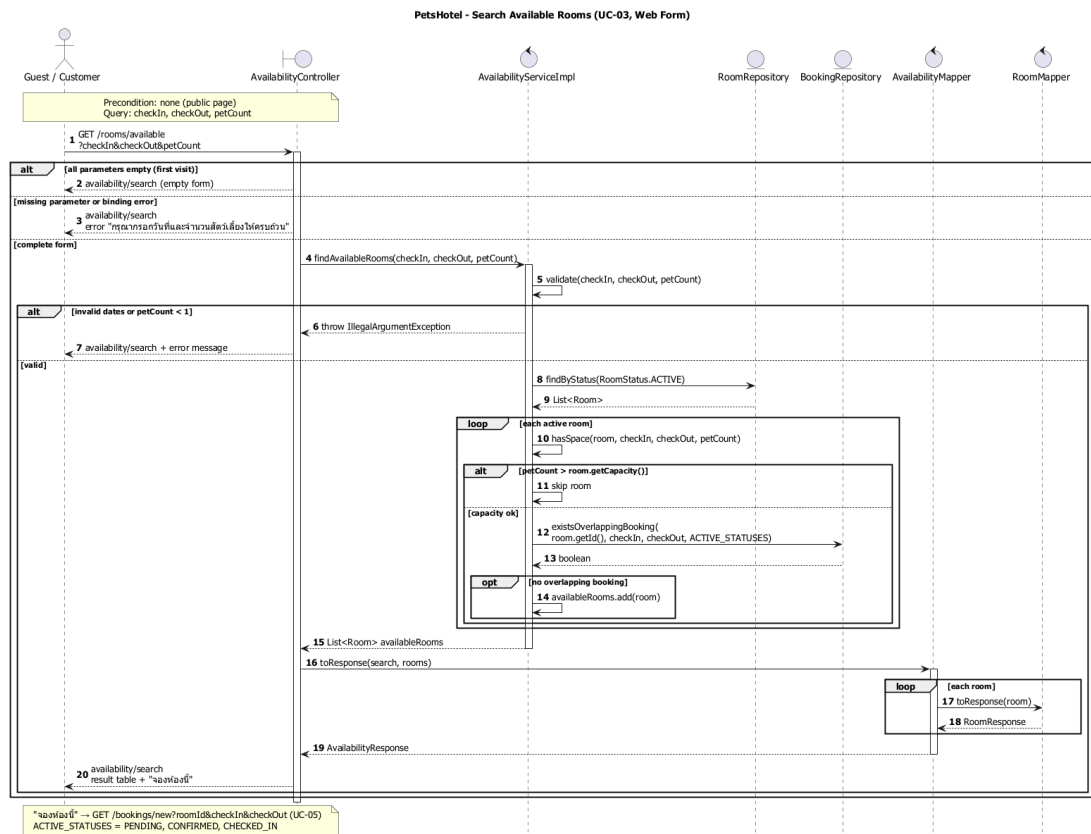
3.5 แผนภาพคลาส (Class Diagram)



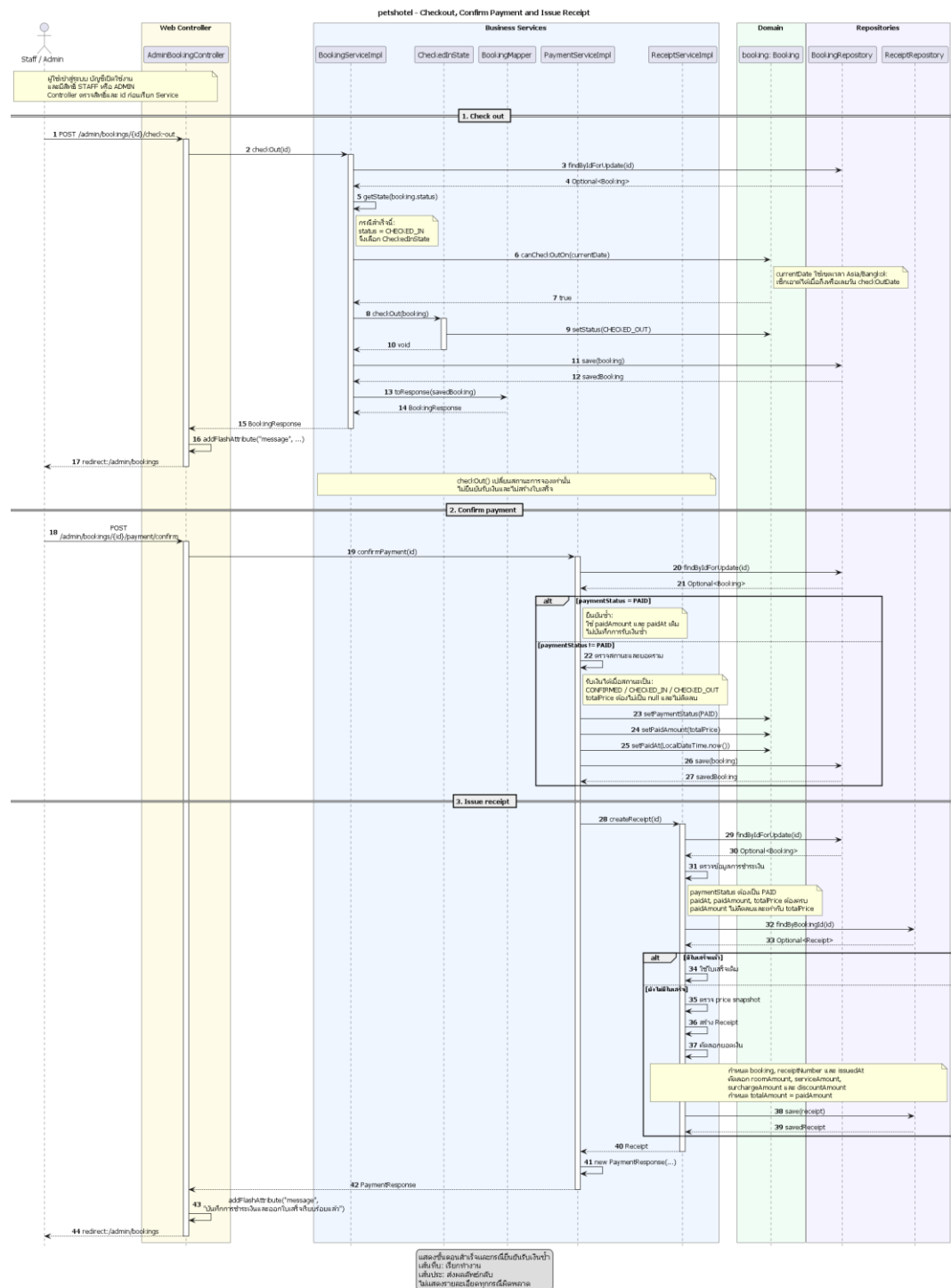
รูปที่ 3.4 Class Diagram

3.6 แผนภาพลำดับ (Sequence Diagram)

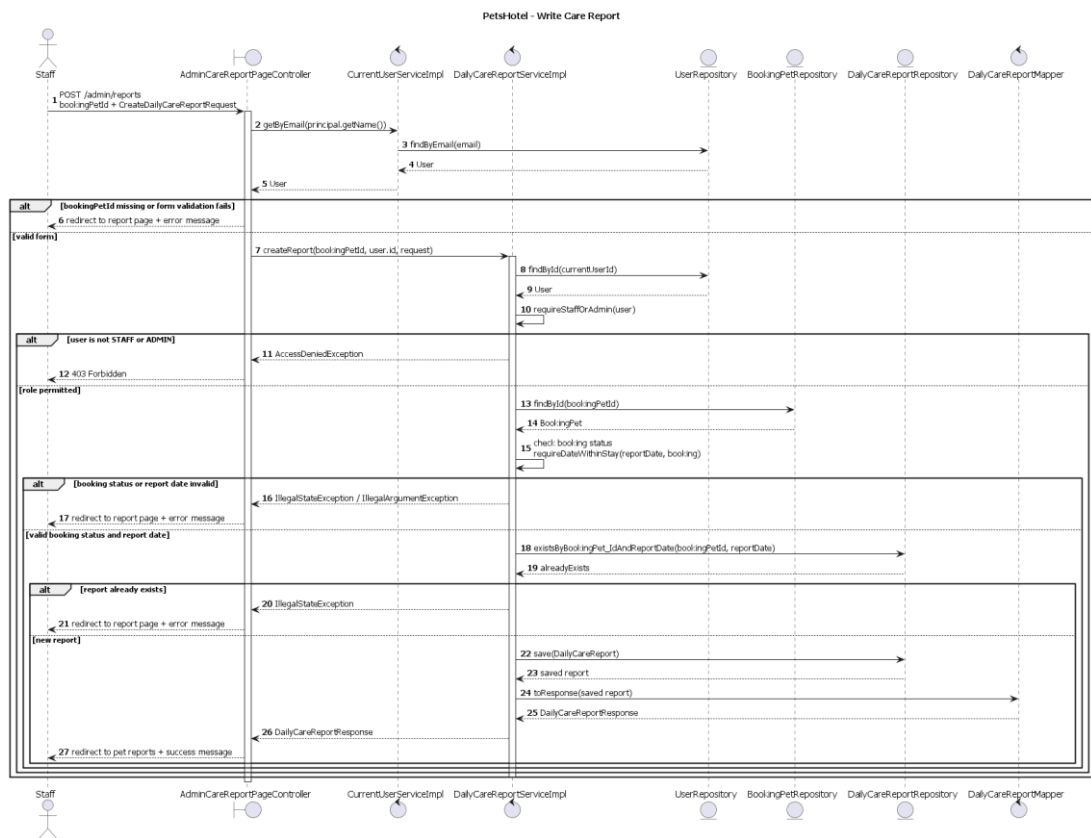
แผนภาพลำดับแสดงการทำงานระหว่างวัตถุใน Use Case สำคัญ 4 กรณี



รูปที่ 3.5 Sequence Diagram: ค้นหาห้องว่าง (UC-03)

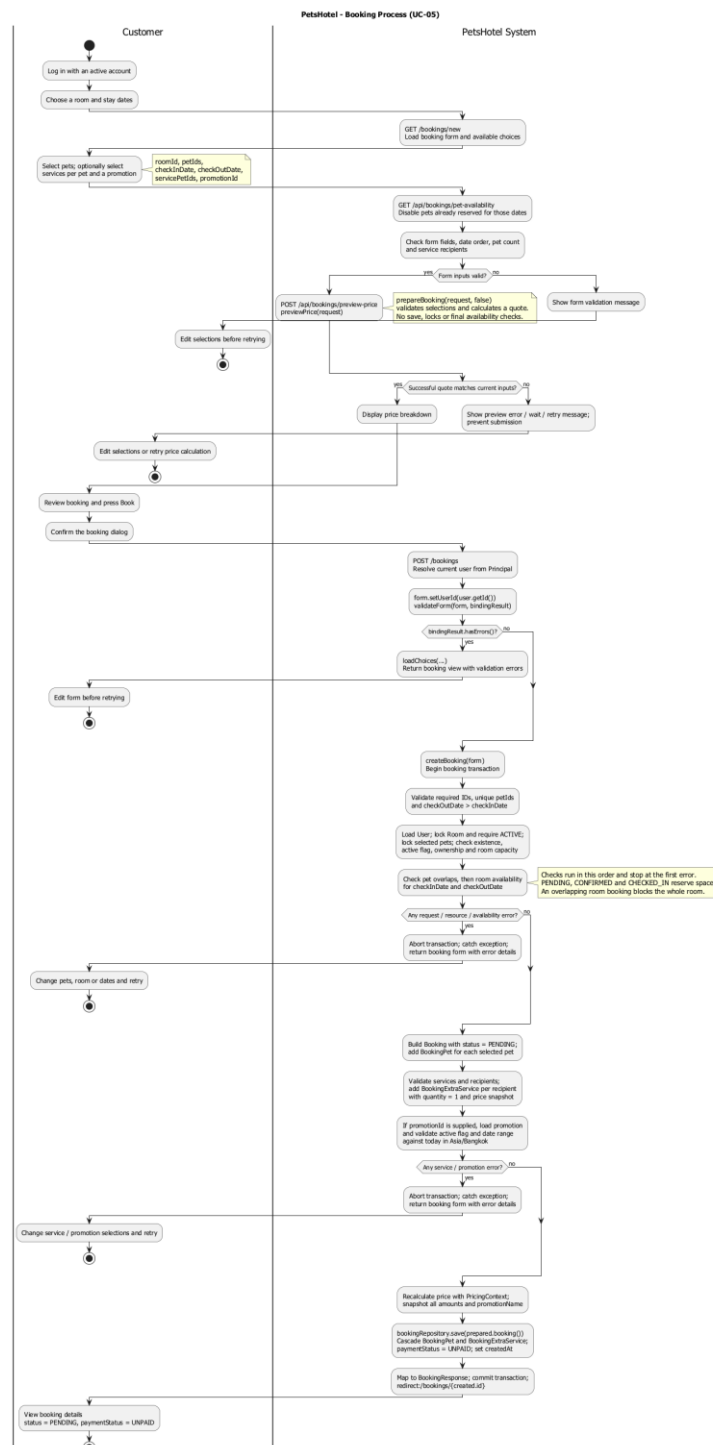


รูปที่ 3.7 Sequence Diagram: เช็คเอาท์ (UC-08)

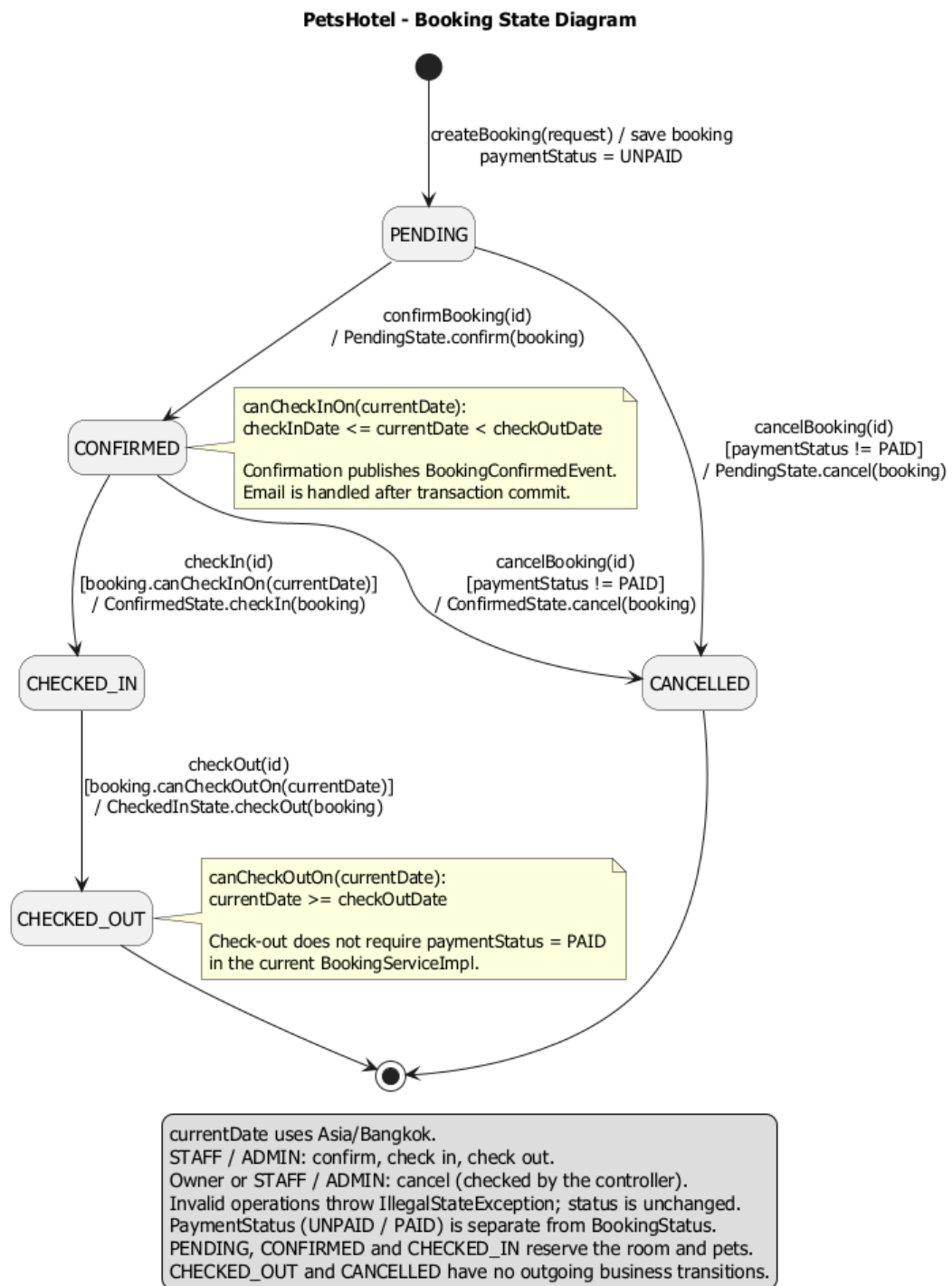


รูปที่ 3.8 Sequence Diagram: บันทึกการรายงานการดูแล (UC-10)

3.7 แผนภาพกิจกรรมและแผนภาพสถานะ



รูปที่ 3.9 Activity Diagram: กระบวนการจอง



รูปที่ 3.10 State Diagram: สถานะการจอง

3.8 กฎทางธุรกิจที่สำคัญ

3.8.1 การตรวจห้องว่าง

AvailabilityServiceImpl.hasSpace() ถือว่าห้องว่างเมื่อครบ 3 เงื่อนไข คือ (1) ห้องมีสถานะ ACTIVE (2) จำนวนสัตว์ไม่เกินความจุของห้อง และ (3) ไม่มีการจองที่สถานะ PENDING, CONFIRMED หรือ CHECKED_IN ทับช่วงวัน โดยเงื่อนไขทับช่วงวันคือ `checkInDate < checkOut AND checkOutDate > checkIn` ทำให้วันเช็คอินของการจองหนึ่งเป็นวันเช็คอินของการจองถัดไปได้

นอกจากนี้ระบบตรวจว่าวันเช็คอินต้องหลังวันเช็คอิน วันเช็คอินต้องไม่เป็นอดีต

จำนวนสัตว์อย่างน้อย 1 ตัว และปฏิทินห้องว่างดูได้ไม่เกิน 366 วัน

3.8.2 การคำนวณราคา

ตารางที่ 3.3 สูตรการคำนวณราคา

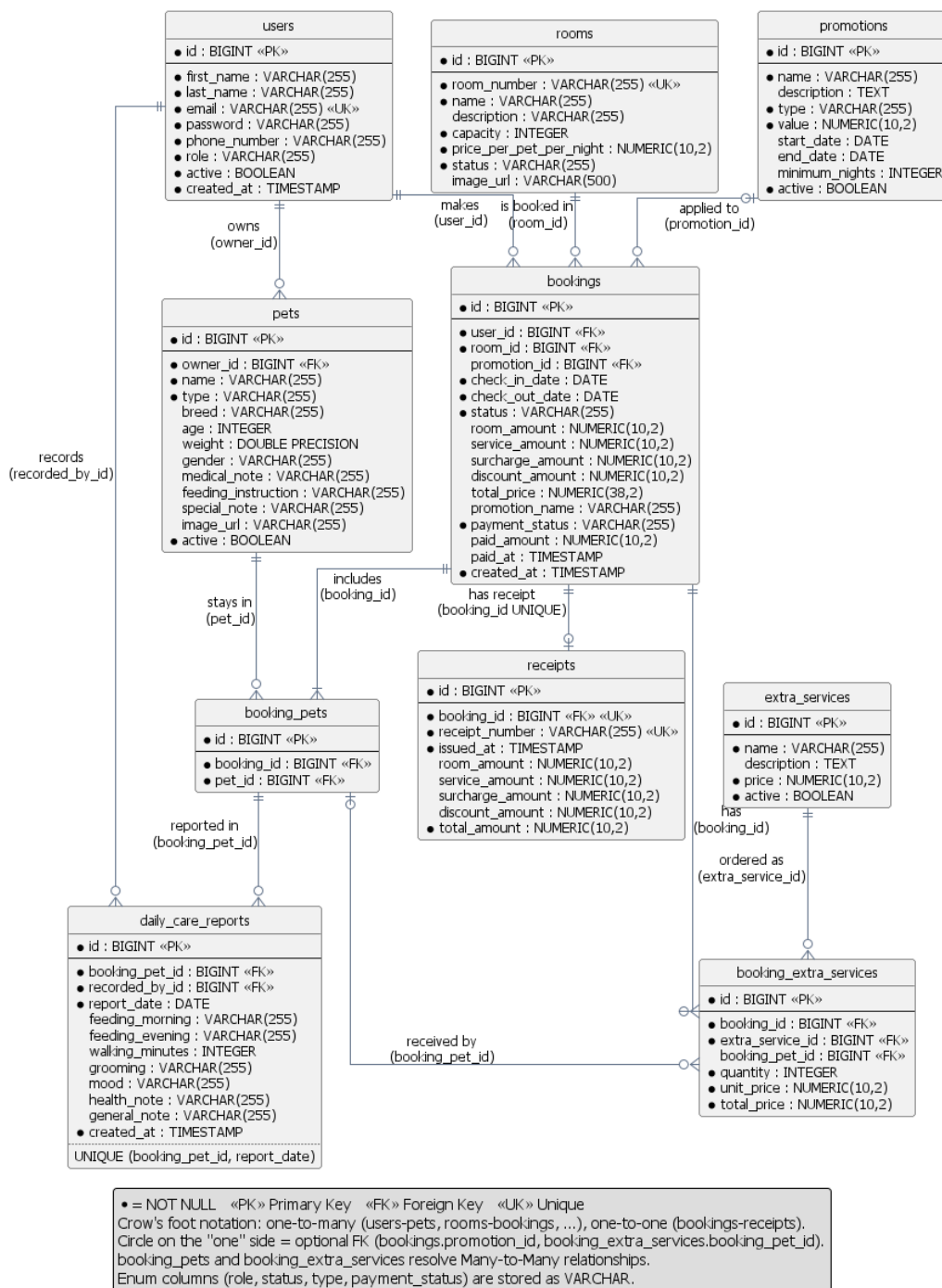
รายการ	สูตร
ค่าห้อง	ราคาต่อสัตว์ต่อคืน × จำนวนคืน × จำนวนสัตว์
บริการเสริม	ผลรวมของ unitPrice × quantity
ค่าธรรมเนียมวันหยุด	ราคาห้องต่อคืนรวมสัตว์ทุกตัว × 30% × จำนวนคืนที่ตรงวันหยุด (1 ม.ค., 13-15 เม.ย., 31 ธ.ค.)
ส่วนลดพักยาว	พักอย่างน้อย 5 คืน ลด 10% ของยอดก่อนหักส่วนลด
ส่วนลดโปรโมชั่น	ลดเป็นเปอร์เซ็นต์หรือจำนวนเงิน ตามเงื่อนไขจำนวนคืนขั้นต่ำ
ยอดสุทธิ	ค่าห้อง + บริการเสริม + ค่าธรรมเนียม - ส่วนลดที่มากที่สุดเพียงรายการเดียว (ไม่ติดลบ ปัดทศนิยม 2 ตำแหน่ง HALF_UP)

3.9 การออกแบบฐานข้อมูล

ฐานข้อมูลมีทั้งหมด 10 ตาราง สร้างด้วย Flyway migration (V1 สร้างตาราง, V2 ข้อมูลตัวอย่าง, V3 เพิ่มคอลัมน์ receipts.customer_name) และ Hibernate ตั้งเป็น `ddl-auto=validate` เพื่อตรวจว่า Entity ตรงกับตารางเท่านั้น

พจนานุกรมข้อมูลฉบับเต็มอยู่ในภาคผนวก ข

PetsHotel - ER Diagram



รูปที่ 3.11 ER Diagram

ตารางที่ 3.4 ตารางในฐานข้อมูล

ตาราง	คำอธิบาย
users	ผู้ใช้ระบบทุกบทบาท (CUSTOMER, STAFF, ADMIN) และสถานะใช้งาน

ตาราง	คำอธิบาย
pets	สัตว์เลี้ยงของลูกค้า
rooms	ห้องพัก ความจุ ราคาต่อสัตว์ต่อคืน และสถานะ
bookings	การจอง ช่วงวัน สถานะ ราคา ณ เวลาจอง และการชำระเงิน
booking_pets	สัตว์เลี้ยงที่อยู่ในการจอง
extra_services	บริการเสริม
booking_extra_services	บริการเสริมที่เลือกในการจอง พร้อมราคา ณ เวลาจอง
promotions	โปรโมชั่นและเงื่อนไข
receipts	ใบเสร็จของการจอง (1 การจอง : 1 ใบเสร็จ)
daily_care_reports	รายงานการดูแลรายวันของสัตว์เลี้ยงในการจอง

3.10 การออกแบบด้วย Design Pattern

สรุป

Pattern	กลุ่ม	ปัญหาที่แก้	ไฟล์ / คลาสหลัก	ผู้รับผิดชอบ
State	GoF Behavioral	แยกพฤติกรรมยืนยัน เช็กอิน เช็กเอาท์ และยกเลิกตามสถานะการ จอง เพื่อป้องกันการเปลี่ยนสถานะ ที่ผิดพลาด	state/ service/impl/BookingServiceImpl.java , domain/entity/Booking.java	กิตติภาดา
Strategy	GoF Behavioral	แยกกฎคำนวณค่าห้อง บริการเสริม ค่าธรรมเนียมวันหยุด และส่วนลด เพื่อเพิ่มหรือแก้กฎแต่ละแบบ ได้แยกกัน	pricing/ service/impl/PricingServiceImpl.java	นาเดีย
Observer	GoF Behavioral	แยกการแจ้งเตือนลูกค้าออกจาก logic ยืนยันการจอง และส่งอีเมลเฉพาะเมื่อบันทึกสำเร็จจริง	notification/	กิตติดิษฐ์
Layered / MVC / Repository / Service	Enterprise	แบ่งหน้าที่รับคำขอ แสดงผล ตรวจสอบธุรกิจ	controller/ service/	ทั้งทีม

Pattern	กลุ่ม	ปัญหาที่แก้	ไฟล์ / คลาสหลัก	ผู้รับผิดชอบ
Layer / DTO + Mapper / DI		เข้าถึงฐานข้อมูล และแปลงข้อมูล พร้อมให้ Spring จัดการ dependency เพื่อใช้ logic ร่วมกันและทดสอบแต่ละส่วนได้	repository/, domain/, dto/, mapper/, resources/templates/	

State Pattern

ปัญหาที่แก้

การจองใน PetsHotel มีสถานะ **PENDING, CONFIRMED, CHECKED_IN, CHECKED_OUT** และ **CANCELLED** ซึ่งอนุญาตให้ทำรายการต่างกัน เช่น การจองที่รอกยืนยันยังเช็คอินไม่ได้ การจองที่เช็คอินแล้วต้องเช็คเอาท์ก่อนจบการเข้าพัก และการจองที่ถูกยกเลิกไม่สามารถกลับมายืนยันได้

หากตรวจสอบทุกสถานะด้วย **if** หรือ **switch** ซ้ำในเมธอดยืนยัน เช็คอิน เช็คเอาท์ และยกเลิก กฎของสถานะเดียวกันจะกระจายหลายจุด ทำให้แก้ไขยากและเสี่ยงอนุญาตให้เปลี่ยนสถานะผิดพลาด

วิธีแก้ด้วย State

กำหนด interface **BookingState** ที่มีเมธอด **confirm(booking)**, **checkIn(booking)**, **checkOut(booking)** และ **cancel(booking)** แล้วให้แต่ละ Concrete State กำหนดว่ารายการใดทำได้ หากทำได้จะเปลี่ยน **booking.status** หากทำไม่ได้จะโยน **IllegalStateException**

BookingServiceImpl ทำหน้าที่ Context โดยโหลดการจอง เลือก State ผ่าน **getState(booking.getStatus())** ตรวจสอบเงื่อนไขเพิ่มเติม และเรียกพฤติกรรมผ่าน interface เดียวกัน ก่อนบันทึกผลลงฐานข้อมูล

บทบาทใน Pattern	คลาสในระบบ	ไฟล์
Context: เลือก State และประสานการทำรายการ	BookingServiceImpl	service/impl/BookingServiceImpl.java
State: สัญญาของการเปลี่ยนสถานะ	BookingState	state/BookingState.java

บทบาทใน Pattern	คลาสในระบบ	ไฟล์
Concrete State: รอยืนยัน	PendingState	state/PendingState.java
Concrete State: ยืนยันแล้ว	ConfirmedState	state/ConfirmedState.java
Concrete State: เช็กอินแล้ว	CheckedInState	state/CheckedInState.java
Concrete State: เช็กเอาท์แล้ว	CheckedOutState	state/CheckedOutState.java
Concrete State: ยกเลิกแล้ว	CancelledState	state/CancelledState.java
ข้อมูลที่ State เปลี่ยน	Booking	domain/entity/Booking.java
ค่าสถานะที่บันทึกในฐานข้อมูล	BookingStatus	domain/enums/BookingStatus.java

พาธ Java ในตารางอ้างอิงจาก

code/src/main/java/com/example/petshotel/

รายการที่แต่ละ State อนุญาต

สถานะปัจจุบัน / Concrete State	confirm()	checkIn()	checkOut()	cancel()
PENDING / PendingState	→ CONFIRMED	ไม่อนุญาต	ไม่อนุญาต	→ CANCELLED
CONFIRMED / ConfirmedState	ไม่อนุญาต	→ CHECKED_IN	ไม่อนุญาต	→ CANCELLED
CHECKED_IN / CheckedInState	ไม่อนุญาต	ไม่อนุญาต	→ CHECKED_OUT	ไม่อนุญาต
CHECKED_OUT / CheckedOutState	ไม่อนุญาต	ไม่อนุญาต	ไม่อนุญาต	ไม่อนุญาต
CANCELLED / CancelledState	ไม่อนุญาต	ไม่อนุญาต	ไม่อนุญาต	ไม่อนุญาต

ตารางนี้แสดงพฤติกรรมของคลาส State การทำรายการผ่าน Service ต้องผ่านเงื่อนไขวันที่ การชำระเงิน และสิทธิ์ผู้ใช้ที่อธิบายด้านล่างด้วย

รายละเอียดที่ออกแบบไว้

- เลือก State จากข้อมูลที่บันทึกจริง getState(status) ใช้ switch จับคู่

BookingStatus กับ Concrete State และสร้างด้วย new

ทุกครั้งที่เรียกทำรายการ Booking เก็บ enum ในฐานข้อมูล ไม่ได้เก็บ object ของ

BookingState เป็น field

- State รับผิดชอบการเปลี่ยนสถานะ เช่น `PendingState.confirm()` เรียก `booking.setStatus(BookingStatus.CONFIRMED)` ส่วนการโหลดบันทึก และประกาศเหตุการณ์อยู่ใน `BookingServiceImpl`
- ตรวจสอบวันที่ยกเลิกก่อนเปลี่ยนสถานะ `checkIn()` เรียก `Booking.canCheckInOn(currentDate)` เมื่อเป็น `CONFIRMED` และ `checkOut()` เรียก `Booking.canCheckOutOn(currentDate)` เมื่อเป็น `CHECKED_IN` โดยใช้วันปัจจุบันในเขตเวลา `Asia/Bangkok`
- เช็กอินได้ภายในช่วงเข้าพัก ต้องเป็น `CONFIRMED` และ `checkInDate <= currentDate < checkOutDate` จึงเช็กอินก่อนกำหนดหรือในวันเช็กเอาต์ไม่ได้
- เช็กเอาต์ได้ตั้งแต่วันที่ออกที่กำหนด ต้องเป็น `CHECKED_IN` และ `currentDate >= checkOutDate` โค้ดปัจจุบันไม่ได้บังคับให้ `paymentStatus` เป็น `PAID` ก่อนเช็กเอาต์
- ป้องกันยกเลิกหลังชำระเงิน `cancelBooking()` ตรวจสอบ `paymentStatus == PAID` แล้วปฏิเสธก่อนเรียก `state.cancel()` จึงยกเลิกได้เฉพาะ `PENDING` หรือ `CONFIRMED` ที่ยังไม่ชำระเงิน
- ตรวจสอบสิทธิ์ก่อนเรียก `Service Controller` จำกัดการยืนยัน เช็กอิน และเช็กเอาต์ให้ `STAFF / ADMIN` ลูกค้ายกเลิกได้เฉพาะการจองของตนเอง ส่วนพนักงานหรือผู้ดูแลยกเลิกผ่านช่องทางที่มีสิทธิ์ได้ `State` ไม่รับผิดชอบการตรวจบัญชีหรือ `role`
- ใช้ `transaction` และล็อกการจองระหว่างแก้ไข เมธอดเปลี่ยนสถานะโหลดผ่าน `findBookingForUpdate()` ซึ่งเรียก `BookingRepository.findByIdForUpdate()` ที่ใช้ `PESSIMISTIC_WRITE` เพื่อไม่ให้สอง `transaction` เปลี่ยนการจองเดียวกันพร้อมกัน
- รายการที่ไม่อนุญาตไม่บันทึกสถานะใหม่ เมื่อ `State` หรือเงื่อนไขใน `Service` โยน `IllegalStateException` จะไม่เรียก `save()` ในเส้นทางนั้น
เว็บแสดงข้อความผิดพลาด ส่วน `REST API` แปลงเป็น `409 Conflict` ผ่าน `GlobalExceptionHandler`

- ทำงานร่วมกับ Observer หลัง `confirm()` เปลี่ยนเป็น **CONFIRMED** และบันทึกแล้ว Service ประกาศ **BookingConfirmedEvent** การส่งอีเมลเกิดหลัง transaction commit ตามรายละเอียดในหัวข้อ Observer
- แยกสถานะการจองกับการชำระเงิน การจองใหม่มี **status = PENDING** และ **paymentStatus = UNPAID** การเปลี่ยน **BookingStatus** ผ่าน State ไม่ได้เปลี่ยน **PaymentStatus** โดยอัตโนมัติ

Class Diagram และ State Diagram

ดูรูปที่ 3.4 Class Diagram ของระบบ

กรอบ **Booking Management - State Pattern** แสดง

BookingServiceImpl ในบทบาท Context, interface **BookingState** และ Concrete State ทั้งห้าคลาสที่ implements interface เดียวกัน

State Diagram ของสถานะการจองแสดงในรูปที่ 3.10

ตัวอย่างโค้ดและลำดับการทำงาน

ตัวอย่างจาก **BookingServiceImpl.confirmBooking()**:

```
Booking booking = findBookingForUpdate(id);
BookingState state = getState(booking.getStatus());

state.confirm(booking);

Booking savedBooking = bookingRepository.save(booking);
eventPublisher.publishEvent(
    new BookingConfirmedEvent(this, savedBooking));

return bookingMapper.toResponse(savedBooking);
```

1. พนักงานหรือผู้ดูแลส่งคำขอยืนยัน Controller ตรวจสอบสิทธิ์แล้วเรียก **confirmBooking(id)**
2. Service โหลดและล็อกการจอง ถ้าไม่พบจะโยน **ResourceNotFoundException**
3. **getState(booking.getStatus())** เลือกคลาสตามสถานะ เช่น **PENDING** จะได้ **PendingState**
4. **state.confirm(booking)** เปลี่ยนเป็น **CONFIRMED** หากเป็น State ที่ไม่อนุญาต จะโยน **IllegalStateException** และหยุดก่อนบันทึกหรือประกาศเหตุการณ์
5. Service บันทึกการจอง ประกาศ **BookingConfirmedEvent** และแปลงผลด้วย **BookingMapper**

6. เมื่อ transaction commit สำเร็จ Listener ใน Observer Pattern
จึงรับเหตุการณ์เพื่อส่งอีเมลยืนยัน

ประโยชน์และขอบเขตของการออกแบบ

กฎของแต่ละสถานะรวมอยู่ในคลาสเดียว ทำให้ตรวจสอบเส้นทางที่อนุญาตและปฏิเสธได้ชัดเจน
การแก้พฤติกรรมของสถานะเดิมทำใน Concrete State นั้นได้ โดยผู้เรียกยังใช้เมธอดของ

BookingService เหมือนเดิม

การเพิ่มสถานะใหม่ยังต้องเพิ่มค่าใน **BookingStatus**, สร้าง Concrete State และปรับ
getState() รวมถึงกฎวันที่ การตรวจห้อง/สัปดาห์ และส่วนแสดงผลที่อ้างอิงสถานะ
จึงไม่ได้หมายความว่าเพิ่มสถานะใหม่ได้โดยไม่แก้ไขโค้ดเดิมเลย

ชุดทดสอบที่มีอยู่ใน **BookingServiceTest.java** มีตัวอย่างตรวจการยืนยัน **PENDING**,
การปฏิเสธยืนยัน **CANCELLED**, การยกเลิก **CONFIRMED** ที่ยังไม่ชำระเงิน,
การห้ามยกเลิกหลังชำระเงิน และข้อจำกัดวันเช็คอิน/เช็คเอาท์

โค้ดหลัก: **BookingServiceImpl.java**, **BookingState.java**, **Booking.java**,
BookingRepository.java

Observer Pattern

ปัญหาที่แก้

เมื่อพนักงานยืนยันการจอง ระบบต้องแจ้งลูกค้าทางอีเมล ถ้าเขียนการส่งอีเมลไว้ใน
BookingServiceImpl.confirmBooking() โดยตรง จะเกิดปัญหา 3 ข้อ

1. ผูกติดกัน (tight coupling) **BookingServiceImpl** ต้องรู้จักระบบอีเมล
ถ้าวันหลังอยากเพิ่ม SMS หรือ LINE ต้องกลับไปแก้ service การจอง ซึ่งผิดหลัก
Open/Closed
2. ส่งอีเมลก่อนบันทึกจริง ถ้าส่งอีเมลระหว่าง transaction แล้วการบันทึกล้มเหลวทีหลัง
ลูกค้าจะได้อีเมลยืนยันการจองที่ไม่ได้ถูกยืนยันจริง
3. อีเมลล้มทำให้การยืนยันล้ม ถ้า SMTP ใช้งานไม่ได้
ไม่ควรทำให้การยืนยันการจองล้มเหลวไปด้วย

วิธีแก้ด้วย Observer

BookingServiceImpl (Subject) แค่ประกาศเหตุการณ์
BookingConfirmedEvent ผ่าน **ApplicationEventPublisher** ของ Spring

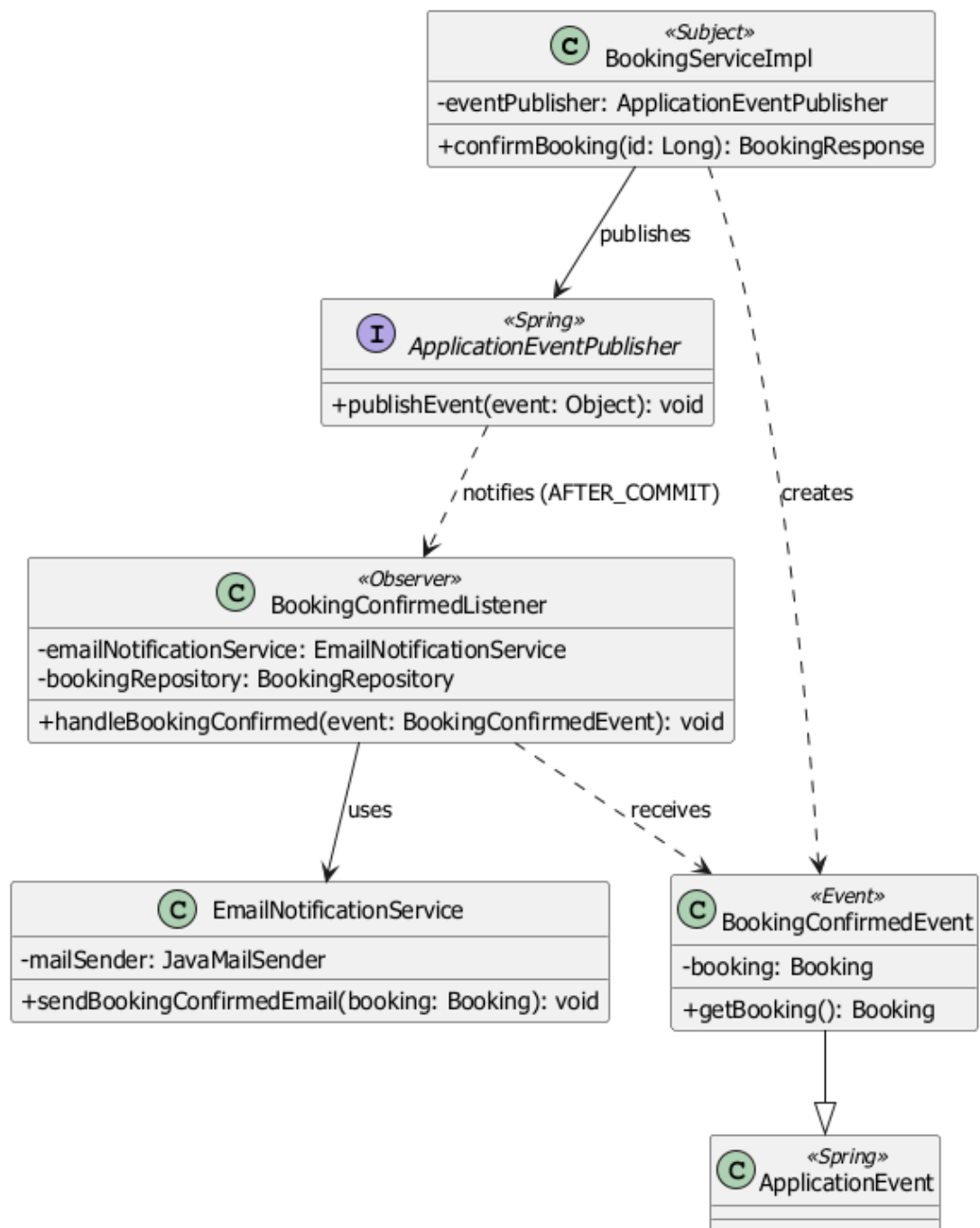
โดยไม่รู้ว่ามีใครรับฟังอยู่ ส่วน **BookingConfirmedListener** (Observer) รับเหตุการณ์แล้วส่ง **EmailNotificationService** ส่งอีเมล

บทบาทใน Pattern	คลาสในระบบ	ไฟล์
Subject (ผู้ประกาศ)	<code>BookingServiceImpl.confirmBooking()</code> ผ่าน <code>ApplicationEventPublisher</code>	<code>service/impl/BookingServiceImpl.java</code>
Event (ข้อมูลที่ส่ง)	<code>BookingConfirmedEvent</code> extends <code>ApplicationEvent</code>	<code>notification/BookingConfirmedEvent.java</code>
Observer (ผู้รับฟัง)	<code>BookingConfirmedListener</code>	<code>notification/BookingConfirmedListener.java</code>
ผู้ทำงานจริง	<code>EmailNotificationService</code> ใช้ <code>JavaMailSender</code>	<code>notification/EmailNotificationService.java</code>

รายละเอียดที่ออกแบบไว้

- **@TransactionalEventListener(phase = AFTER_COMMIT)** Listener ทำงานหลังบันทึกสถานะ **CONFIRMED** ลงฐานข้อมูลสำเร็จแล้วเท่านั้น ถ้า transaction ถูก rollback จะไม่ส่งอีเมล (แก้ปัญหาคำข้อ 2)
- **@Transactional(propagation = REQUIRES_NEW, readOnly = true)** Listener เปิด transaction ใหม่แบบอ่านอย่างเดียว เพื่อโหลดข้อมูลการจอง (ลูกค้า ห้อง วันที่ ราคา) จากฐานข้อมูลอีกครั้ง เพราะ transaction เดิมปิดไปแล้ว
- จับ **MailException / MessagingException** ส่งอีเมลไม่สำเร็จจะบันทึก log แต่การจองยังเป็น **CONFIRMED** (แก้ปัญหาคำข้อ 3)
- เพิ่มช่องทางแจ้งเตือนใหม่ได้โดยไม่แก้ไขโค้ดเดิม เช่น เพิ่ม `SmsNotificationListener` ที่รับ `BookingConfirmedEvent` เหมือนกัน `BookingServiceImpl` ไม่ต้องแก้ไขแค่บรรทัดเดียว (แก้ปัญหาคำข้อ 1)

Class Diagram



รูปที่ 3.12 Class Diagram ของ Observer Pattern

ลำดับการทำงาน

- พนักงานกดยืนยันการจอง → `BookingServiceImpl.confirmBooking()`
เปลี่ยนสถานะผ่าน State Pattern แล้วบันทึก

2. `eventPublisher.publishEvent(new BookingConfirmedEvent(this, savedBooking))`
 3. transaction commit สำเร็จ → Spring เรียก `BookingConfirmedListener.handleBookingConfirmed()`
 4. Listener โหลดการจองใหม่ด้วย `bookingRepository.findById()`
 5. `EmailNotificationService.sendBookingConfirmedEmail()`
สร้างอีเมล HTML แล้วส่งผ่าน `JavaMailSender`
- ทดสอบได้จริงด้วย Docker: อีเมลจะไปแสดงที่ Mailpit <http://localhost:8025>

Strategy Pattern

ปัญหาที่แก้

ราคาการจองของ PetsHotel ประกอบด้วยค่าห้องพัก ค่าบริการเสริม ค่าธรรมเนียมวันหยุด และส่วนลดหลายประเภท หากรวมทุกสูตรไว้ในเมธอดเดียว จะทำให้โค้ดยาว แก้ไขยาก และการเพิ่มกฎใหม่อาจกระทบสูตรเดิม

ระบบจึงแยกกฎแต่ละแบบเป็นคลาสที่ implements `PricingStrategy` และให้ `PricingServiceImpl` เรียกคำนวณผ่าน interface เดียวกัน

วิธีแก้ด้วย Strategy

`PricingServiceImpl` เป็น Context ที่ประสานการคำนวณ โดยรับ `List<PricingStrategy>` ผ่าน constructor injection ของ Spring ส่วนแต่ละ Concrete Strategy รับผิชอบสูตรและเงื่อนไขของตนเอง

`PricingContext` เป็นข้อมูลที่ส่งให้ Strategy เช่น ห้อง จำนวนสัตว์ จำนวนคืน ช่วงวันเข้าพัก บริการเสริม และโปรโมชั่น โดยเป็นคนละบทบาทกับ Context ที่ควบคุมการเรียก Strategy

บทบาทใน Pattern	คลาสในระบบ	ไฟล์
Context: ประสานการคำนวณ	<code>PricingServiceImpl</code>	<code>service/impl/PricingServiceImpl.java</code>
Strategy: สัญญาการคำนวณ	<code>PricingStrategy</code>	<code>pricing/PricingStrategy.java</code>
Concrete Strategy: ค่าห้อง	<code>BaseRoomPriceStrategy</code>	<code>pricing/BaseRoomPriceStrategy.java</code>
Concrete Strategy: บริการเสริม	<code>ExtraServicePricingStrategy</code>	<code>pricing/ExtraServicePricingStrategy.java</code>
Concrete Strategy: ค่าธรรมเนียมวันหยุด	<code>HolidaySurchargeStrategy</code>	<code>pricing/HolidaySurchargeStrategy.java</code>
Concrete Strategy:	<code>LongStayDiscountS</code>	<code>pricing/LongStayDiscountSt</code>

บทบาทใน Pattern	คลาสในระบบ	ไฟล์
ส่วนลดพักระยะยาว	trategy	rategy.java
Concrete Strategy: ส่วนลดโปรโมชั่น	PromotionDiscount Strategy	pricing/PromotionDiscountS trategy.java
ข้อมูลสำหรับคำนวณ	PricingContext	pricing/PricingContext.jav a
หมวดผลการคำนวณ	PricingCategory	domain/enums/PricingCatego ry.java
ผลลัพธ์ราคา	BookingPriceRespo nse	dto/response/BookingPriceR esponse.java

รายละเอียดที่ออกแบบไว้

- **PricingStrategy** กำหนดเมธอด `calculate(context, subtotal)` ซึ่งคืนจำนวนเงินเป็น **BigDecimal** และ `category()` ซึ่งคืนหมวดราคา
- Concrete Strategy ทั้งห้าคลาส implements interface เดียวกันและลงทะเบียนเป็น Spring bean ด้วย **@Component**
- **PricingServiceImpl** รับ Strategy เป็นรายการผ่าน constructor โดยไม่สร้าง Concrete Strategy ด้วย **new** เอง
- ระบบเรียกหลาย Strategy เพื่อประกอบราคาหนึ่งรายการ โดยแยกการคำนวณค่าใช้จ่ายออกจากส่วนลด

Strategy	หมวด	สูตรหรือเงื่อนไข
BaseRoomPriceStrategy	ROOM	ราคาต่อสัตว์ต่อคืน × จำนวนคืน × จำนวนสัตว์
ExtraServicePricingStrategy	EXTRA_SERVICE	รวม <code>unitPrice × quantity</code> ของรายการบริการเสริม
HolidaySurchargeStrategy	HOLIDAY_SURCHARGE	ราคาห้องต่อคืนรวมสัตว์ทุกตัว × 30% × จำนวนคืนที่ตรงกับวันหยุดในโค้ด
LongStayDiscountStrategy	DISCOUNT	พักอย่างน้อย 5 คืน รับส่วนลด 10% ของยอดก่อนหักส่วนลด
PromotionDiscountStrategy	DISCOUNT	ลดเป็นเปอร์เซ็นต์หรือจำนวนเงินตามโปรโมชั่น และตรวจจำนวนคืนขั้นต่ำถ้ามี

- วันหยุดที่ **HolidaySurchargeStrategy** กำหนดไว้ในโค้ดคือ 1 มกราคม, 13-15 เมษายน และ 31 ธันวาคม โดยตรวจวันเข้าพักแต่ละคืน ไม่รวมวันเช็กเอาท์
- **PromotionDiscountStrategy** คืนส่วนลด 0 เมื่อไม่มีโปรโมชั่น
โปรโมชั่นปิดใช้งาน หรือจำนวนคืนไม่ถึงขั้นต่ำ

- การตรวจช่วงวันที่ของโปรโมชั่นก่อนสร้างการจองอยู่ใน **BookingServiceImpl** ส่วน **PromotionDiscountStrategy** รับผิดชอบสูตรส่วนลดและเงื่อนไขจำนวนคืน
- หากมีส่วนลดหลายแบบ ระบบเลือกจำนวนเงินส่วนลดที่สูงที่สุดด้วย **max()** โดยไม่รวมส่วนลดทุกแบบเข้าด้วยกัน
- ส่วนลดสุดท้ายไม่เกินยอดก่อนหักส่วนลด
จึงไม่ทำให้ยอดสุทธิติดลบสำหรับข้อมูลราคาที่ถูกต้อง
- ระบบปัดจำนวนเงินเป็นทศนิยม 2 ตำแหน่งด้วย **RoundingMode.HALF_UP**
- สามารถเพิ่ม Strategy ในหมวดเดิมได้โดยสร้างคลาสที่ implements **PricingStrategy** และลงทะเบียนเป็น Spring bean
ส่วนการเพิ่มหมวดใหม่อาจต้องปรับ **PricingCategory** และการรวมผลใน **PricingServiceImpl**

Class Diagram

รูปที่ 3.4 Class Diagram ของระบบ

กรอบ **Pricing - Strategy Pattern** แสดง **PricingServiceImpl** ที่ถือรายการ **PricingStrategy** คลาส Concrete Strategy ทั้งห้าที่ implements interface รวมถึง **PricingContext**, **PricingCategory** และ **BookingPriceResponse**

ลำดับการทำงาน

1. ผู้เรียกส่งข้อมูลห้อง จำนวนสัตว์ วันเช็คอิน วันเช็คเอาท์ บริการเสริม และโปรโมชั่นให้ **PricingServiceImpl.calculate()**
2. Service ตรวจสอบข้อมูลและคำนวณจำนวนคืนจากวันเช็คอินถึงวันเช็คเอาท์ แล้วสร้าง **PricingContext** สำหรับใช้งาน โดยไม่ใช้จำนวนคืนที่ผู้เรียกส่งมาโดยตรง
3. Service เรียก Strategy ที่ไม่ใช่หมวด **DISCOUNT** และรวมผลตามหมวดราคา
4. ปัดค่าห้อง บริการเสริม และค่าธรรมเนียมวันหยุดเป็นทศนิยม 2 ตำแหน่ง
แล้วรวมเป็นยอดก่อนหักส่วนลด
5. Service เรียก Strategy หมวด **DISCOUNT** โดยส่งยอดก่อนหักส่วนลดให้แต่ละคลาส
6. เลือกส่วนลดที่สูงที่สุด ปัดเป็นทศนิยม 2 ตำแหน่ง และจำกัดไม่ให้นเกินยอดก่อนหักส่วนลด
7. คำนวณยอดสุทธิและคืน **BookingPriceResponse** ซึ่งมีค่าห้อง บริการเสริม ค่าธรรมเนียมวันหยุด ส่วนลด และยอดรวม

ตัวอย่างการคำนวณ

สมมติว่ามีสัตว์เลี้ยง 1 ตัว ราคาห้อง 500.00 บาทต่อตัวต่อคืน เข้าพักวันที่ 1-6 ตุลาคม 2026 รวม 5 คืน มีบริการอาบน้ำ 250.00 บาท และเลือกโปรโมชั่นส่วนลด 20% ที่ผ่านการตรวจวันและเงื่อนไขแล้ว

รายการ	การคำนวณ	จำนวนเงิน
ค่าห้อง	$500.00 \times 5 \times 1$	2,500.00 บาท
บริการเสริม	250.00×1	250.00 บาท
ค่าธรรมเนียมวันหยุด	ไม่มีคืนที่ตรงกับวันหยุดในโค้ด	0.00 บาท
ยอดก่อนหักส่วนลด	$2,500.00 + 250.00 + 0.00$	2,750.00 บาท
ส่วนลดพักระยะยาวที่คำนวณได้	$2,750.00 \times 10\%$	275.00 บาท
ส่วนลดโปรโมชั่นที่คำนวณได้	$2,750.00 \times 20\%$	550.00 บาท
ส่วนลดที่เลือกใช้	ค่าสูงสุดระหว่าง 275.00 กับ 550.00	550.00 บาท
ยอดสุทธิ	$2,750.00 - 550.00$	2,200.00 บาท

ตัวอย่างนี้แสดงว่าทั้งสอง Strategy คำนวณส่วนลดได้ แต่ Service เลือกใช้ส่วนลดที่สูงกว่าเพียงจำนวนเดียว

Enterprise Patterns

ปัญหาที่แก้

ระบบมีทั้งหน้าเว็บสำหรับลูกค้า/พนักงานและ REST API ซึ่งใช้กฎการจองชุดเดียวกัน หากรวมการรับ HTTP การตรวจสอบธุรกิจ การอ่านเขียนฐานข้อมูล และการจัดรูปแบบผลลัพธ์ไว้ด้วยกัน จะเกิดโค้ดซ้ำและแก้ไขแต่ละส่วนได้ยาก

PetsHotel จึงแบ่งส่วนเหล่านี้ด้วย Layered Architecture, MVC, Repository, Service Layer, DTO + Mapper และ Dependency Injection โดยแต่ละแนวทางทำงานร่วมกันตามหน้าที่ของตนเอง

1. Layered Architecture

ชั้น / ส่วนของระบบ	หน้าที่	ตัวอย่างในโค้ด
Presentation	รับคำขอ ตรวจสอบข้อมูลจาก HTTP/ฟอร์ม และส่งผลให้ผู้ใช้	controller/web/BookingController , controller/api/BookingRestContro

ชั้น / ส่วนของระบบ	หน้าที่	ตัวอย่างในโค้ด
		ller, resources/templates/booking.html
Service / Application	ประสานขั้นตอนธุรกิจและกำหนด transaction	service/BookingService, service/impl/BookingServiceImpl
Domain	เก็บข้อมูลและกฎของการจอง รวมถึงพฤติกรรมตามสถานะ	domain/entity/Booking, domain/enums/BookingStatus, state/
Data Access	อ่าน เขียน ค้นหา และล็อกข้อมูลผ่าน Spring Data JPA	repository/BookingRepository, RoomRepository, PetRepository
DTO / Mapping	กำหนดข้อมูลเข้าออกและแปลง Entity เป็นผลลัพธ์	dto/request/, dto/response/, mapper/BookingMapper

เส้นทางหลักของการจองคือ **Controller** → **BookingService** → **Repository** → ฐานข้อมูล โดย Service ใช้ Entity, State และบริการคำนวณราคา แล้วส่ง **BookingResponse** กลับผ่าน Mapper การแบ่งชั้นช่วยให้เปลี่ยนหน้าเว็บหรือรูปแบบ API ได้โดยยังเรียกกฎธุรกิจชุดเดิม

2. Model-View-Controller (MVC)

- **Model** คือข้อมูลและกฎของระบบ เช่น **Booking**, **BookingStatus** และการทำรายการผ่าน **BookingService** ส่วน `org.springframework.ui.Model` เป็นตัวส่งข้อมูลที่ Controller เตรียมให้ View
- **View** คือ Thymeleaf templates เช่น `templates/booking.html`, `my-bookings.html`, `booking-detail.html` และ `admin/bookings.html`
- **Controller** เช่น **BookingController** รับคำขอจากลูกค้า เรียก Service ใส่ข้อมูลใน Model และเลือก View หรือ redirect

ตัวอย่าง `BookingController.showMyBookings()` อ่านผู้ใช้จาก **Principal** เรียก `bookingService.getBookingsByUserId(user.getId())` ใส่ผลใน Model ชื่อ `bookings` แล้วคืนชื่อ View `my-bookings`

BookingRestController เป็นช่องทาง HTTP ที่คืน DTO เป็น JSON แทนการเลือก Thymeleaf View และใช้ **BookingService** ร่วมกับหน้าเว็บ

3. Repository Pattern

BookingRepository extends **JpaRepository<Booking, Long>** รวมการเข้าถึงข้อมูลการจองไว้ที่ interface เดียว Service จึงเรียกเมธอดของ Repository โดยไม่เขียนคำสั่งฐานข้อมูลซ้ำใน Controller

เมธอด	หน้าที่
save(booking)	บันทึกหรืออัปเดตการจองผ่าน Spring Data JPA
findById(userId)	ค้นหาการจองของลูกค้า
findById(id)	อ่านการจองตามรหัส
findByIdForUpdate(id)	โหลดการจองพร้อม PESSIMISTIC_WRITE lock ภายใน transaction
existsOverlappingBooking(...)	ตรวจสอบว่าห้องมีการจองทับช่วงวันในสถานะที่ยังครองห้องหรือไม่

Spring Data JPA สร้าง implementation ของ Repository ให้ ส่วน query เฉพาะธุรกิจอยู่ใน **@Query** ของ interface การแยกนี้ช่วยรวมรายละเอียดการเข้าถึงข้อมูลและใช้ Repository แทนด้วย mock ในการทดสอบ Service ได้

4. Service Layer

BookingService กำหนดรายการที่ระบบให้บริการ เช่น **createBooking()**, **previewPrice()**, **confirmBooking()**, **checkIn()**, **checkOut()** และ **cancelBooking()** ส่วน **BookingServiceImpl** ประสาน Repository, Availability, Pricing, State, Mapper และ Event Publisher

- การสร้างการจองตรวจสอบข้อมูล เจ้าของสัตว์ ความจุ สัตว์/ห้องที่จองซ้อน บริการ และโปรโมชั่น ก่อนคำนวณราคาและบันทึก
- การเปลี่ยนสถานะใช้ State Pattern และตรวจเงื่อนไขวันที่/การชำระเงินใน Service
- **@Transactional** ที่คลาสประกอบการทำรายการ ส่วนเมธอดอ่านข้อมูลและราคา preview ใช้ **@Transactional(readonly = true)**
- Controller ฟังเว็บและ API เรียก Service เดียวกัน จึงใช้กฎธุรกิจร่วมกัน โดย Controller ยังรับผิดชอบรูปแบบ HTTP การตรวจบัญชี/สิทธิ์ และการแสดงข้อผิดพลาด

5. DTO + Mapper

DTO กำหนดข้อมูลที่แต่ละคำขอหรือผลลัพธ์ต้องใช้ โดยแยกจาก JPA Entity ที่มีความสัมพันธ์กับข้อมูลอื่น

DTO / Mapper	หน้าที่
CreateBookingApiRequest	รับ JSON ที่มี roomId, petIds, วันที่ บริการ และโปรโมชั่น โดยไม่มี userId ให้ client กำหนด
CreateBookingRequest	รับข้อมูลฟอร์มและส่งข้อมูลให้ Service รวม userId ที่ Controller กำหนดจากบัญชีที่ล็อกอิน
BookingPriceResponse	ส่งผลราคา preview และส่วนประกอบราคา
BookingResponse	ส่งข้อมูลการจอง สถานะ ราคา ห้อง สัตว์ และรายการบริการให้เว็บ/API
BookingMapper.toResponse(booking)	แปลง Entity และความสัมพันธ์ที่ต้องใช้ให้เป็น BookingResponse

BookingMapper เป็นคลาส @Component ที่แปลงข้อมูลด้วยโค้ด Java เช่นดึง userId, roomId, รายชื่อสัตว์ และรายการบริการ พร้อมคำนวณ checkInAllowed / checkOutAllowed จากวันปัจจุบันใน Asia/Bangkok จึงไม่ส่ง JPA Entity ทั้งกราฟออกไปโดยตรง

ทั้ง API และเว็บกำหนดผู้จองจากผู้ใช้ที่ล็อกอิน ฝั่งเว็บใช้ @InitBinder จำกัด field ที่รับจากฟอร์มและกำหนด form.setUserId(user.getId()) ใหม่ ส่วน API สร้าง CreateBookingRequest จาก DTO และบัญชีปัจจุบัน

6. Dependency Injection (DI)

Spring จัดการ dependency ของ Controller, Service และ Mapper ผ่าน constructor injection เช่น BookingServiceImpl ประกาศ dependency เป็น private final และใช้ Lombok @RequiredArgsConstructor สร้าง constructor ให้ Spring ส่ง bean ที่เหมาะสมเข้ามา

ตัวอย่าง dependency ได้แก่ BookingRepository, PricingService, AvailabilityService, BookingMapper และ ApplicationEventPublisher ส่วน PricingServiceImpl รับ List<PricingStrategy> ผ่าน constructor เพื่อใช้ Strategy ทุกตัวที่ลงทะเบียนเป็น bean

การออกแบบนี้ทำให้คลาสระบุสิ่งที่ต้องใช้ชัดเจน และชุดทดสอบส่ง mock ของ dependency เข้าไปได้โดยไม่ต้องเชื่อมฐานข้อมูลหรือส่งอีเมลจริง ทั้งนี้ Concrete State ใน `getState()` ยังสร้างด้วย `new` ตามโค้ดปัจจุบัน ไม่ได้เป็น bean ที่ Spring inject ให้

ตัวอย่างการทำงานร่วมกัน

1. View ส่งฟอร์มไป **BookingController** หรือ client ส่ง JSON ไป **BookingRestController**
2. Controller ตรวจสอบคำขอและบัญชีผู้ใช้ แล้วเตรียม **CreateBookingRequest** โดยกำหนด **userId** จาก **Principal**
3. Controller เรียก **BookingService.createBooking()** โดย Spring inject **BookingServiceImpl** ให้
4. Service ตรวจสอบธุรกิจและประสาน Repository/Availability/Pricing ภายใน transaction
5. Repository บันทึก **Booking** และรายการที่มี cascade ผ่าน JPA
6. **BookingMapper** แปลง Entity เป็น **BookingResponse** และ transaction commit เมื่อทำการสำเร็จ
7. ฝั่งเว็บ redirect ไปหน้ารายละเอียดการจอง ส่วน REST API คืน **201 Created** พร้อม **BookingResponse** เป็น JSON

โค้ดอ้างอิง: `BookingController.java`, `BookingRestController.java`,

`BookingService.java`, `BookingServiceImpl.java`, `BookingRepository.java`,

`BookingMapper.java`

3.11 การวิเคราะห์ตามหลัก SOLID

สมาชิกแต่ละคนวิเคราะห์หลัก SOLID จากโค้ดในส่วนที่ตนรับผิดชอบ โดยระบุไฟล์และบรรทัดที่เกี่ยวข้อง ดังนี้

กิตติธัช: Room, User และ Notification

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
S Single Responsibility	<code>service/impl/RoomServiceImpl.java:26</code> , <code>service/impl/LocalFileStorageService.java:22</code> , <code>mapper/RoomMapper.java:8</code>	<code>RoomServiceImpl</code> ทำเฉพาะกฎของห้อง (ห้ามเลขห้องซ้ำ, สถานะเริ่มต้น ACTIVE) งานบันทึกไฟล์รูปแยกไป <code>LocalFileStorageServ</code>	แต่ละคลาสมีเหตุผลที่ต้องเปลี่ยนเพียงอย่างเดียว เช่นถ้าเปลี่ยนที่เก็บรูปไป cloud จะแก้แค่ storage ไม่กระทบกฎของห้อง

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
		ice และการแปลง Entity เป็น DTO แยกไป RoomMapper ส่วน validation อยู่ใน CreateRoomRequest	
O Open/Closed	service/impl/BookingServiceImpl.java:389-390, notification/BookingConfirmedListener.java:26	ตอนยืนยันการจอง service แค่ publishEvent(new BookingConfirmedEvent(...)) การส่งอีเมลอยู่ใน Listener แยก	เพิ่มช่องทางแจ้งเตือนใหม่ (เช่น SMS) ได้ด้วยการเพิ่ม Listener ใหม่ โดยไม่ต้องแก้ BookingServiceImpl
L Liskov Substitution	service/FileStorageService.java:5, service/impl/LocalFileStorageService.java:22, service/impl/RoomServiceImpl.java:137	RoomServiceImpl เรียก fileStorageService.storeRoomImage() ผ่าน interface ใน test ใช้ mock แทน LocalFileStorageService ได้ และทำงานถูกต้องเหมือนกัน	ตัว implement ใดๆ ของ FileStorageService ใช้แทนกันได้โดย RoomServiceImpl ไม่ต้องรู้และไม่พึง ไม่มีเมธอดไหนโยน UnsupportedOperationException
I Interface Segregation	service/AuthService.java:5, service/CurrentUserService.java:5, service/UserService.java:8, controller/web/AdminUserController.java:26	งานเกี่ยวกับผู้ใช้แยกเป็น 3 interface เล็ก: AuthService (สมัคร), CurrentUserService (หาผู้ใช้ที่ล็อกอิน), UserService (แอดมินจัดการสมาชิก)	คลาสที่ใช้พึ่งเฉพาะเมธอดที่ต้องใช้ เช่น AuthController ไม่ต้องรู้จักเมธอดค้นหาหรือแก้ไขสมาชิกของแอดมิน
D Dependency Inversion	service/impl/RoomServiceImpl.java:30-35, controller/api/RoomRestController.java:37-39	RoomServiceImpl รับ FileStorageService (interface) ผ่าน constructor ไม่ได้ new LocalFileStorageService() เอง และ RoomRestController รับ RoomService (interface) ผ่าน constructor	ชั้นบนพึ่ง abstraction ไม่พึ่งคลาสจริง ใช้ Constructor Injection ตามที่ใบงานกำหนด Spring เป็นผู้ส่งตัวจริงเข้ามา

วันนี้: Availability, Room Search และ Exception Handling

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
S Single Respons ibility	service/impl/AvailabilityServiceImpl.java:173-193, mapper/AvailabilityMapper.java:22-28, controller/web/AvailabilityController.java:29-46, exception/GlobalExceptionHandler.java:22-37	AvailabilityServiceImpl.hasSpace() ทำเฉพาะกฎว่าห้องว่างหรือไม่ (สถานะ ACTIVE, ความจุ, การจองทับวัน) การแปลง Room เป็น AvailabilityResponse แยกไป AvailabilityMapper controller แค่รับฟอร์มและเลือก view ส่วนการแปลง exception เป็น HTTP status รวมอยู่ใน GlobalExceptionHandler ที่เดียว	ถ้าเปลี่ยนกฎห้องว่าง (เช่นให้แชร์ห้องได้) แก่แค่ hasSpace() ถ้าเปลี่ยนรูปแบบ JSON แก่แค่ mapper ถ้าเปลี่ยนรูปแบบ error แก่แค่ handler แต่ละคลาสมีเหตุผลให้เปลี่ยน เพียงอย่างเดียว
O Open/Closed	exception/GlobalExceptionHandler.java:32 , exception/PetNotAvailableException.java:8-9, exception/InvalidBookingStateException.java:3	handler จับ IllegalStateException ไว้กว้าง ๆ เมื่อมีคนเพิ่ม exception ใหม่ที่ extends IllegalStateException (เช่น PetNotAvailableException ที่กิตติยาตาเพิ่มที่หลัง) ระบบตอบ 409 ได้ทันที	เพิ่มประเภท error ใหม่ได้โดยไม่ต้องแก้ GlobalExceptionHandler หรือ controller ใดๆ เปิดให้ขยาย ปิดไม่ให้แก้
L Liskov Substitution	exception/PetNotAvailableException.java:8-9, exception/GlobalExceptionHandler.java:32 , controller/api/AvailabilityRestControllerTest.java:36	PetNotAvailableException ใช้แทน IllegalStateException ได้ทุกที่โดยพฤติกรรมไม่เปลี่ยน (handler ตอบ 409 เหมือนกัน) และใน test ใช้ mock(AvailabilityService.class) แทน AvailabilityServiceImpl ได้ controller ทำงานถูกต้องเหมือนเดิม	subclass และ implementation ใช้แทนพ่อแม่ / interface ได้โดยผู้เรียกไม่ต้องรู้และไม่พัง
I Interfac	service/AvailabilityService.java:9-36,	การค้นหาห้องแบบแบ่งหน้าแยกเป็น interface	RoomSearchRestController ฟังเฉพาะเมธอดค้นหา

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
e Segregation	service/RoomSearchService.java:9-12, controller/api/RoomSearchRestController.java:31	RoomSearchService (1 เมธอด) ไม่ได้ยัดเพิ่มเข้าไปใน AvailabilityService ที่ดูแลกฎห้องว่าง	ไม่ต้องรู้จัก checkRoomAvailable() หรือปฏิทิน และ BookingServiceImpl ก็ไม่ต้องรู้จักเมธอดแบ่งหน้า
D Dependency Inversion	controller/api/AvailabilityRestController.java:40-46, controller/api/RoomSearchRestController.java:33-35, service/impl/AvailabilityServiceImpl.java:38-44, service/impl/BookingServiceImpl.java:84, 195	controller รับ AvailabilityService / RoomSearchService (interface) ผ่าน constructor service รับ RoomRepository / BookingRepository (interface ของ Spring Data) และ BookingServiceImpl เรียก availabilityService.checkRoomAvailable() ผ่าน interface	ชั้นบนพึ่ง abstraction ไม่ new คลาสจริงเอง ใช้ Constructor Injection ตามใบงาน Spring เป็นผู้ส่งตัวจริงเข้ามา จึงเปลี่ยน implementation หรือใช้ mock ใน test ได้

รติมา: Pet และ Daily Care Report

เลขบรรทัดอ้างอิงจาก origin/develop commit 453d8e5

โดยพาธของโค้ดหลักในตารางเริ่มจาก

code/src/main/java/com/example/petshotel/

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
S Single Responsibility	mapper/PetMapper.java:7-8, service/impl/PetServiceImpl.java:55-56, mapper/DailyCareReportMapper.java:9-10, service/impl/DailyCareReportServiceImpl.java:80	PetServiceImpl และ DailyCareReportServiceImpl จัดการกฎและการบันทึกข้อมูล ส่วนการแปลง Entity เป็น Response DTO อยู่ใน PetMapper และ DailyCareReportMapper	งานแปลงข้อมูลมีคลาสรับผิดชอบแยกจากงานบันทึกและตรวจเงื่อนไข หากรูปแบบ Response เปลี่ยน จุดหลักที่ต้องแก้คือ Mapper
O Open/Closed	service/FileStorageService.java:5-9, service/impl/PetServiceImpl.java:143-	PetServiceImpl เรียกบันทึกและลบรูปผ่าน FileStorageService ปัจจุบัน	หากเปลี่ยนไปเก็บรูปบนบริการอื่น สามารถเพิ่ม implementation ของ FileStorageService

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
	145, service/impl/LocalFileStorageService.java:101, service/impl/LocalFileStorageService.java:130	LocalFileStorageService เป็น implementation ที่เก็บไฟล์ในเครื่อง	และกำหนดให้ระบบใช้ตัวใหม่ได้ โดยไม่ต้องแก้ไขขั้นตอนจัดการรูปใน PetServiceImpl
Liskov Substitution	service/PetService.java:10-17, service/impl/PetServiceImpl.java:23, controller/api/PetRestController.java:37-40, code/src/test/java/com/example/petshotel/controller/api/PetRestControllerTest.java:36-42	Controller รับ dependency เป็นชนิด PetService ส่วน PetServiceImpl เป็นคลาสที่ implements interface นี้ ในการทดสอบมีการใช้ mock ของ PetService แทน implementation จริง	ผู้เรียกใช้งานผูกกับสัญญาของ PetService จึงมีจุดให้สลับ implementation ได้ แต่การมี mock ไม่ได้พิสูจน์ว่า implementation ใหม่จะทำงานแทนกันถูกต้องทุกกรณี หากเพิ่ม implementation ใหม่ต้องทดสอบให้รักษาลักษณะและเงื่อนไขเดิม
Interface Segregation	service/CurrentUserService.java:5-6, service/PetService.java:10-17, service/DailyCareReportService.java:14-24, controller/web/PetPageController.java:30-33, controller/web/CustomerCareReportPageController.java:33-38	งานค้นหาผู้ใช้ปัจจุบัน งานจัดการสัตว์เลี้ยง และงานรายงานการดูแลแยกเป็นคนละ interface โดยหน้า Pet รับ PetService ส่วนหน้ารายงานรับ DailyCareReportService และทั้งสองหน้าใช้ CurrentUserService สำหรับข้อมูลผู้ใช้	การแยกสัญญาตามขอบเขตงานทำให้แต่ละหน้าพึ่งบริการที่เกี่ยวข้องกับตัวเอง โดยไม่ต้องใช้ interface เดียวที่รวมงานผู้ใช้ สัตว์เลี้ยง และรายงานไว้ทั้งหมด
Dependency Inversion	controller/web/PetPageController.java:30-33, service/PetService.java:10, service/impl/PetServiceImpl.java:27-33, service/FileStorageService.java:5	PetPageController รับ PetService และ PetServiceImpl รับ FileStorageService ผ่าน constructor ทั้งสองจุดพึ่ง interface แทนการสร้างคลาส implementation ด้วย new	Controller ไม่ต้องรู้รายละเอียดของ PetServiceImpl และ service ไม่ต้องรู้รายละเอียดของ LocalFileStorageService ทำให้เปลี่ยน implementation และทดสอบแยกส่วนได้ง่ายขึ้น

กิตติยาตา: Booking และ State Pattern

เลขบรรทัดอ้างอิงจากโค้ด commit **61e633f** โดยพาธของโค้ดหลักในตารางเริ่มจาก `code/src/main/java/com/example/petshotel/`

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
S Single Respons ibility	<code>controller/web/BookingController.java:99-130,</code> <code>service/impl/BookingServiceImpl.java:135-138,</code> <code>mapper/BookingMapper.java:15-26</code>	Controller รับฟอร์ม ตรวจสอบบัญชีผู้ใช้ และเลือกหน้าหรือ redirect ส่วน Service ประสานกฎธุรกิจและการบันทึก ขณะที่ Mapper แปลง Booking เป็น BookingResponse	แยกเหตุผลในการแก้ไขแต่ละส่วน เช่น หากเปลี่ยนข้อมูลใน Response จุดหลักที่ต้องแก้คือ Mapper หากเปลี่ยนกฎการจอง จุดหลักอยู่ที่ Service
O Open/Closed	<code>controller/web/BookingController.java:45</code> , <code>controller/web/BookingController.java:113-114,</code> <code>service/BookingService.java:11-35,</code> <code>service/impl/BookingServiceImpl.java:561-579</code>	Controller เรียกการจองผ่าน interface BookingService โดยไม่ตรวจสอบชนิด implementation ส่วน Service เลือก Concrete State ผ่าน <code>getState()</code> ซึ่งยังใช้ switch และ new	สามารถเพิ่ม implementation ของ BookingService ที่รักษาสัญญาเดิมและกำหนดให้ Spring ใช้งาน โดยไม่เปลี่ยนจุดเรียกใน Controller แต่การเพิ่มสถานะใหม่ยังต้องปรับ BookingStatus และ <code>getState()</code> จึงยังมีข้อจำกัดด้าน OCP ในการสร้าง State
L Liskov Substitution	<code>state/BookingState.java:5-13,</code> <code>state/PendingState.java:28-29,</code> <code>state/ConfirmedState.java:28-29,</code> <code>service/impl/BookingServiceImpl.java:449-454</code>	PendingState และ ConfirmedState ใช้สัญญา <code>cancel(Booking)</code> เดียวกัน และเปลี่ยนเป็น CANCELLED เมื่ออนุญาตให้ยกเลิก ผู้เรียกใช้ตัวแปรชนิด BookingState	ในเส้นทางยกเลิกที่อนุญาตทั้งสอง implementation ให้ผลตามความหมายของคำสั่งเดียวกัน โดยผู้เรียกไม่ต้องตรวจสอบชนิดคลาส อย่างไรก็ตาม การ implements interface เพียงอย่างเดียวไม่ได้พิสูจน์ LSP ทุกกรณี

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
			ต้องพิจารณาเงื่อนไขและข้อผิดพลาดตามสถานะด้วย
I Interfac e Segregat ion	state/BookingState.java:5-13, service/BookingService.java:19-35	BookingState มีเฉพาะ confirm, checkIn, checkOut และ cancel ส่วนการสร้างการจอง ค้นข้อมูล และ preview ราคาอยู่ใน BookingService	แยกสัญญาของพฤติกรรมตาม สถานะออกจากบริการการจอง Concrete State จึงไม่ต้อง implements การคำนวณราคา การค้นฐานข้อมูล หรือการออกไปเสร็จ
D Depend ency Inversio n	controller/web/BookingController.java:42-50, service/impl/BookingServiceImpl.java:66-86	Controller รับ BookingService ผ่าน constructor injection ส่วน BookingServiceImpl รับ PricingService, AvailabilityService และ Repository เป็น dependency โดยใช้ private final และ RequiredArgsConstructor	งานระดับบนพึ่ง interface ของบริการที่ใช้ และให้ Spring ส่ง implementation เข้ามา จึงเปลี่ยน dependency หรือใช้ mock ในการทดสอบได้ ทั้งนี้การสร้าง Concrete State ใน getState() ยังผูกกับคลาสจริง

ตัวอย่างการประยุกต์

S — แยกการแปลงข้อมูลออกจากกฎธุรกิจ

`BookingServiceImpl.createBooking()` เตรียมการจองและบันทึกผ่าน
Repository แล้วเรียก `BookingMapper.toResponse()` เพื่อแปลงผลลัพธ์

ตัวอย่างเช่น หากต้องเพิ่มข้อมูลที่แสดงใน `BookingResponse` สามารถปรับ DTO และ
Mapper โดยไม่ต้องนำ logic แปลงข้อมูลไปใส่ใน `PendingState` หรือ `ConfirmedState`

Concrete State รับผิดชอบการเปลี่ยนสถานะ ส่วนการโหลดข้อมูล การบันทึก
และการประกาศเหตุการณ์อยู่ใน Service

O — เรียกบริการผ่าน interface และระบุข้อจำกัดของ State

`BookingController` เรียก `bookingService.createBooking(form)`
ผ่าน `BookingService` จึงไม่จำเป็นต้องรู้ว่าคลาสใดเป็น implementation

หากเพิ่ม implementation ใหม่ ต้องรักษาความหมายของเมธอดและกำหนดให้ Spring เลือก bean ที่ต้องการ จุดเรียกใน Controller จึงใช้ได้

สำหรับ State Pattern ปัจจุบัน getState() ยังสร้าง PendingState, ConfirmedState, CheckedInState, CheckedOutState และ CancelledState โดยตรง
การเพิ่มสถานะใหม่จึงต้องแก้จุดเลือก State ด้วย
ไม่ควรอ้างว่าส่วนนี้รองรับการเพิ่มสถานะโดยไม่แก้โค้ดเดิมเลย

L — เรียกพฤติกรรมผ่าน BookingState

ตัวอย่างจากเส้นทางยกเลิกการจอง:

```
BookingState state = getState(booking.getStatus());
state.cancel(booking);
```

เมื่อเป็น PENDING หรือ CONFIRMED และผ่านเงื่อนไขการชำระเงินแล้ว State ที่เลือกจะเปลี่ยนสถานะเป็น CANCELLED

ส่วน State ที่ไม่อนุญาต เช่น CheckedInState จะโยน IllegalStateException และไม่เปลี่ยนสถานะ การวิเคราะห์ LSP จึงต้องพิจารณาแล้วว่าคำสั่งอาจถูกปฏิเสธตามสถานะ ไม่ใช่คาดหวังว่าทุก State ต้องทำทุกคำสั่งสำเร็จ

ปัจจุบัน interface ไม่ได้เขียนเงื่อนไขของแต่ละคำสั่งเป็น JavaDoc อย่างชัดเจน จึงควรใช้ตารางสถานะและชุดทดสอบประกอบการอธิบายพฤติกรรม

I — จำกัดหน้าที่ของ interface State

BookingState กำหนดเฉพาะคำสั่งในวงจรสถานะการจอง
ส่วนงานอื่นใช้สัญญาหรือคลาสที่รับผิดชอบแยกกัน เช่น:

- BookingService: สร้าง ค้นหา และประสานการทำรายการจอง
- PricingService: คำนวณราคา
- BookingRepository: อ่านและบันทึกข้อมูล
- BookingMapper: แปลงผลลัพธ์เป็น DTO

การที่บาง State ปฏิเสธคำสั่งด้วย IllegalStateException เป็นกฎของสถานะนั้น ไม่ใช่หลักฐานโดยลำพังว่า interface ละเมิดหรือผ่าน ISP

D — ส่ง dependency ผ่าน constructor

BookingServiceImpl รับบริการและ Repository ผ่าน constructor ที่ Lombok สร้างจาก private final fields โดย Spring เป็นผู้ส่ง bean เข้ามา

ตัวอย่างใน BookingServiceTest.setUp() สร้าง mock ของ Repository, PricingService, AvailabilityService และ ApplicationEventPublisher แล้วส่งเข้า constructor ของ BookingServiceImpl จึงตรวจพฤติกรรม Service ได้โดยไม่ต้องให้ dependency เหล่านั้นทำงานจริง

อย่างไรก็ตาม BookingServiceImpl ยังพึ่ง BookingMapper ซึ่งเป็นคลาสจริง และสร้าง Concrete State ด้วย new จึงควรอธิบาย DIP เฉพาะส่วนที่พึ่ง abstraction ไม่อ้างว่าทุก dependency ในคลาสเป็น interface

หลักฐานเพิ่มเติมจากไฟล์ทดสอบ

ไฟล์

code/src/test/java/com/example/petshotel/service/BookingServiceTest.java มีตัวอย่างที่เกี่ยวข้อง:

บรรทัดเริ่มต้น	เมธอดทดสอบ	พฤติกรรมที่ตรวจ
618	confirmPendingBookingShouldPublishEvent()	ยืนยัน PENDING เป็น CONFIRMED บันทึกและประกาศ BookingConfirmedEvent
635	confirmCancelledBookingShouldRejectWithoutEvent()	ปฏิเสธการยืนยันรายการที่ยกเลิกแล้วโดยไม่ประกาศเหตุการณ์
649	cancelUnpaidConfirmedBookingShouldSucceed()	ยกเลิก CONFIRMED ที่ยังไม่ชำระเงินเป็น CANCELLED
662	cancelPaidBookingShouldKeepOriginalData()	ปฏิเสธการยกเลิกหลังชำระเงิน และรักษาข้อมูลเดิม
772	pendingBookingCannotCheckIn()	ปฏิเสธการเช็คอินขณะยังเป็น PENDING

รายการนี้อ้างอิงกรณีทดสอบที่มีอยู่ในโค้ด ไม่ใช่รายงานผลการรันทดสอบครั้งใหม่ และไม่ได้พิสูจน์ SOLID ครบทุกสถานการณ์

ขอบเขตของการวิเคราะห์

- การแบ่ง Controller, Service, State และ Mapper ช่วยแยกหน้าที่ แต่ BookingServiceImpl ยังประสานงานหลายขั้นตอน จึงต้องรักษาขอบเขตของแต่ละส่วนเมื่อเพิ่มความสามารถ
- การเพิ่มสถานะใหม่ต้องตรวจ getState(), BookingStatus, กฎวันที่ การตรวจห้อง/สัตว์ซ้อน และส่วนแสดงผลที่ใช้สถานะ

- BookingService รวมทั้งงานอ่าน สร้าง และเปลี่ยนสถานะ
หากขอบเขตระบบขยายมากขึ้น อาจพิจารณาแยก interface ตามผู้เรียกและหน้าที่
- ข้อสังเกตเหล่านี้เป็นการวิเคราะห์ให้โค้ดปัจจุบัน
ไม่ได้หมายความว่ามีการแก้ไขโค้ดตามข้อเสนอแล้ว

นาเดีย: Pricing และ Promotion

เลขบรรทัดอ้างอิงจากโค้ด commit 9990a12 โดยพารของโค้ดหลักในตารางเริ่มจาก
code/src/main/java/com/example/petshotel/

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
S Single Respons ibility	service/impl/Promoti onServiceImpl.java:5 5-65, mapper/PromotionMapp er.java:11-23	PromotionServiceImpl ตรวจเงื่อนไขและบันทึกโปรโม ชัน ส่วน PromotionMapper แปลง Entity เป็น PromotionResponse	แยกงานจัดการโปรโมชั่นออก จากงานแปลงข้อมูล หากรูปแบบ Response เปลี่ยน จุดหลักที่ต้องแก้คือ Mapper โดยไม่ต้องเปลี่ยนกฎตรวจสอบ ลด
O Open/Cl osed	pricing/PricingStrat egy.java:7-9, service/impl/Pricing ServiceImpl.java:56- 71, service/impl/Pricing ServiceImpl.java:89- 93, pricing/LongStayDisc ountStrategy.java:7- 8	Service วนเรียก Strategy ผ่าน interface และจัดผลตาม category() โดยไม่ตรวจสอบชื่อ Concrete Strategy แต่ละคลาสลงทะเบียนเป็น Spring bean	สามารถเพิ่มสูตรในหมวดเดิม เช่นส่วนลดอีกแบบ โดยเพิ่มคลาสที่ implements PricingStrategy และลงทะเบียนเป็น bean โดยไม่ต้องเพิ่มเงื่อนไขชื่อคลา สใน Service ทั้งนี้การเพิ่มหมวดราคาใหม่อาจ ต้องปรับการรวมผล
L Liskov Substitu tion	pricing/PricingStrat egy.java:7-9, pricing/LongStayDisc ountStrategy.java:15 -28, pricing/PromotionDis countStrategy.java:1 5-48, service/impl/Pricing ServiceImpl.java:89- 91	ส่วนลดพักระยะยาวและส่วนล ดโปรโมชั่น implements PricingStrategy เหมือนกัน รับ Context กับยอดก่อนหักส่วนลด และคืนจำนวนเงินส่วนลด โดย Service เรียกผ่าน interface เดียวกัน	สำหรับข้อมูลที่ถูกต้อง ทั้งสองคลาสทำหน้าที่ตามสัญ ญาณการคำนวณส่วนลดได้โดยผู้ เรียกไม่ต้องรู้ชนิดคลาส การเพิ่ม implementation ใหม่ต้องรักษาเงื่อนไขการรับข ้อมูลและความหมายของผลลัพธ์ ร่วมกัน การ implements interface เพียงอย่างเดียวไม่ได้พิสูจน์

หลักการ	ไฟล์ : บรรทัด	สิ่งที่เห็นในโค้ด	เหตุผล
			LSP ครบทุกกรณี
I Interfac e Segregat ion	pricing/PricingStrategy.java:7-9, pricing/LongStayDiscountStrategy.java:15 , pricing/LongStayDiscountStrategy.java:27 , service/impl/PricingServiceImpl.java:57-58	PricingStrategy มีเพียง calculate() และ category() ซึ่งเกี่ยวข้องกับการคำนวณราคา และ Service ใช้งานทั้งสองเมธอด	Strategy แต่ละคลาสไม่ต้อง implements เมธอดจัดการโปรโมชั่น บันทึกฐานข้อมูล หรือออกใบเสร็จที่ไม่เกี่ยวข้อง กับหน้าที่ของตน
D Depend ency Inversio n	service/impl/PricingServiceImpl.java:19-23, pricing/PricingStrategy.java:7-9, pricing/LongStayDiscountStrategy.java:7-8	PricingServiceImpl รับ List<PricingStrategy> ผ่าน constructor โดยพึ่ง interface และไม่ได้สร้าง Concrete Strategy ด้วย new เอง ส่วนคลาส Strategy ลงทะเบียนด้วย @Component	งานประสานการคำนวณพึ่ง abstraction ของกฎราคา โดย Spring ส่ง implementation เข้ามา ทำให้เปลี่ยนหรือเพิ่ม Strategy และจัดชุด dependency สำหรับทดสอบได้

ตัวอย่างการประยุกต์

- SRP: หากเพิ่มข้อมูลใน **PromotionResponse** ให้ปรับการแปลงข้อมูลใน **PromotionMapper** ส่วนกฎตรวจสอบมูลค่าส่วนลดยังคงอยู่ใน **PromotionServiceImpl**
- OCP: หากต้องการเพิ่มส่วนลดสมาชิก สามารถสร้าง Strategy ใหม่ในหมวด **DISCOUNT** โดยระบบจะนำผลไปเปรียบเทียบกับส่วนลดเดิมตามกติกาเลือกค่าสูงสุด
- LSP: Strategy ส่วนลดใหม่ควรรับ Context และยอดก่อนหักส่วนลดที่ถูกต้อง
คืนจำนวนเงินส่วนลดเป็น **BigDecimal** และคืน 0 เมื่อไม่เข้าเงื่อนไขของส่วนลดนั้น
- ISP: คลาสคำนวณส่วนลดรับผิดชอบเฉพาะ **calculate()** และ **category()**
ไม่ต้องมีเมธอดสร้างหรือแก้ไขโปรโมชั่น
- DIP: ในการทดสอบสามารถส่งรายการ Strategy เข้า constructor ของ **PricingServiceImpl** ได้โดยตรง โดยไม่ต้องให้ Service สร้างคลาสเหล่านั้นเอง

หลักฐานเพิ่มเติมจากไฟล์ทดสอบ

ไฟล์

code/src/test/java/com/example/petshotel/service/PricingServiceTest.java มีตัวอย่างที่เกี่ยวข้อง:

- บรรทัด 31–37 จัดชุด Concrete Strategy ทั้งห้าและส่งเข้า constructor ของ **PricingServiceImpl**
- บรรทัด 94–116 ตรวจสอบกรณีเลือกส่วนลดโปรโมชั่นที่สูงกว่าส่วนลดพักระยะยาว
- บรรทัด 149–165 ตรวจสอบกรณีใช้ส่วนลดพักระยะยาวเมื่อไม่มีโปรโมชั่น

ตัวอย่างเหล่านี้แสดงการใช้งาน Strategy

ผ่านกลไกเดียวกันและระบุผลลัพธ์ที่คาดหวังสำหรับบางกรณี แต่ไม่ได้พิสูจน์พฤติกรรมของทุก implementation สำหรับข้อมูลทุกแบบ

3.12 การออกแบบ REST API

REST API อยู่ใต้ /api ทุก endpoint มีเอกสารใน Swagger UI ที่ /swagger-ui.html คำขอที่แก้ไขข้อมูลต้องแนบ CSRF token และต้องเข้าสู่ระบบตามสิทธิ์ที่กำหนด

ตารางที่ 3.5 รายการ REST API

Method	Endpoint	คำอธิบาย
GET	/api/rooms	ดูห้องทั้งหมด
GET	/api/rooms/{id}	ดูรายละเอียดห้อง
POST	/api/rooms	เพิ่มห้องใหม่ (201)
PUT	/api/rooms/{id}	แก้ไขข้อมูลห้อง
PATCH	/api/rooms/{id}/status	เปลี่ยนสถานะห้อง
PATCH	/api/rooms/{id}/deactivate	ปิดใช้งานห้อง
GET	/api/rooms/available	ค้นหาห้องว่างตามช่วงวันและจำนวนสัตว์
GET	/api/rooms/{roomId}/availability-calendar	ปฏิทินห้องว่างรายวันของห้อง (ไม่เกิน 366 วัน)
GET	/api/rooms/search	ค้นหาห้องแบบแบ่งหน้าและเรียงลำดับ (keyword, minCapacity, maxPrice, page, size, sort)
POST	/api/bookings	สร้างการจอง (201)
POST	/api/bookings/preview-price	คำนวณราคาก่อนจอง
GET	/api/bookings/pet-availability	ดูสัตว์ที่ติดจองในช่วงวันที่

Method	Endpoint	คำอธิบาย
GET	/api/bookings	ดูการจองทั้งหมด
GET	/api/bookings/{id}	ดูรายละเอียดการจอง
POST	/api/bookings/{id}/confirm	ยืนยันการจอง
POST	/api/bookings/{id}/check-in	เช็คอิน
POST	/api/bookings/{id}/check-out	เช็คเอาท์
POST	/api/bookings/{id}/cancel	ยกเลิกการจอง
POST	/api/bookings/{id}/payment/confirm	ยืนยันรับชำระเงิน
POST / GET / PUT / DELETE	/api/owners/{ownerId}/pets/{petId}	จัดการสัตว์เลี้ยงของเจ้าของ
POST / GET / PUT	/api/care-reports/...	เขียน ดู และแก้ไขรายงานการดูแล
GET / POST / PUT / DELETE	/api/extra-services/{id}	จัดการบริการเสริม
GET / POST / PUT / DELETE	/api/promotions/{id}, /api/promotions/active	จัดการโปรโมชั่น และดูโปรโมชั่นที่ใช้ได้

3.12.1 การจัดการข้อผิดพลาด

GlobalExceptionHandler (@RestControllerAdvice เฉพาะ package controller.api) แปลง exception เป็น HTTP status และส่งกลับในรูปแบบ ErrorResponse เดียวกันทุก endpoint ประกอบด้วย timestamp, status, error, message, path และ fieldErrors (กรณีข้อมูลไม่ถูกต้อง) ทำให้ Controller ไม่ต้องดักจับ error เอง

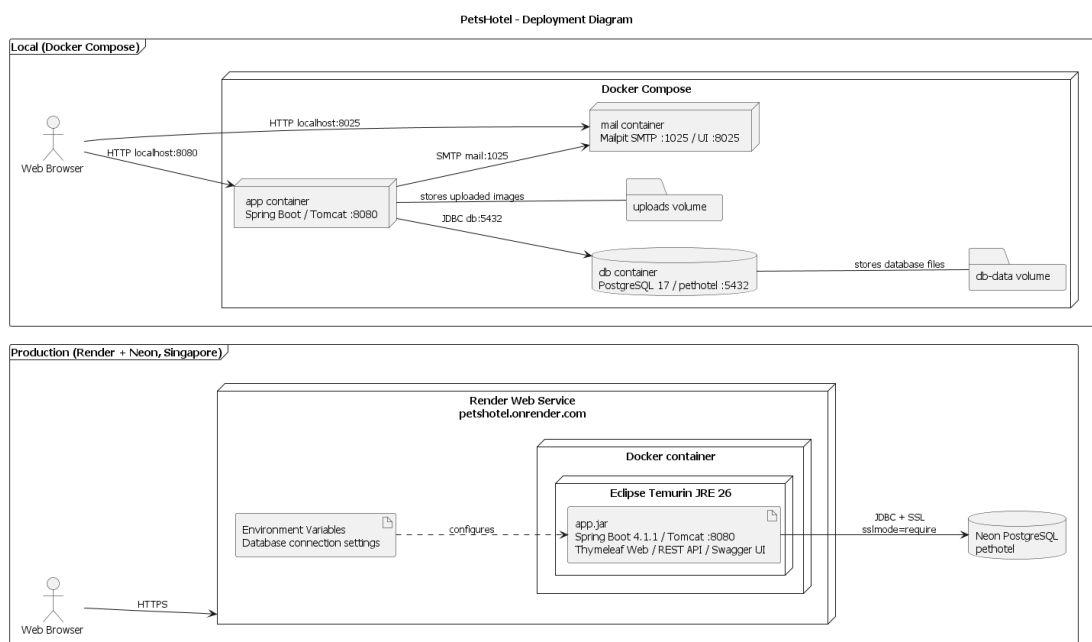
ตารางที่ 3.6 การแปลง Exception เป็น HTTP Status

Exception	HTTP Status
ResourceNotFoundException	404 Not Found
RoomNotAvailableException, DuplicateResourceException, IllegalStateException (และคลาสลูก เช่น PetNotAvailableException)	409 Conflict
IllegalArgumentException, BindException / MethodArgumentNotValidException, คำขอผิดรูปแบบ	400 Bad Request
AccessDeniedException	403 Forbidden
ResponseStatusException	ตาม status ที่กำหนด
Exception อื่น ๆ	500 Internal Server Error

3.13 การออกแบบความปลอดภัย

- เข้าสู่ระบบด้วยอีเมลและรหัสผ่านแบบ session รหัสผ่านเข้ารหัสด้วย BCrypt ผ่าน CustomUserDetailsService
- Remember-me 30 วัน และออกจากระบบผ่านหน้าบัญชีของฉันพร้อมการยืนยัน
- ป้องกัน CSRF ด้วย CookieCsrfTokenRepository ทั้งฟอร์มและ REST API
- กำหนดสิทธิ์ตามบทบาทใน SecurityConfig และซ่อนเมนูด้วย sec:authorize ใน Thymeleaf
- RoleBasedLoginSuccessHandler พาไปหน้าที่ขอไว้ก่อนเข้าสู่ระบบ (ไม่นับ /error, /.well-known/, /favicon) หรือหน้าตามบทบาท
- บัญชีที่ถูกกระงับ (active = false) เข้าสู่ระบบไม่ได้ และหน้า login แสดงข้อความแจ้ง
- หน้าบัญชีของฉันใช้อีเมลจาก Principal ไม่รับ id จาก URL เพื่อป้องกันการดูข้อมูลของผู้อื่น

3.14 การออกแบบการติดตั้งใช้งาน



รูปที่ 3.13 Deployment Diagram

บนเครื่องผู้พัฒนาใช้ Docker Compose รัน 3 service ได้แก่ แอป PetsHotel, PostgreSQL และ Mailpit (ดูอีเมลที่ระบบส่ง) ส่วนระบบจริงรันบน Render ซึ่ง build อิมเมจจาก

code/Dockerfile และเชื่อมต่อฐานข้อมูล Neon PostgreSQL (Singapore)
ค่าความลับทั้งหมดตั้งเป็น environment variable บน Render

บทที่ 4

ผลการดำเนินงาน

4.1 สภาพแวดล้อมการพัฒนาและการติดตั้ง

ตารางที่ 4.1 สภาพแวดล้อมการพัฒนา

รายการ	รายละเอียด
JDK	Java 26
Build tool	Maven Wrapper (mvnw)
ฐานข้อมูล	PostgreSQL (local / Docker), Neon PostgreSQL (production)
IDE	VS Code / IntelliJ IDEA
รันด้วย Docker	cd code แล้ว docker compose up --build → http://localhost:8080
รันบนเครื่อง	ตั้ง SPRING_DATASOURCE_PASSWORD แล้ว ./mvnw spring-boot:run
Swagger UI	http://localhost:8080/swagger-ui.html
Mailpit	http://localhost:8025
Live demo	https://petshotel.onrender.com
Source code	https://github.com/Tanut-oam/PetsHotel_Project

4.2 ผลการพัฒนาส่วนต่าง ๆ ของระบบ

ส่วนนี้แสดงหน้าจอการทำงานของระบบแยกตามบทบาทผู้ใช้

4.2.1 ส่วนของผู้เยี่ยมชมและลูกค้า

หน้าเข้าสู่ระบบ (/login)

ผู้ใช้กรอกอีเมลและรหัสผ่าน ระบบจดจำการเข้าสู่ระบบไว้ 30 วัน โดยอัตโนมัติ
จนกว่าผู้ใช้จะออกจากระบบ หากบัญชีถูกระงับ ระบบจะแจ้งข้อความ

รูปที่ 4.1 หน้าเข้าสู่ระบบ (/login)

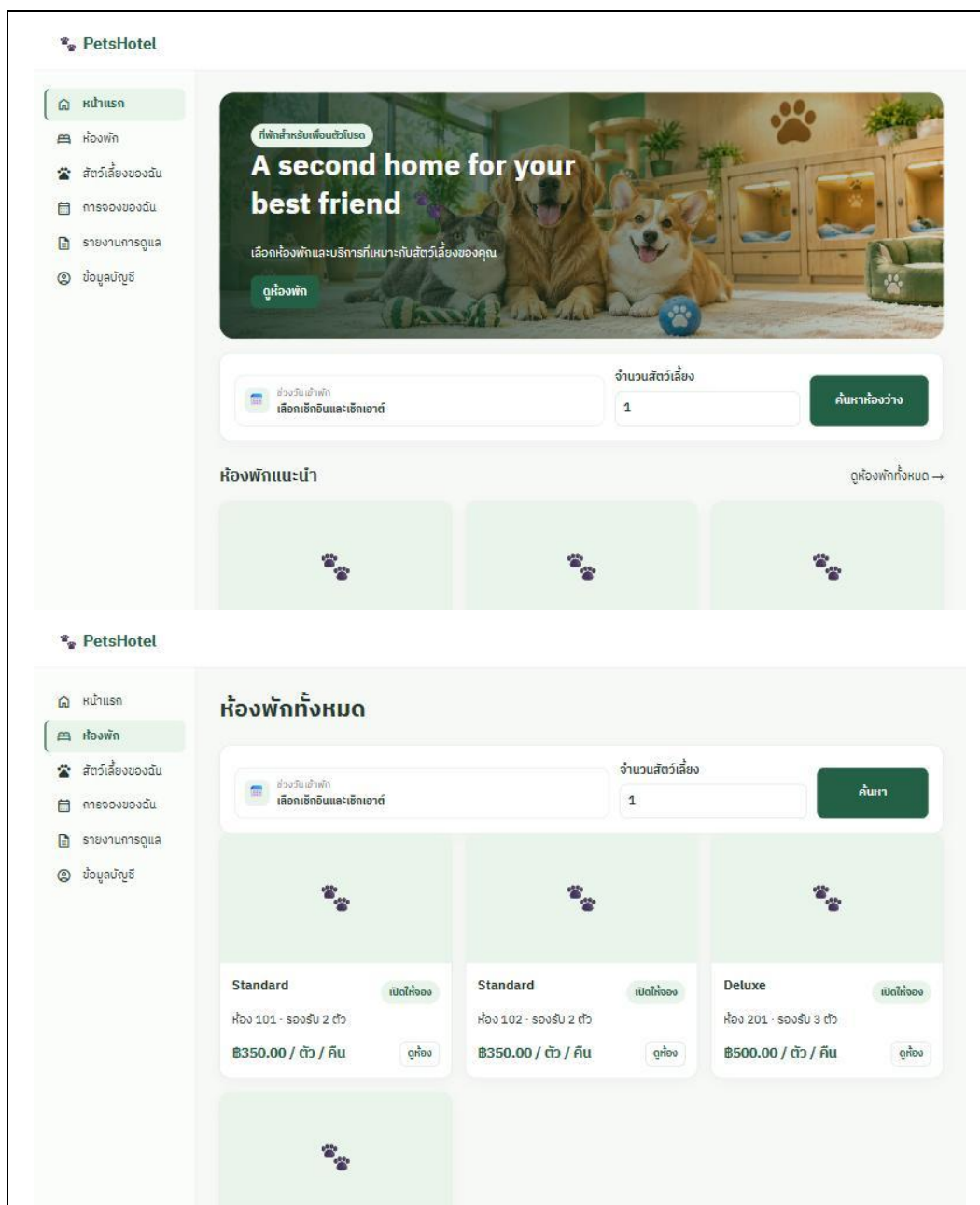
หน้าสมัครสมาชิก (/register)

ตรวจสอบรูปแบบอีเมล เบอร์โทร รหัสผ่านอย่างน้อย 8 ตัว และอีเมลต้องไม่ซ้ำ

รูปที่ 4.2 หน้าสมัครสมาชิก (/register)

หน้าแรกและรายการห้องพัก (/ , /rooms)

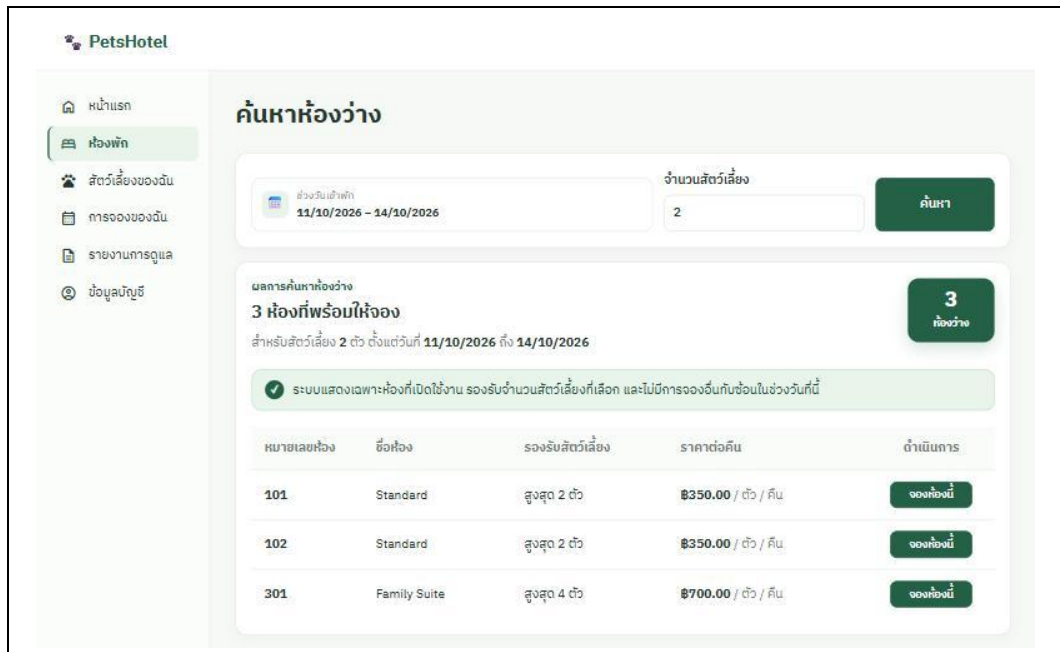
แสดงห้องพัก รูป ความจุ และราคาต่อสัตว์ต่อคืน



รูปที่ 4.3 หน้าแรกและรายการห้องพัก (/ , /rooms)

หน้าค้นหาห้องพัก (/rooms/available)

เลือกวันเช็กอิน วันเช็กเอาท์ และจำนวนสัตว์ ระบบแสดงเฉพาะห้องที่ว่างพร้อมปุ่ม “จองห้องนี้” หากยังไม่เข้าสู่ระบบจะพาไปหน้า login แล้วกลับมาที่ฟอร์มจองพร้อมข้อมูลเดิม



ค้นหาห้องพัก

ช่วงวันเข้าพัก: 11/10/2026 - 14/10/2026 จำนวนสัตว์เลี้ยง: 2 **ค้นหา**

ผลการค้นหาห้องพัก
3 ห้องที่พร้อมให้จอง
สำหรับสัตว์เลี้ยง 2 ตัว ตั้งแต่วันที่ 11/10/2026 ถึง 14/10/2026

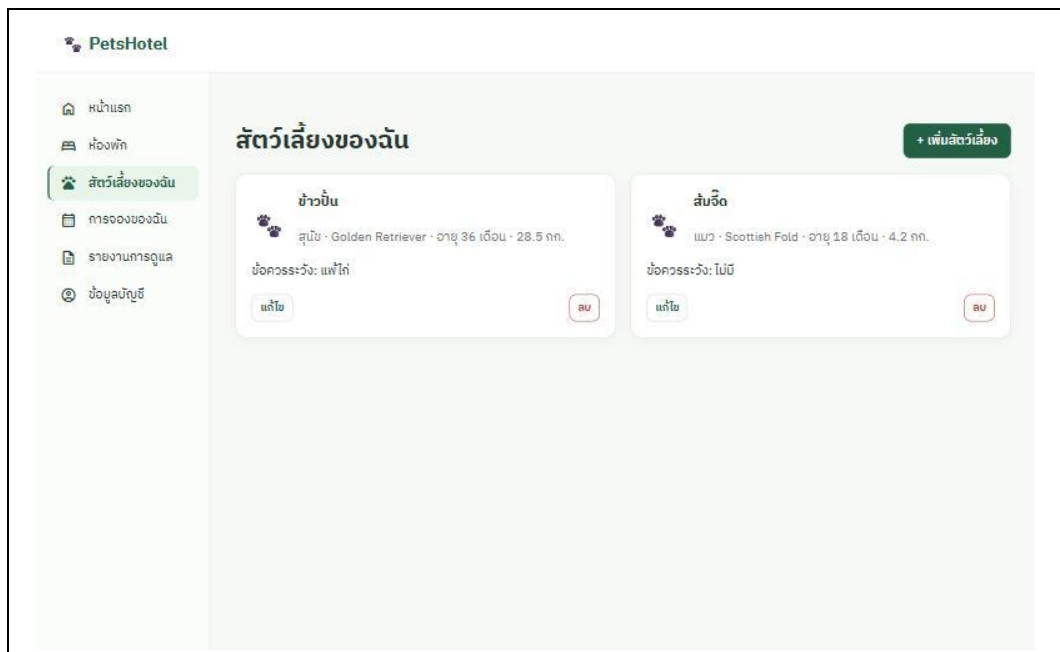
✓ ระบบแสดงเฉพาะห้องพักที่เปิดใช้งาน รองรับจำนวนสัตว์เลี้ยงที่เลือก และไม่มีการจองอื่นทับซ้อนในช่วงวันที่นี้

หมายเลขห้อง	ชื่อห้อง	รองรับสัตว์เลี้ยง	ราคาต่อคืน	ดำเนินการ
101	Standard	สูงสุด 2 ตัว	฿350.00 / ตัว / คืน	จองห้องนี้
102	Standard	สูงสุด 2 ตัว	฿350.00 / ตัว / คืน	จองห้องนี้
301	Family Suite	สูงสุด 4 ตัว	฿700.00 / ตัว / คืน	จองห้องนี้

รูปที่ 4.4 หน้าค้นหาห้องพัก (/rooms/available)

หน้าสัตว์เลี้ยงของฉัน (/pets)

เพิ่ม แก้ไข อัปเดตรูป และปิดใช้งานสัตว์เลี้ยง



สัตว์เลี้ยงของฉัน + เพิ่มสัตว์เลี้ยง

ข้าวปั้น
สุนัข · Golden Retriever · อายุ 36 เดือน · 28.5 กก.
ชื่อควรระวัง: แพ้ไก่
แก้ไข **ลบ**

ส้มจี๊ด
แมว · Scottish Fold · อายุ 18 เดือน · 4.2 กก.
ชื่อควรระวัง: ไม่มี
แก้ไข **ลบ**

รูปที่ 4.5 หน้าสัตว์เลี้ยงของฉัน (/pets)

หน้าจองห้องพัก (/bookings/new)

เลือกสัตว์ บริการเสริม โปรโมชั่น และดูราคาก่อนยืนยัน

PetsHotel

หน้าแรก
ห้องพัก
สัตว์เลี้ยงของฉัน
การจองของฉัน
รายงานการดูแล
ข้อมูลบัญชี

สร้างการจอง

เลือกห้อง สัตว์เลี้ยง และบริการก่อนยืนยันการจอง

1. ห้องและวันที่ 2. สัตว์เลี้ยง 3. บริการเสริม 4. ตรวจสอบข้อมูล

บริการเสริมและโปรโมชั่น

คิดราคาต่อสัตว์ที่เลือกตลอดการจอง ไม่ดูเบจำนวนคืน

☒ **อาบน้ำ** – 300.00 บาท/ตัว/การจอง

อาบน้ำ นวด และหรีดย

สัตว์เลี้ยงรับบริการ

☒ **ข้าวปั้น**

☐ **พาเดินเล่น** – 150.00 บาท/ตัว/การจอง

พาเดินเล่นนอกห้อง 30 นาที

สรุปการเลือก

ห้องพัก	Standard
จำนวนคน	3 คน
จำนวนสัตว์	1 ตัว
อาบน้ํา: ข้าวปั้น	
ค่าห้องพัก	1,050.00 บาท
บริการเสริม	300.00 บาท
ค่าธรรมเนียมเพิ่มเติม	0.00 บาท
ส่วนลด	0.00 บาท
ยอดรวมประมาณการ	1,350.00 บาท

คำนวณจากตัวเลือกปัจจุบันแล้ว ยังไม่ได้จองหรือชำระเงิน

หมายเหตุ: ไม่ได้คิดค่าธรรมเนียมห้อง 30 นาที

รูปที่ 4.6 หน้าจองห้องพัก (/bookings/new)

หน้าการจองของฉันและรายละเอียดการจอง

ดูสถานะ ยกเลิกการจองที่ยังยกเลิกได้ และดูใบเสร็จ

PetsHotel

หน้าแรก
ห้องพัก
สัตว์เลี้ยงของฉัน
การจองของฉัน
รายงานการดูแล
ข้อมูลบัญชี

การจองของฉัน

สร้างการจอง

ค้นหาเลขที่ ห้องพัก หรือสัตว์เลี้ยง สถานะ

พิมพ์คำค้นหา ทั้งหมด

แสดง 2 จาก 2 รายการ

เลขที่	สัตว์เลี้ยง	ห้องพัก	วันเข้า-วันออก	สถานะ	การชำระเงิน	ยอดรวม	จัดการ
3	[ข้าวปั้น, สับจิต]	Family Suite	01/11/2026 – 07/11/2026	ยืนยันแล้ว	ยังไม่ชำระ	7740.00 บาท	รายละเอียด
1	[ข้าวปั้น]	Deluxe	10/10/2026 – 13/10/2026	เข้าพักแล้ว	ชำระแล้ว	1950.00 บาท	รายละเอียด

PetsHotel

หน้าแรก ห้องพัก สัตว์เลี้ยงของฉัน การจองของฉัน รายงานการดูแล ข้อมูลบัญชี

รายละเอียดการจอง #1

เข้าพักแล้ว ชำระแล้ว

ข้อมูลการจอง

เลขที่การจอง: 1

ห้องพัก: Deluxe

สัตว์เลี้ยง: ช้างปิ่น

วันเช็คอิน: 10/10/2026

วันเช็คเอาท์: 13/10/2026

สถานะ: [เข้าพักแล้ว](#)

การชำระเงิน: [ชำระแล้ว](#)

โปรโมชั่นตอนจอง: ไม่ใช้โปรโมชั่น

รายละเอียดราคา

ค่าห้องพัก	1500.00 บาท
ค่าบริการเสริม	450.00 บาท
ค่าธรรมเนียมเพิ่เติม	0.00 บาท
ส่วนลด	0.00 บาท
ยอดรวม	1950.00 บาท

บริการเสริมที่จอง

บริการ	สัตว์ที่รับบริการ	ราคาต่อหน่วย (บาท)	รวม (บาท)
อาบน้ำ	ช้างปิ่น	300.00	300.00

รูปที่ 4.7 หน้าการจองของฉันและรายละเอียดการจอง

หน้ารายงานการดูแล (/reports)

ดูรายงานการดูแลรายวันของสัตว์เลี้ยงของฉัน

PetsHotel

หน้าแรก ห้องพัก สัตว์เลี้ยงของฉัน การจองของฉัน รายงานการดูแล ข้อมูลบัญชี

← กลับไปรายการรายงาน

รายงานการดูแลของ ช้างปิ่น

การจอง #1 - 1 รายงาน

วันที่ 10/10/2026 การจอง #1

อาหาร	กิจกรรมและความสะอาด	อารมณ์และสุขภาพ
เช้า: กินหมด	เดินเล่น: 30 นาที	อารมณ์: ร่าเริง ชอบเล่นบอล
เย็น: กินหมด	ดูแลความสะอาด: อาบน้ำและป้อนนม	สุขภาพ: สุขภาพปกติ
หมายเหตุเพิ่มเติม: ปรับตัวกับห้องได้ดี		

รูปที่ 4.8 หน้ารายงานการดูแล (/reports)

หน้าบัญชีของฉัน (/profile)

ดูข้อมูลบัญชี (ชื่อ อีเมล และเบอร์โทร) และออกจากระบบพร้อมกล่่องยืนยัน

รูปที่ 4.9 หน้าบัญชีของฉัน (/profile)

4.2.2 ส่วนของพนักงาน

หน้าจัดการการจอง (/admin/bookings)

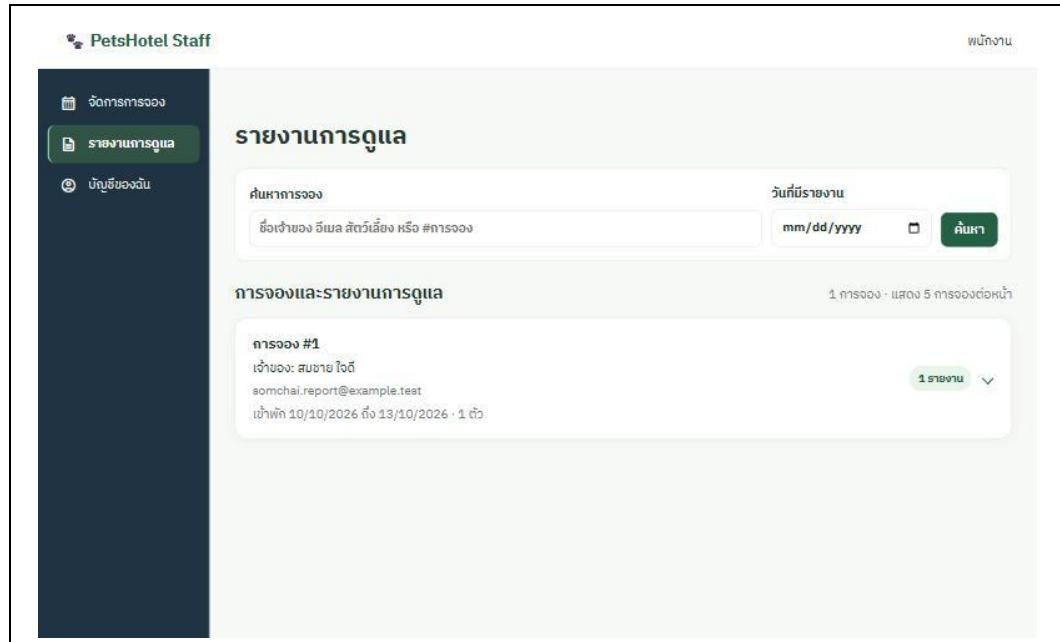
ยืนยัน เช็กอิน เช็กเอาท์ ยกเลิก และยืนยันรับชำระเงิน

การจอง	ห้องพักและสัตว์เลี้ยง	ช่วงวันเข้าพัก	สถานะ	ยอดรวม	ดำเนินการ
การจอง # 2 ยกเลิกการจอง: 5	Standard สัตว์เลี้ยง: [ไม่มี]	เช็คอิน 20/10/2026 → เช็คเอาท์ 22/10/2026	รอยืนยัน ยังไม่ชำระ	฿700.00	ดูรายละเอียด ยืนยันการจอง ยกเลิก
การจอง # 1 ยกเลิกการจอง: 4	Deluxe สัตว์เลี้ยง: [ไม่มี]	เช็คอิน 10/10/2026 → เช็คเอาท์ 13/10/2026	เข้าพักแล้ว ชำระแล้ว	฿1,950.00	ดูรายละเอียด ดูใบเสร็จ แจ้งเลิกเช่า เช็คเอาท์ได้ 13/10/2026

รูปที่ 4.10 หน้าจัดการการจอง (/admin/bookings)

หน้ารายงานการดูแล (/admin/reports)

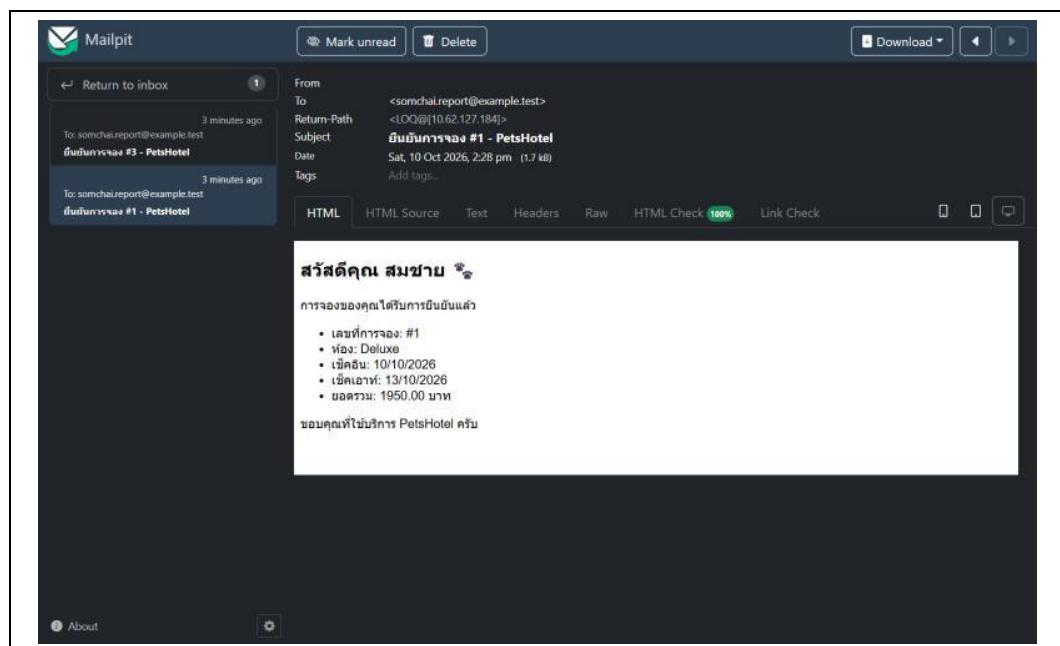
บันทึกและแก้ไขรายงานการดูแลของสัตว์ที่เข้าพัก



รูปที่ 4.11 หน้ารายงานการดูแล (/admin/reports)

อีเมลยืนยันการจองใน Mailpit

อีเมลที่ระบบส่งเมื่อพนักงานยืนยันการจอง (Observer Pattern)

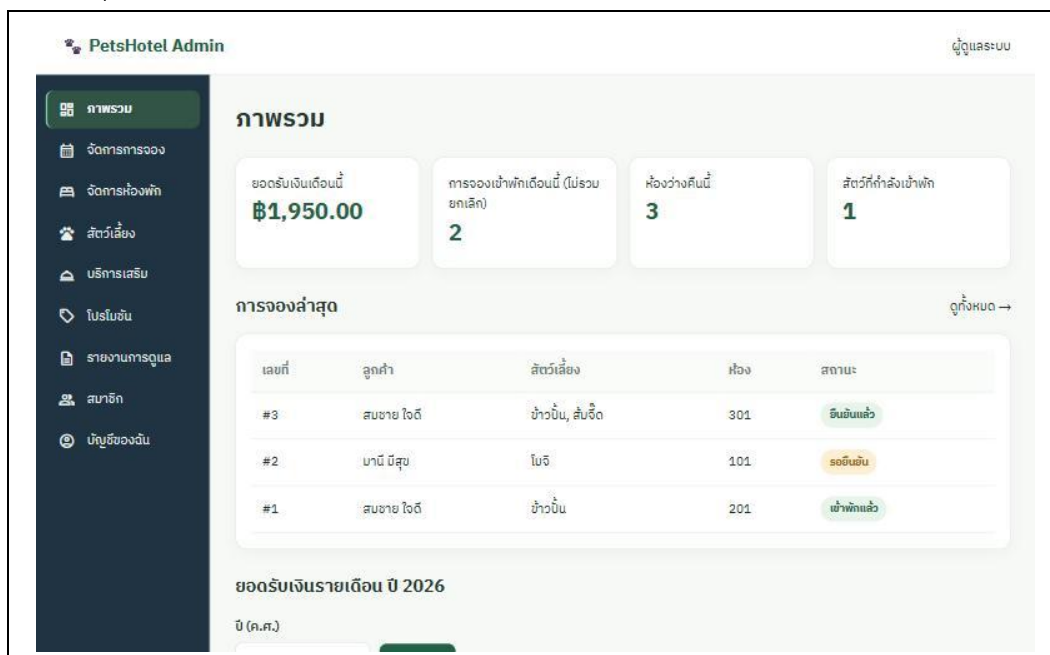


รูปที่ 4.12 อีเมลยืนยันการจองใน Mailpit

4.2.3 ส่วนของผู้ดูแลระบบ

หน้า Dashboard รายได้ (/admin)

สรุปรายได้และสถิติการจอง



รูปที่ 4.13 หน้า Dashboard รายได้ (/admin)

หน้าจัดการห้องพัก

เพิ่ม แก้ไข เปลี่ยนสถานะ และปิดใช้งานห้อง

The screenshot shows the 'จัดการห้องพัก' (Manage Rooms) page in the 'PetsHotel Admin' dashboard. It features a table with columns: หมายเลขห้อง, ชื่อห้อง, ราคา/ตัว/คืน, รองรับ, สถานะ, and จัดการ. There are buttons for '+ เพิ่มห้อง' and '+ ปิดใช้งาน' at the top right. The table lists four room types: 101 (Standard, ฿350.00, 2 ตัว, เปิดห้อง), 102 (Standard, ฿350.00, 2 ตัว, เปิดห้อง), 201 (Deluxe, ฿500.00, 3 ตัว, เปิดห้อง), and 301 (Family Suite, ฿700.00, 4 ตัว, เปิดห้อง). Each row has buttons for 'แก้ไข', 'เปิด/ปิดห้อง', and 'เปลี่ยนสถานะ'.

หมายเลขห้อง	ชื่อห้อง	ราคา/ตัว/คืน	รองรับ	สถานะ	จัดการ
101	Standard	฿350.00	2 ตัว	เปิดห้อง	แก้ไข, เปิด/ปิดห้อง, เปลี่ยนสถานะ
102	Standard	฿350.00	2 ตัว	เปิดห้อง	แก้ไข, เปิด/ปิดห้อง, เปลี่ยนสถานะ
201	Deluxe	฿500.00	3 ตัว	เปิดห้อง	แก้ไข, เปิด/ปิดห้อง, เปลี่ยนสถานะ
301	Family Suite	฿700.00	4 ตัว	เปิดห้อง	แก้ไข, เปิด/ปิดห้อง, เปลี่ยนสถานะ

รูปที่ 4.14 หน้าจัดการห้องพัก

หน้าจัดการโปรโมชั่น

เพิ่ม แก้ไข และปิดใช้งานโปรโมชั่น

PetsHotel Admin ผู้ดูแลระบบ

โปรโมชั่น + เพิ่มโปรโมชั่น

ค้นหาชื่อ: สถานะเปิดใช้งาน: ทั้งหมด ▼ ล้างตัวกรอง

ชื่อโปรโมชั่น	ส่วนลด	วันหมดอายุ	สถานะ	จัดการ
พักยาวลด 10%	10.00%	31/12/2027	เปิดใช้	แก้ไข
ลด 100 บาท	฿100.00	31/12/2027	เปิดใช้	แก้ไข

รูปที่ 4.15 หน้าจัดการโปรโมชั่น

หน้าจัดการบริการเสริม

เพิ่ม แก้ไข และปิดใช้งานบริการเสริม

PetsHotel Admin ผู้ดูแลระบบ

บริการเสริม + เพิ่มบริการ

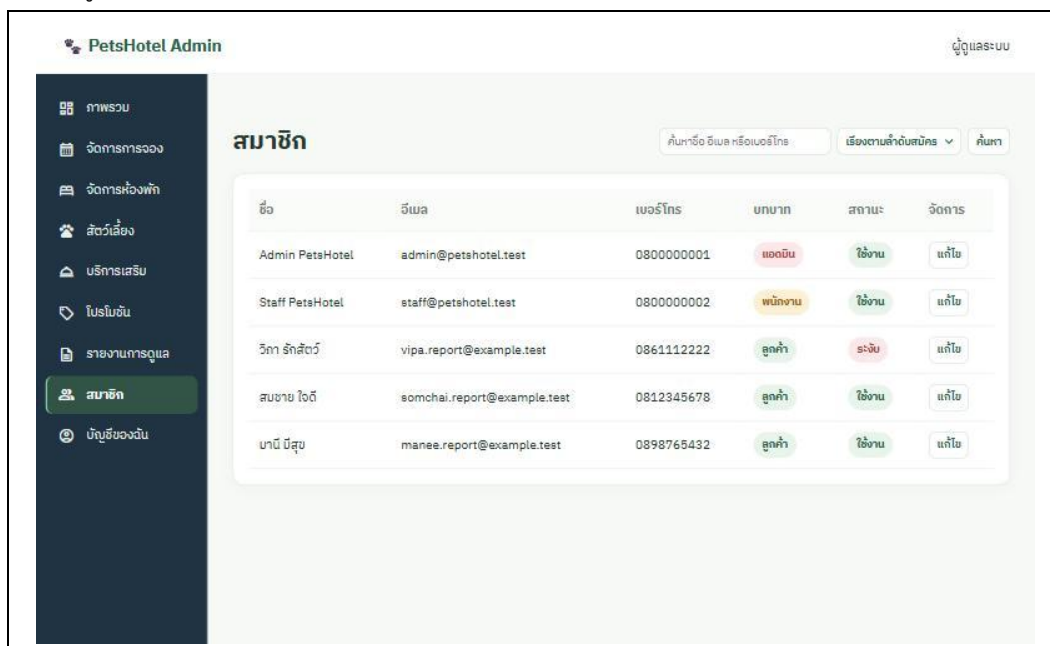
ค้นหาชื่อ: สถานะเปิดใช้งาน: ทั้งหมด ▼ ล้างตัวกรอง

ชื่อบริการ	ราคา/ตัว/การจอง	สถานะ	จัดการ
อาบน้ำ	฿300.00	เปิดใช้	แก้ไข
พาเดินเล่น	฿150.00	เปิดใช้	แก้ไข
ตัดเล็บ	฿200.00	เปิดใช้	แก้ไข
ตรวจสุขภาพ	฿500.00	เปิดใช้	แก้ไข

รูปที่ 4.16 หน้าจัดการบริการเสริม

หน้าจัดการสมาชิก

ดู แก้ไขบทบาท ระบุ และเปิดใช้บัญชี



รูปที่ 4.17 หน้าจัดการสมาชิก

4.3 ผลการพัฒนา REST API

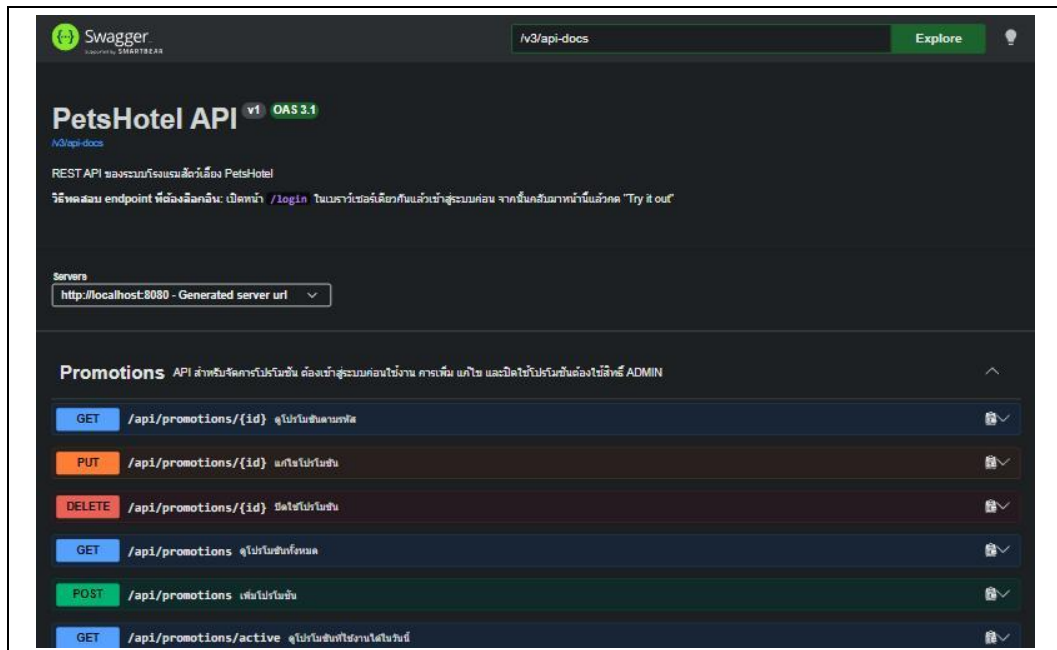
ทุก endpoint ในหัวข้อ 3.12 แสดงและทดลองเรียกได้ใน Swagger UI

ตัวอย่างการค้นหาห้องแบบแบ่งหน้า GET

/api/rooms/search?keyword=&minCapacity=2&page=0&size=10&sort=pricePerPetPerNight,desc ระบบแบ่งหน้าที่ฐานข้อมูลด้วย Pageable จำกัดไม่เกิน 50

รายการต่อหน้า และเรียงได้เฉพาะ roomNumber, name, capacity, pricePerPetPerNight

หากส่งค่าไม่ถูกต้องจะตอบ 400 พร้อม ErrorResponse



รูปที่ 4.18 หน้า Swagger UI แสดงรายการ API



รูปที่ 4.19 ตัวอย่างผลลัพธ์ JSON ของ GET /api/rooms/search และตัวอย่าง ErrorResponse (400/404/409)

4.4 ผลการทดสอบระบบ

โค้ดทดสอบอยู่ใน code/src/test/java ส่วนรายงานผลที่สร้างจากการรัน test เก็บไว้ในโฟลเดอร์ test/ ของ repository ผลด้านล่างรันกับโค้ดชุดเดียวกับที่ merge เข้า main เมื่อวันที่ 9 ตุลาคม 2569

สรุปผล

รายการ	ผล
Test class	42
Tests ทั้งหมด	458
ผ่าน	458
Failures / Errors / Skipped	0 / 0 / 0
Line coverage	84.7%
Branch coverage	70.9%

ประเภทการทดสอบ

ประเภท	Class	Tests
Service	15	188
REST Controller	8	97
Web Controller	13	144
Repository	2	9
Config / Security	2	8
Exception Handler	1	11
Application context	1	1

Last Published: 2026-10-08 | Version: 0.0.1-SNAPSHOT

Surefire Report

Summary

[Summary] [Package List] [Test Cases]

Tests	Errors	Failures	Skipped	Success Rate	Time
458	0	0	0	100%	17.85 s

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

Package List

[Summary] [Package List] [Test Cases]

Package	Tests	Errors	Failures	Skipped	Success Rate	Time
com.example.petshote!repository	9	0	0	0	100%	1.217 s
com.example.petshote!config	8	0	0	0	100%	4.283 s
com.example.petshote!service	188	0	0	0	100%	1.346 s
com.example.petshote!controller.web	144	0	0	0	100%	3.029 s
com.example.petshote!exception	11	0	0	0	100%	0.022 s
com.example.petshote!	1	0	0	0	100%	5.960 s
com.example.petshote!controller.api	97	0	0	0	100%	1.996 s

รูปที่ 4.20 สรุปผลการทดสอบ

Element	Missed Instructions	Cov	Missed Branches	Cov	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.example.petshote!service.impl	82%	71%	139	391	176	1,039	34	164	0	19		
com.example.petshote!controller.web	81%	75%	55	204	132	719	9	94	0	16		
com.example.petshote!notification	12%	n/a	4	8	30	36	4	8	0	3		
com.example.petshote!state	43%	n/a	13	25	14	30	13	25	1	5		
com.example.petshote!pricing	83%	58%	34	64	10	88	0	19	0	5		
com.example.petshote!domain.entity	66%	47%	18	30	11	36	2	8	2	4		
com.example.petshote!controller.api	96%	86%	4	68	6	174	0	53	0	9		
com.example.petshote!dto.response	95%	n/a	1	22	1	30	1	22	1	20		
com.example.petshote!	37%	n/a	1	2	2	3	1	2	0	1		
com.example.petshote!config	99%	85%	2	27	1	83	0	20	0	4		
com.example.petshote!exception	98%	100%	1	29	1	56	1	27	0	7		
com.example.petshote!mapper	100%	80%	4	35	0	171	0	25	0	10		
com.example.petshote!dto.request	100%	n/a	0	13	0	13	0	13	0	13		
com.example.petshote!domain.enums	100%	n/a	0	7	0	29	0	7	0	7		
Total	1,661 of 10,912	84%	251 of 863	70%	276	925	384	2,507	65	487	4	123

Created with JaCoCo 0.8.15.202006040825

รูปที่ 4.21 สรุป Code coverage จาก JaCoCo

4.4.1 ตัวอย่างกรณีทดสอบที่สำคัญ

ตารางที่ 4.2 ตัวอย่างกรณีทดสอบ

กรณีทดสอบ	ผลที่คาดหวัง	ผล
ค้นหาห้องที่มีการจอง CONFIRMED ทับช่วงวัน	ห้องนั้นไม่แสดงในผลลัพธ์	ผ่าน
ค้นหาโดยวันเช็กเอาต์ก่อนวันเช็กอิน	แจ้งข้อผิดพลาด / API ตอบ 400	ผ่าน
จองห้องที่ไม่ว่าง	API ตอบ 409 Conflict	ผ่าน
เช็กอินการจองที่ยังเป็น PENDING	ปฏิเสธการเปลี่ยนสถานะ (409)	ผ่าน
ยืนยันการจองแล้วการบันทึกสำเร็จ	ส่งอีเมลหลัง commit เท่านั้น	ผ่าน
พัก 5 คืนขึ้นไป และมีโปรโมชั่น	เลือกส่วนลดที่มากที่สุดรายการเดียว	ผ่าน
ลูกค้าเรียก API ของผู้ดูแล	403 Forbidden	ผ่าน

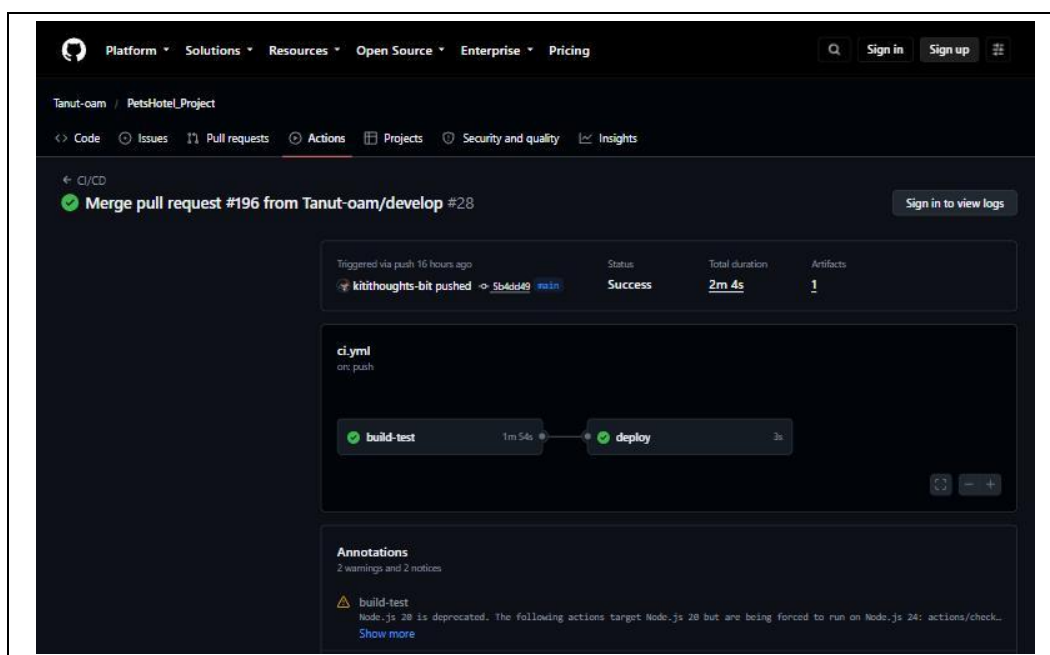
กรณีทดสอบ	ผลที่คาดหวัง	ผล
เข้าสู่ระบบด้วยบัญชีที่ถูกระงับ	แสดงข้อความบัญชีถูกระงับ	ผ่าน
สมัครด้วยอีเมลที่มีอยู่แล้ว	แสดง “อีเมลนี้ถูกใช้สมัครแล้ว”	ผ่าน

4.5 ผลการติดตั้งใช้งานและ CI/CD

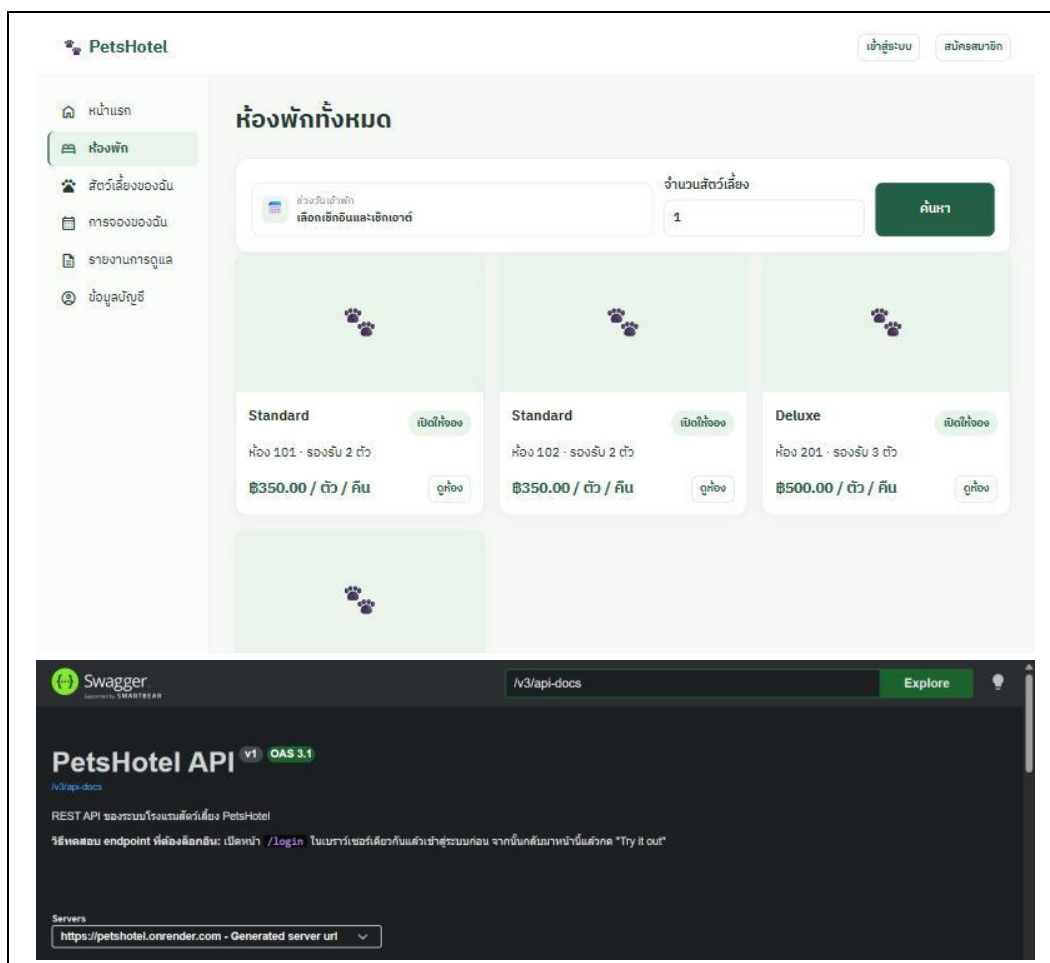
ระบบเปิดใช้งานจริงที่ <https://petshotel.onrender.com> (Render แบบฟรีจะหลับเมื่อไม่มีผู้ใช้ 15 นาที การเปิดครั้งแรกอาจรอประมาณ 1 นาที)

ไฟล์ `.github/workflows/ci.yml` กำหนด pipeline 2 งาน ได้แก่

1. `build-test`: ทำงานเมื่อ `push` หรือเปิด `pull request` เข้า `develop` หรือ `main` โดยเปิด PostgreSQL service ชั่วคราว รัน `bash mvnw -B package` (รวม test ทั้งหมด) เก็บรายงานผลการทดสอบ และ `docker build`
2. `deploy`: ทำงานเฉพาะเมื่อ `push` เข้า `main` และ `build-test` ผ่าน โดยเรียก Render Deploy Hook จาก GitHub Secret `RENDER_DEPLOY_HOOK_URL`



รูปที่ 4.22 ผลการรัน GitHub Actions ที่ผ่านทั้ง `build-test` และ `deploy`



รูปที่ 4.23 ระบบที่ deploy แล้วบน Render (petshotel.onrender.com)

4.6 การทำงานเป็นทีมด้วย Git

สมาชิกแต่ละคนพัฒนาบน branch ของตน (เช่น tanut_673380041-0_01) แล้วเปิด Pull Request เข้า branch develop โดย GitHub Actions จะ build และรัน test ให้อัตโนมัติในทุก Pull Request เมื่อพร้อมส่งงานจึงเปิด Pull Request จาก develop เข้า main แบบ merge commit เพื่อเก็บประวัติของทุกคน และ deploy อัตโนมัติ

ตารางที่ 4.3 Branch และผลงานของสมาชิก

สมาชิก	Branch	จำนวน commit / PR
นายกิตติธัช คำโท	kittithat_673380029-0_01	95 commit / 64 PR
นายณัฏฐ พนมเรงไชย	tanut_673380041-0_01	77 commit / 30 PR
นางสาวรติมา สวัสดิ์สินธุ์	ratima_673380055-9_01	95 commit / 34 PR
นางสาวกิติยาดา กองคำ	kityada_673380509-6_01	115 commit / 36 PR

สมาชิก	Branch	จำนวน commit / PR
นางสาวนาเดีย คิตอ่าน	nadia_673380513-5_01	86 commit / 31 PR

นับเฉพาะ commit ที่ไม่ใช่ merge commit บน branch main และ Pull Request ที่ merge แล้ว ณ วันที่ 9 ตุลาคม 2569 (commit 5b4dd49) โดยรวม commit ของรหัสที่ใช้ชื่อผู้ commit ต่างกัน 4 ชื่อ (Tanut-oam, Tanut_Ph, Tanut และ Tanu_Ph) เข้าด้วยกัน

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

คณะผู้จัดทำได้พัฒนาระบบ PetsHotel ซึ่งเป็นเว็บแอปพลิเคชันจัดการโรงแรมรับฝากสัตว์เลี้ยงครบตามวัตถุประสงค์ ระบบรองรับผู้ใช้ 3 บทบาท มี Use Case 16 รายการ ฐานข้อมูล 10 ตาราง และ REST API พร้อมเอกสาร Swagger สรุปผลตามวัตถุประสงค์ได้ดังนี้

ตารางที่ 5.1 สรุปผลตามวัตถุประสงค์

วัตถุประสงค์	ผลที่ได้
1. ระบบจองที่ป้องกันการจองซ้อน	ตรวจสอบความจุและการจองทับช่วงวันทั้งตอนค้นหาและตอนสร้างการจอง และตรวจสอบสัตว์ตัวเดียวกันไม่ให้ถูกจองซ้อน
2. ลูกค้าจองและติดตามการดูแลตัวเอง	ค้นหาห้องพักโดยไม่ต้องเข้าสู่ระบบ จองพร้อมดูราคา และดูรายงานการดูแลรายวันได้
3. พนักงาน/ผู้ดูแลจัดการได้จากที่เดียว	มีหน้าจัดการการจอง ห้อง โปรโมชั่น บริการเสริม สมาชิก และ Dashboard รายได้
4. ประยุกต์ใช้ Architecture, Pattern, SOLID	Layered Architecture, State, Strategy, Observer และวิเคราะห์ SOLID ครบทุก module
5. ฝึกกระบวนการพัฒนาเป็นทีม	ใช้ Git branch + Pull Request, test 458 กรณีผ่านทั้งหมด (Line 84.7%, Branch 70.9%), CI/CD และ deploy บน Render

5.2 อภิปรายผล

การใช้ State Pattern ทำให้กฎการเปลี่ยนสถานะการจองรวมอยู่ในคลาสของแต่ละสถานะ ไม่กระจายเป็น if/switch หลายจุด Strategy Pattern ทำให้เพิ่มกฎราคาใหม่ได้โดยไม่แก้ PricingServiceImpl และ Observer Pattern ร่วมกับการรับ event หลัง commit ทำให้อีเมลไม่ถูกส่งเมื่อการบันทึกล้มเหลว และอีเมลที่ส่งไม่สำเร็จไม่ทำให้การยืนยันการจองล้มเหลวตาม

การแยก interface ตามหลัก ISP เช่น RoomSearchService กับ AvailabilityService และการรวมการจัดการ error ไว้ที่ GlobalExceptionHandler ทำให้ API

ตอบรูปแบบเดียวกันทั้งระบบ และเพิ่ม exception ใหม่ได้โดยไม่ต้องแก้ handler การใช้ Dependency Injection ผ่าน interface ทำให้เขียน unit test ด้วย mock ได้ง่าย ส่งผลให้มี coverage สูง

เมื่อเทียบกับความคาดหวัง ตอนเริ่มโครงการ สิ่งทีกลุ่มทำได้ดี คือการแบ่ง module ตามสมาชิก ทำให้แต่ละคน พัฒนาและทดสอบส่วนของตน ได้ค่อนข้างอิสระ มีชุดทดสอบอัตโนมัติ 458 กรณี และ CI ช่วยตรวจพบปัญหา ก่อนขึ้นระบบจริง เช่น กรณีเพิ่มคอลัมน์ใน Entity แต่ยังไม่ migration ส่วนสิ่งที่ควรปรับ คือ ควรกำหนด branch protection ให้ทุก Pull Request ต้องมีผู้ review และผ่าน CI ก่อน merge ควรเพิ่มชุดทดสอบ ของส่วนรับชำระเงินและการแจ้งเตือน ซึ่งยังมี coverage ต่ำกว่าส่วนอื่น และควรตกลงรูปแบบเอกสารร่วมกัน ตั้งแต่ต้น เพื่อลดงานปรับแก้ในช่วงท้าย

5.3 ปัญหาและแนวทางแก้ไข

ตารางที่ 5.2 ปัญหาที่พบและแนวทางแก้ไข

ปัญหา	แนวทางแก้ไข
หลังเข้าสู่ระบบบางครั้งไปหน้า error (status 999) เพราะ Spring Security จำคำขอ /error ไว้	ให้ RoleBasedLoginSuccessHandler ข้าม /error, /.well-known/ และ /favicon และอนุญาตให้เข้าถึง /error
ผู้ดูแลระบบเปิดหน้าแรกแล้วเห็นหน้าของลูกค้า	HomeController พาผู้ดูแลไปหน้า /admin ตามบทบาท
โครงสร้างตารางของแต่ละคนไม่ตรงกันเมื่อรวมโค้ด	ใช้ Flyway migration และ ddl-auto=validate ให้ทุกเครื่องมือ schema เดียวกัน
รหัสผ่านฐานข้อมูลของแต่ละเครื่องต่างกัน และไม่ควรอยู่ใน repository	อ่านค่าจาก environment variable ทั้งบนเครื่อง, CI และ Render
โค้ดชนกัน (merge conflict) เมื่อหลายคนแก้ไขไฟล์เดียวกัน	แยก module ตามคน, pull develop บ่อย ๆ และรวมผ่าน Pull Request ที่ต้องผ่าน CI
ส่งอีเมลผ่าน SMTP บน Render ไม่ได้	ใช้ Mailpit ทดสอบบนเครื่อง และวางแผนเปลี่ยนเป็น Email API (เช่น Brevo)
เพิ่มฟิลด์ customerName ใน Entity Receipt แต่ไม่ได้เพิ่ม migration ทำให้ CI ล้มเหลว เพราะ ddl-auto=validate ตรวจพบว่าตาราง receipts ไม่มีคอลัมน์ customer_name	เพิ่มไฟล์ V3__add_receipt_customer_name.sql แทนการแก้ V1 ที่รันบนฐานข้อมูลจริงไปแล้ว และให้ CI ทดสอบกับฐานข้อมูลเปล่าทุกครั้ง
Deploy ครั้งแรกบน Neon ไม่ผ่าน เพราะฐานข้อมูลมีตารางที่ Hibernate	ล้าง schema บน Neon แล้วให้ Flyway สร้างตารางใหม่ตาม migration ตั้งแต่ V1

ปัญหา	แนวทางแก้ไข
สร้างไว้ก่อนใช้ Flyway (non-empty schema)	
รูปห้องและรูปสัตว์ที่อัปโหลดหายทุกครั้งที่ Render deploy ใหม่	อัปโหลดรูปใหม่หลัง deploy ครั้งสุดท้าย และเสนอแนวทางย้ายไปเก็บบน Cloud Storage

5.4 ข้อจำกัดของระบบ

- ยังไม่มีระบบชำระเงินจริง พนักงานเป็นผู้ยืนยันรับชำระเงินเอง
- ส่งอีเมลยังไม่ได้บน Render เนื่องจากข้อจำกัดของ SMTP บนบริการฟรี
- วันหยุดสำหรับคิดค่าธรรมเนียมกำหนดไว้ในโค้ด (hard-code)
- ยังไม่มีฟังก์ชันลิมิตส์ผ่าน
- รูปภาพเก็บบน disk ของเซิร์ฟเวอร์ ซึ่งบน Render เป็นพื้นที่ชั่วคราว อาจหายเมื่อ deploy ใหม่
- Render แบบฟรีกลับเมื่อไม่มีผู้ใช้ 15 นาที การเปิดครั้งแรกจึงช้า

5.5 ข้อเสนอแนะและแนวทางพัฒนาต่อ

- เชื่อมต่อการชำระเงินผ่าน PromptPay QR หรือ Payment Gateway เช่น Omise / Stripe
- ส่งอีเมลผ่าน Email API (เช่น Brevo) แทน SMTP
- สร้างตาราง holidays ให้ผู้ดูแลระบบกำหนดวันหยุดเอง
- เพิ่มการรีเซ็ตรหัสผ่านทางอีเมล
- ย้ายการเก็บรูปภาพไปที่ Cloud Storage เช่น Cloudinary หรือ Amazon S3
- เพิ่มการแจ้งเตือนช่องทางอื่น เช่น LINE หรือ SMS โดยเพิ่ม Listener ใหม่ตาม Observer Pattern
- เพิ่มชุดทดสอบของส่วนรับชำระเงิน (PaymentServiceImpl) และการแจ้งเตือน (notification) ซึ่งยังมี coverage ต่ำ และตั้ง branch protection ให้ทุก Pull Request ต้องผ่าน review และ CI ก่อน merge

บรรณานุกรม

- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.
- Martin, R. C. (2003). Agile Software Development, Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall.
- Docker Inc. (n.d.). Docker Documentation. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://docs.docker.com/>
- Gingr. (n.d.). Features. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://www.gingrapp.com/features>
- GitHub. (n.d.). GitHub Actions Documentation. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://docs.github.com/actions>
- JUnit Team. (n.d.). JUnit 5 User Guide. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://junit.org/junit5/docs/current/user-guide/>
- OpenAPI Initiative. (n.d.). OpenAPI Specification. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://spec.openapis.org/oas/latest.html>
- PostgreSQL Global Development Group. (n.d.). PostgreSQL Documentation. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://www.postgresql.org/docs/>
- Redgate. (n.d.). Flyway Documentation. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://documentation.red-gate.com/flyway>
- Revelation Pets. (n.d.). Effective pet-care management. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก <https://www.revelationpets.com/pet-care-software-pr>

Spring. (n.d.). Spring Boot Reference Documentation. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก
<https://docs.spring.io/spring-boot/>

Spring. (n.d.). Spring Security Reference. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก
<https://docs.spring.io/spring-security/reference/>

Thymeleaf. (n.d.). Tutorial: Using Thymeleaf. สืบค้นเมื่อ 10 ตุลาคม 2569, จาก
<https://www.thymeleaf.org/documentation.html>

ภาคผนวก ก

คำอธิบาย Use Case (Use Case Description)

UC-01 สมัครสมาชิก (Register)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้เยี่ยมชม (Guest)
คำอธิบาย	ผู้เยี่ยมชมสร้างบัญชีลูกค้าเพื่อใช้จองห้องพัก
เงื่อนไขก่อนเริ่ม	ยังไม่ได้เข้าสู่ระบบ
เงื่อนไขหลังจบ	มีบัญชีใหม่บทบาท CUSTOMER สถานะใช้งาน และรหัสผ่านถูกเข้ารหัสด้วย BCrypt

ขั้นตอนหลัก

1. ผู้เยี่ยมชมเปิดหน้าสมัครสมาชิก
2. กรอกชื่อ นามสกุล อีเมล เบอร์โทร รหัสผ่าน และยืนยันรหัสผ่าน
3. ระบบตรวจสอบความถูกต้องของข้อมูล
4. ระบบแปลงอีเมลเป็นตัวพิมพ์เล็กและตรวจว่ายังไม่ถูกใช้
5. ระบบบันทึกบัญชีบทบาท CUSTOMER พร้อมรหัสผ่านที่เข้ารหัสแล้ว
6. ระบบพาไปหน้าเข้าสู่ระบบพร้อมข้อความว่าสมัครสำเร็จ

กรณีผิดปกติ

- 3a. อีเมลผิดรูปแบบ, เบอร์โทรไม่ขึ้นต้นด้วย 0 หรือไม่ใช่ 9-10 หลัก, รหัสผ่านสั้นกว่า 8 ตัว → แสดงข้อผิดพลาดที่ช่องนั้น
- 3b. รหัสผ่านกับยืนยันรหัสผ่านไม่ตรงกัน → แสดง "รหัสผ่านไม่ตรงกัน"
- 4a. อีเมลถูกใช้แล้ว → แสดง "อีเมลนี้ถูกใช้สมัครแล้ว" ที่ช่องอีเมล

โค้ดที่เกี่ยวข้อง: `AuthController`, `AuthServiceImpl.register()`,

`RegisterRequest`

UC-02 เข้าสู่ระบบ (Log In)

หัวข้อ	รายละเอียด
Actor หลัก	ลูกค้า, พนักงาน, ผู้ดูแลระบบ
คำอธิบาย	ผู้ใช้นั่นตัวตนด้วยอีเมลและรหัสผ่าน แล้วระบบพาไปหน้าที่เหมาะกับบทบาท
เงื่อนไขก่อนเริ่ม	มีบัญชีที่ยังใช้งานอยู่
เงื่อนไขหลังจบ	ผู้ใช้เข้าสู่ระบบ ระบบจัดการเข้าสู่ระบบไว้ 30 วันจนกว่าจะออกจากระบบ

ขั้นตอนหลัก

1. ผู้ใช้เปิดหน้าเข้าสู่ระบบ แล้วกรอกอีเมลและรหัสผ่าน

2. ระบบค้นหาบัญชีด้วยอีเมลตัวพิมพ์เล็ก และเทียบรหัสผ่านกับ BCrypt hash
3. ระบบตรวจว่าบัญชียังใช้งานอยู่
4. ระบบพาไปหน้าตามบทบาท: ผู้ดูแลระบบ → /admin, พนักงาน → /admin/bookings, ลูกค้า → หน้าแรก

ทางเลือก

- 4a. ผู้ใช้ถูกพามาหน้าเข้าสู่ระบบเพราะเปิดหน้าที่ต้องล็อกอินก่อน → หลังเข้าสู่ระบบแล้วกลับไปหน้าที่ตั้งใจเปิด

กรณีผิดพลาด

- 2a. อีเมลหรือรหัสผ่านไม่ถูกต้อง → แจ้งข้อผิดพลาด
- 3a. บัญชีถูกระงับ (active = false) → เข้าสู่ระบบไม่ได้

โค้ดที่เกี่ยวข้อง: SecurityConfig, CustomUserDetailsService, RoleBasedLoginSuccessHandler

UC-03 ค้นหาห้องว่าง (Search Available Rooms)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้เยี่ยมชม, ลูกค้า
คำอธิบาย	ผู้ใช้ระบุช่วงวันเข้าพักและจำนวนสัตว์เลี้ยง ระบบแสดงห้องที่ว่างทั้งห้องตลอดช่วงนั้นและรับสัตว์ได้ตามจำนวน
เงื่อนไขก่อนเริ่ม	ไม่ต้องเข้าสู่ระบบ
เงื่อนไขหลังจบ	ผู้ใช้เห็นรายการห้องว่าง และกดจองห้องที่เลือกต่อได้ (UC-05)

ขั้นตอนหลัก

1. ผู้ใช้เปิดหน้าค้นหาห้องว่าง แล้วเลือกวันเช็กอิน วันเช็กเอาท์ และจำนวนสัตว์เลี้ยง
2. ระบบตรวจว่าครบครบ วันเช็กเอาท์อยู่หลังวันเช็กอิน และจำนวนสัตว์อย่างน้อย 1 ตัว
3. ระบบดึงห้องที่สถานะ **ACTIVE** ทั้งหมด
4. ระบบคัดเฉพาะห้องที่รับสัตว์ได้ไม่น้อยกว่าจำนวนที่ระบุ และไม่มีการจองสถานะ **PENDING, CONFIRMED** หรือ **CHECKED_IN** ทับช่วงวันที่ (หนึ่งห้องต่อการจอง)
5. ระบบแสดงรายการห้องว่างพร้อมราคาต่อตัวต่อคืน และปุ่ม "จองห้องนี้"
6. ผู้ใช้กด "จองห้องนี้" ระบบพาไปฟอร์มจองพร้อมห้องและวันที่ที่เลือก (ไป UC-05)

ทางเลือก

- 6a. ผู้ใช้ยังไม่ได้เข้าสู่ระบบ → ระบบพาไปหน้าเข้าสู่ระบบ
แล้วกลับมาที่ฟอร์มจองพร้อมห้องและวันที่เดิม (UC-02 ทางเลือก 4a)
- 1a. ระบบภายนอกเรียกผ่าน REST API `GET /api/rooms/available` ได้ผลเป็น JSON ด้วยกฎเดียวกัน
- 1b. ค้นหาห้องแบบแบ่งหน้าและเรียงลำดับผ่าน `GET /api/rooms/search`
(keyword, minCapacity, maxPrice, page, size, sort)

กรณีผิดปกติ

- 2a. กรอกไม่ครบ → แจ้ง "กรุณกรอกวันที่และจำนวนสัตว์เลี้ยงให้ครบถ้วน"
- 2b. วันเช็คอินอยู่หลังวันเช็คเอาท์ → แจ้ง "วันที่ Check-out ต้องอยู่หลังวันที่ Check-in"
- 2c. จำนวนสัตว์น้อยกว่า 1 → แจ้ง "จำนวนสัตว์เลี้ยงต้องอย่างน้อย 1 ตัว"
- 5a. ไม่มีห้องว่างในช่วงที่เลือก → แสดง "ไม่มีห้องว่างในช่วงวันที่เลือก"
- REST API: ข้อมูลไม่ถูกต้องตอบ 400 พร้อม `ErrorResponse`

โค้ดที่เกี่ยวข้อง: `AvailabilityController`,

`AvailabilityRestController`,

`AvailabilityServiceImpl.findAvailableRooms()`,

`BookingRepository.existsOverlappingBooking()`,

`AvailabilityMapper`, `RoomSearchServiceImpl`

UC-04 จัดการสัตว์เลี้ยง

หัวข้อ	รายละเอียด
Actor หลัก	ลูกค้า
Actor รอง	—
คำอธิบาย	ลูกค้าเพิ่ม แก้ไข หรือนำสัตว์เลี้ยงของตนออกจากรายการที่ใช้งาน และอาจแนบรูปภาพสัตว์เลี้ยง
เงื่อนไขก่อนเริ่ม (Pre-condition)	ลูกค้าเข้าสู่ระบบ และบัญชียังใช้งานอยู่
เงื่อนไขหลังจบ (Post-condition)	ข้อมูลสัตว์เลี้ยงถูกบันทึกหรือแก้ไขตามคำขอ หากนำออกจากรายการ ระบบเปลี่ยน <code>active</code> เป็น <code>false</code> โดยไม่ลบข้อมูลออกจากรฐานข้อมูล
Include / Extend	—

ขั้นตอนหลัก (Main Flow)

1. ลูกค้าเปิดหน้าสัตว์เลี้ยงของฉัน
2. ระบบแสดงสัตว์เลี้ยงที่ยังใช้งานอยู่และเป็นของลูกค้าคนนั้น
3. ลูกค้าเลือกเพิ่มสัตว์เลี้ยงและกรอกชื่อ ประเภท และข้อมูลอื่นที่ต้องการ
4. ลูกค้ากดบันทึก
5. ระบบตรวจสอบข้อมูลที่กรอก และผูกสัตว์เลี้ยงกับบัญชีของลูกค้า
6. ระบบบันทึกสัตว์เลี้ยงใหม่โดยกำหนด **active = true**
7. ระบบกลับไปหน้ารายการและแสดงข้อความว่าบันทึกสำเร็จ

ทางเลือก (Alternative Flow)

- 3a. ลูกค้าแนบรูปภาพ → หลังบันทึกข้อมูลสัตว์เลี้ยง
ระบบจัดเก็บรูปภาพและเชื่อมรูปกับสัตว์เลี้ยง
- 3b. ลูกค้าเลือกแก้ไขสัตว์เลี้ยงที่มีอยู่ → ระบบตรวจสอบว่าเป็นเจ้าของ
แล้วบันทึกข้อมูลที่แก้ไข
- 3c. ลูกค้าเลือกนำสัตว์เลี้ยงออกจากรายการ → ระบบตรวจสอบว่าเป็นเจ้าของ
แล้วเปลี่ยน **active** เป็น **false** สัตว์เลี้ยงจึงไม่ปรากฏในรายการที่ใช้งาน

กรณีผิดปกติ (Exception Flow)

- 5a. ข้อมูลที่กรอกไม่ผ่านการตรวจสอบ เช่น ไม่ระบุชื่อหรือประเภท อายุเป็นค่าติดลบ
หรือน้ำหนักไม่มากกว่าศูนย์ → ระบบแจ้งให้ตรวจสอบข้อมูลและไม่บันทึกค่าขอนั้น
- 3b-1. ไม่พบสัตว์เลี้ยง หรือสัตว์เลี้ยงไม่ได้เป็นของลูกค้า →
ระบบไม่อนุญาตให้แก้ไขและตอบว่าไม่พบข้อมูล
- 3c-1. ไม่พบสัตว์เลี้ยง หรือสัตว์เลี้ยงไม่ได้เป็นของลูกค้า →
ระบบไม่เปลี่ยนสถานะและตอบว่าไม่พบข้อมูล
- 3a-1. อัปโหลดรูปภาพไม่สำเร็จ → ระบบแจ้งข้อผิดพลาด
โดยข้อมูลสัตว์เลี้ยงที่บันทึกหรือแก้ไขก่อนหน้านี้ยังคงอยู่

โค้ดที่เกี่ยวข้อง: `PetPageController`, `PetServiceImpl.createPet()`,

`PetServiceImpl.updatePet()`, `PetServiceImpl.deactivatePet()`,

`PetServiceImpl.updatePetImage()`, `CreatePetRequest`,

`UpdatePetRequest`

UC-05 จองห้องพัก

หัวข้อ	รายละเอียด
Actor หลัก	ลูกค้า

หัวข้อ	รายละเอียด
Actor รอง	—
คำอธิบาย	ลูกค้าเลือกห้อง ช่วงวันเข้าพัก และสัตว์เลี้ยง เพื่อสร้างการจองที่รอฟังก์ชันยืนยัน
เงื่อนไขก่อนเริ่ม (Pre-condition)	ลูกค้าเข้าสู่ระบบแล้ว และมีสัตว์เลี้ยงที่ยังใช้งานอยู่อย่างน้อย 1 ตัว
เงื่อนไขหลังจบ (Post-condition)	มีการจองใหม่สถานะ PENDING และสถานะชำระเงิน UNPAID บันทึกราคาไว้ ณ เวลาที่จอง
Include / Extend	include: ตรวจสอบห้องว่าง, คำนวณราคา · extend: เลือกบริการเสริม, ใช้โปรโมชั่น

ขั้นตอนหลัก (Main Flow)

1. ลูกค้าเลือกห้องจากหน้ารายการห้องหรือผลค้นหาห้องว่าง
2. ลูกค้าเลือกวันเช็คอินและเช็คเอาท์
3. ระบบแสดงฟอร์มจอง พร้อมรายการสัตว์เลี้ยงของลูกค้า
(ปิดตัวที่มีการจองซ้อนช่วงเดียวกัน)
4. ลูกค้าเลือกสัตว์เลี้ยง
5. ระบบคำนวณและแสดงราคาโดยประมาณ
6. ลูกค้ากดยืนยันการจอง
7. ระบบตรวจสอบห้องว่าง ความจุ และสัตว์ที่จองซ้อน
8. ระบบคำนวณราคาอีกครั้งและบันทึกการจองเป็นสถานะ **PENDING**
9. ระบบแสดงหน้ารายละเอียดการจอง

ทางเลือก (Alternative Flow)

- 4a. ลูกค้าเลือกบริการเสริมให้สัตว์แต่ละตัว → ระบบรวมค่าบริการในราคา
- 4b. ลูกค้าเลือกโปรโมชั่น → ระบบตรวจสอบว่าโปรโมชั่นใช้ได้ ณ วันที่จอง แล้วหักส่วนลด

กรณีผิดปกติ (Exception Flow)

- 7a. ห้องไม่ว่างในช่วงที่เลือก → แจ้ง "ห้องไม่ว่าง" และไม่บันทึกการจอง
- 7b. จำนวนสัตว์เกินความจุห้อง → แจ้งข้อผิดพลาด
- 7c. สัตว์มีการจองซ้อนในช่วงเดียวกัน → แจ้งข้อสัตว์และช่วงวันที่ที่ซ้อน
- 4b-1. โปรโมชั่นหมดอายุหรือยังไม่เริ่ม → แจ้งข้อผิดพลาดและไม่บันทึกการจอง

โค้ดที่เกี่ยวข้อง: `BookingController`,

`BookingServiceImpl.createBooking()`,

AvailabilityService.checkRoomAvailable(),
PricingService.calculate()

UC-06 ยกเลิกการจอง

หัวข้อ	รายละเอียด
Actor หลัก	ลูกค้า, พนักงาน (ผู้ดูแลระบบทำได้เช่นกัน)
Actor รอง	—
คำอธิบาย	ผู้ใช้ยกเลิกการจองที่ยังไม่ชำระเงินและอยู่ในสถานะรอยืนยันหรือยืนยันแล้ว
เงื่อนไขก่อนเริ่ม (Pre-condition)	ผู้ใช้เข้าสู่ระบบและบัญชียังใช้งานอยู่ ลูกค้าต้องเป็นเจ้าของการจอง ส่วนพนักงานหรือผู้ดูแลระบบทำรายการผ่านหน้าจัดการการจองได้
เงื่อนไขหลังจบ (Post-condition)	การจองเปลี่ยนเป็น CANCELLED โดยยังเก็บข้อมูลไว้ในฐานข้อมูล และไม่นับรายการนี้เป็นการสงวนห้องหรือสัตว์เลี้ยงในช่วงวันที่เดิม
Include / Extend	—

ขั้นตอนหลัก (Main Flow)

1. ลูกค้าเปิดหน้าการจองของฉันหรือหน้ารายละเอียดการจอง
2. ระบบแสดงปุ่มยกเลิกสำหรับรายการที่มีสถานะ **PENDING** หรือ **CONFIRMED** และสถานะชำระเงิน **UNPAID**
3. ลูกค้ากดยกเลิกและยืนยันในหน้าต่างยืนยัน
4. ระบบตรวจสอบบัญชีผู้ใช้และตรวจสอบว่าลูกค้าเป็นเจ้าของการจอง
5. ระบบโหลดและล็อกการจอง แล้วตรวจสอบว่ายังไม่ชำระเงิน
6. ระบบเลือก State ตามสถานะปัจจุบัน และเรียก **cancel(booking)** ผ่าน **PendingState** หรือ **ConfirmedState**
7. ระบบเปลี่ยนสถานะเป็น **CANCELLED** และบันทึกการจอง
8. ระบบกลับไปหน้ารายละเอียดการจอง พร้อมข้อความ “ยกเลิกการจองเรียบร้อยแล้ว”

ทางเลือก (Alternative Flow)

- 3a. ลูกค้าปิดหรือปฏิเสธหน้าต่างยืนยัน → ระบบไม่ส่งคำขอยกเลิก และการจองยังมีสถานะเดิม
- 1a. พนักงานหรือผู้ดูแลระบบเปิดหน้าจัดการการจอง **/admin/bookings** → เลือกรายการและยืนยันการยกเลิก ระบบตรวจสอบสิทธิ์ **STAFF / ADMIN**

แล้วใช้กฎสถานะและการชำระเงินเดียวกัน

เมื่อสำเร็จกลับไปหน้าจัดการการจองพร้อมข้อความแจ้งผล

กรณีผิดปกติ (Exception Flow)

- 4a. ยังไม่เข้าสู่ระบบ → ต้องเข้าสู่ระบบก่อนทำรายการ
- 4b. บัญชีถูกระงับ → ระบบปฏิเสธการทำรายการ
- 4c. ลูกค้าพยายามยกเลิกการจองของผู้อื่น → ระบบไม่อนุญาต
โดยเส้นทางเว็บตอบว่าไม่พบข้อมูล และไม่เปลี่ยนสถานะ
- 5a. ไม่พบการจอง → ระบบแจ้งว่าไม่พบข้อมูล
- 5b. การจองมีสถานะชำระเงิน PAID → ระบบปฏิเสธการยกเลิก
และไม่เปลี่ยนข้อมูลการชำระเงิน
- 6a. การจองอยู่ในสถานะ CHECKED_IN, CHECKED_OUT หรือ CANCELLED →
State ปฏิเสธการยกเลิกด้วย `IllegalStateException`
และไม่บันทึกสถานะใหม่

กฎการทำงานเพิ่มเติม

- ยกเลิกได้เฉพาะการจองสถานะ PENDING หรือ CONFIRMED ที่ยังไม่ชำระเงิน
- การยกเลิกเปลี่ยนสถานะการจอง โดยไม่ลบ Booking, BookingPet
หรือรายการบริการเสริม
- การตรวจห้องและสัตว์ที่ติดจองนับเฉพาะ PENDING, CONFIRMED และ
CHECKED_IN จึงไม่นับการจองที่เป็น CANCELLED
- ระบบตรวจเงื่อนไขซ้ำที่ Service แม้หน้าเว็บจะแสดงปุ่มเฉพาะรายการที่ยกเลิกได้แล้ว
- REST API ใช้ POST `/api/bookings/{id}/cancel` และตอบ 409
Conflict เมื่อสถานะหรือการชำระเงินไม่อนุญาตให้ยกเลิก

โค้ดที่เกี่ยวข้อง: `BookingController.cancelBooking()`,

`AdminBookingController.performAction()`,

`BookingRestController.cancelBooking()`,

`BookingServiceImpl.cancelBooking()`, `PendingState.cancel()`,

`ConfirmedState.cancel()`, `BookingRepository.findByIdForUpdate()`,

`AvailabilityServiceImpl`

UC-07 ยืนยันการจอง

หัวข้อ	รายละเอียด
--------	------------

หัวข้อ	รายละเอียด
Actor หลัก	พนักงาน (ผู้ดูแลระบบทำได้เช่นกัน)
Actor รอง	ระบบอีเมล
คำอธิบาย	พนักงานตรวจสอบและยืนยันการจองที่รออยู่ ระบบส่งอีเมลแจ้งเตือนลูกค้าอัตโนมัติ
เงื่อนไขก่อนเริ่ม	พนักงานเข้าสู่ระบบ และการจองอยู่ในสถานะ PENDING
เงื่อนไขหลังจบ	การจองเปลี่ยนเป็น CONFIRMED และลูกค้าได้รับอีเมลยืนยัน
Include / Extend	include: ส่งอีเมลยืนยันการจอง

ขั้นตอนหลัก

1. พนักงานเปิดหน้าจัดการการจอง
2. ระบบแสดงรายการการจองทั้งหมด
3. พนักงานกด "ยืนยัน" ที่การจองที่มีสถานะ **PENDING**
4. ระบบเปลี่ยนสถานะเป็น **CONFIRMED** ผ่าน State Pattern
(`PendingState.confirm()`) แล้วบันทึก
5. ระบบประกาศเหตุการณ์ **BookingConfirmedEvent**
6. หลังบันทึกลงฐานข้อมูลสำเร็จ (after commit) **BookingConfirmedListener**
รับเหตุการณ์
7. **EmailNotificationService** ส่งอีเมลยืนยันถึงลูกค้า
8. ระบบแสดงข้อความยืนยันสำเร็จ

กรณีผิดปกติ

- 4a. การจองไม่ได้อยู่ในสถานะ **PENDING** (เช่น ถูกยกเลิกแล้ว) →
แจ้งว่าเปลี่ยนสถานะไม่ได้ ไม่ส่งอีเมล
- 7a. ส่งอีเมลไม่สำเร็จ → บันทึก log ข้อผิดพลาด แต่การจองยังคงเป็น **CONFIRMED**
(ไม่ rollback)

โค้ดที่เกี่ยวข้อง: `AdminBookingController`,

`BookingServiceImpl.confirmBooking()`,

`notification/BookingConfirmedEvent`, `BookingConfirmedListener`,

`EmailNotificationService`

UC-08 เช็กอิน / เช็กเอาท์

หัวข้อ	รายละเอียด
Actor หลัก	พนักงาน (ผู้ดูแลระบบทำได้เช่นกัน)

หัวข้อ	รายละเอียด
Actor รอง	—
คำอธิบาย	พนักงานบันทึกการเริ่มและสิ้นสุดการเข้าพักของสัตว์เลี้ยง โดยเปลี่ยนสถานะการจองตามลำดับและตรวจวันที่ก่อนทำรายการ
เงื่อนไขก่อนเริ่ม (Pre-condition)	ผู้ใช้เข้าสู่ระบบด้วยบทบาท STAFF หรือ ADMIN บัญชียังใช้งานอยู่ และมีการจองที่ต้องการทำรายการ
เงื่อนไขหลังจบ (Post-condition)	เมื่อเช็กอินสำเร็จ การจองเปลี่ยนจาก CONFIRMED เป็น CHECKED_IN เมื่อเช็กเอาท์สำเร็จ เปลี่ยนจาก CHECKED_IN เป็น CHECKED_OUT
Include / Extend	—

ขั้นตอนหลัก (Main Flow)

A. เช็กอิน

1. พนักงานเปิดหน้าจัดการการจอง /admin/bookings
2. พนักงานเลือกรายการที่ยืนยันแล้วและมีสถานะ CONFIRMED
3. พนักงานกดเช็กอินและยืนยันการทำรายการ
4. ระบบตรวจสอบสิทธิ์ผู้ใช้ แล้วโหลดและล็อกการจองจากฐานข้อมูล
5. ระบบตรวจวันที่ปัจจุบัน โดยต้องเป็นไปตามเงื่อนไข $checkInDate \leq currentDate < checkOutDate$
6. ระบบเลือก ConfirmedState และเรียก `checkIn(booking)`
เพื่อเปลี่ยนสถานะเป็น CHECKED_IN
7. ระบบบันทึกการจอง แล้วกลับไปหน้าจัดการการจองพร้อมข้อความแจ้งผลสำเร็จ

B. เช็กเอาท์

1. พนักงานเปิดหน้าจัดการการจอง /admin/bookings
2. พนักงานเลือกรายการที่สัตว์เลี้ยงกำลังเข้าพักและมีสถานะ CHECKED_IN
3. พนักงานกดเช็กเอาท์และยืนยันการทำรายการ
4. ระบบตรวจสอบสิทธิ์ผู้ใช้ แล้วโหลดและล็อกการจองจากฐานข้อมูล
5. ระบบตรวจวันที่ปัจจุบัน โดยต้องเป็นไปตามเงื่อนไข $currentDate \geq checkOutDate$
6. ระบบเลือก CheckedinState และเรียก `checkOut(booking)`
เพื่อเปลี่ยนสถานะเป็น CHECKED_OUT
7. ระบบบันทึกการจอง แล้วกลับไปหน้าจัดการการจองพร้อมข้อความแจ้งผลสำเร็จ

ทางเลือก (Alternative Flow)

- A3/B3-1. พนักงานปิดหรือปฏิเสธหน้าต่างยืนยัน → ระบบไม่ส่งคำขอ และการจองยังมีสถานะเดิม
- A5-1. พนักงานเช็คอินหลังวันเข้าที่กำหนด แต่ยังไม่ถึงวันออก → ระบบอนุญาตเมื่อการจองยังเป็น **CONFIRMED**
- B5-1. พนักงานเช็คอินหลังวันออกที่กำหนด → ระบบอนุญาตเมื่อการจองยังเป็น **CHECKED_IN**

กรณีผิดปกติ (Exception Flow)

- A4/B4-1. ยังไม่เข้าสู่ระบบ → ต้องเข้าสู่ระบบก่อนทำรายการ
- A4/B4-2. บัญชีถูกระงับหรือไม่มีบทบาท **STAFF / ADMIN** → ระบบปฏิเสธการทำรายการ
- A4/B4-3. ไม่พบการจอง → ระบบตอบว่าไม่พบข้อมูล
- A5-2. วันที่ปัจจุบันอยู่ก่อนวันเช็คอิน หรือถึง/เลยวันเช็คอินแล้ว → ระบบปฏิเสธการเช็คอินและไม่เปลี่ยนสถานะ
- A6-1. สถานะปัจจุบันไม่ใช่ **CONFIRMED** เช่น ยังรอยืนยัน ถูกยกเลิก หรือเช็คอินแล้ว → State ปฏิเสธการเช็คอิน
- B5-2. วันที่ปัจจุบันยังไม่ถึงวันเช็คอิน → ระบบปฏิเสธการเช็คอินและไม่เปลี่ยนสถานะ
- B6-1. สถานะปัจจุบันไม่ใช่ **CHECKED_IN** เช่น ยังไม่ได้เช็คอินหรือเช็คอินแล้ว → State ปฏิเสธการเช็คอิน

เมื่อสถานะหรือวันที่ไม่อนุญาต ระบบโยน **IllegalStateException**

โดยไม่บันทึกสถานะใหม่ ฝั่งเว็บกลับไปหน้าจัดการการจองพร้อมข้อความผิดพลาด ส่วน REST API

ตอบ 409 Conflict

กฎการทำงานเพิ่มเติม

- วันปัจจุบันใช้เขตเวลา Asia/Bangkok
- การตรวจวันที่ใช้ **Booking.canCheckInOn(currentDate)** และ **Booking.canCheckOutOn(currentDate)**
- ระบบตรวจสอบสถานะและวันที่เข้าพักที่ Service ก่อนเปลี่ยนสถานะจริง
- การจองสถานะ **CHECKED_IN** ยืนยันว่าสงวนห้องและสัตว์เลี้ยง ส่วน **CHECKED_OUT** ไม่นับเป็นรายการที่สงวนห้องหรือสัตว์แล้ว
- การเช็คอินและเช็คอินไม่เปลี่ยน **PaymentStatus** และไม่คำนวณราคาการจองใหม่โดยอัตโนมัติ

- โค้ดปัจจุบันไม่ได้บังคับให้ชำระเงินก่อนเช็คอิน การยืนยันรับชำระเงินเป็นรายการแยกใน UC-09

- REST API ใช้ `POST /api/bookings/{id}/check-in` และ `POST /api/bookings/{id}/check-out`

โค้ดที่เกี่ยวข้อง: `AdminBookingController.performAction()`,

`BookingRestController.checkIn()`,

`BookingRestController.checkOut()`, `BookingServiceImpl.checkIn()`,

`BookingServiceImpl.checkOut()`, `ConfirmedState.checkIn()`,

`CheckedInState.checkOut()`, `Booking.canCheckInOn()`,

`Booking.canCheckOutOn()`, `BookingRepository.findByIdForUpdate()`

UC-09 ยืนยันรับชำระเงิน (Confirm Payment)

หัวข้อ	รายละเอียด
Actor หลัก	พนักงาน (ผู้ดูแลระบบทำได้เช่นกัน)
คำอธิบาย	พนักงานบันทึกว่าลูกค้าชำระเงินเต็มจำนวนแล้ว และระบบออกใบเสร็จ
เงื่อนไขก่อนเริ่ม	พนักงานเข้าสู่ระบบ การจองอยู่ในสถานะ CONFIRMED , CHECKED_IN หรือ CHECKED_OUT
เงื่อนไขหลังจบ	การจองมีสถานะชำระเงิน PAID บันทึกยอดและเวลารับเงิน และมีใบเสร็จ 1 ใบ
Include	ออกใบเสร็จ (Issue Receipt)

ขั้นตอนหลัก

1. พนักงานเปิดหน้าจัดการการจอง แล้วกด "ยืนยันรับชำระเงิน"
2. ระบบล็อกแถวการจอง (`findByIdForUpdate`) เพื่อป้องกันการยืนยันซ้ำพร้อมกัน
3. ระบบตรวจสอบสถานะการจองและยอดเงิน
4. ระบบบันทึก `payment_status = PAID`, ยอดที่ชำระเท่ากับยอดรวมของการจอง และเวลาที่รับเงิน
5. ระบบออกใบเสร็จเลขที่ `REC-<UUID>` โดยใช้ราคาที่บ้านที่ไว้ตอนจอง ไม่คำนวณราคาใหม่
6. ระบบแสดงข้อความสำเร็จ

ทางเลือก

- 2a. การจองชำระเงินแล้ว → คืนใบเสร็จเดิม ไม่บันทึกเวลารับเงินใหม่

กรณีผิดปกติ

- 3a. การจองเป็น **PENDING** หรือ **CANCELLED** → แจ้งว่ายังรับชำระเงินไม่ได้
- 3b. ยอดรวมของการจองไม่ถูกต้อง → แจ้งข้อผิดพลาด
- 5a. ออกใบเสร็จไม่สำเร็จ → ย้อนการบันทึกการชำระเงินทั้งหมด (อยู่ใน transaction เดียวกัน)

โค้ดที่เกี่ยวข้อง: AdminBookingController, PaymentRestController,
PaymentServiceImpl.confirmPayment(),
ReceiptServiceImpl.createReceipt()

UC-10 บันทึกรายงานการดูแล

หัวข้อ	รายละเอียด
Actor หลัก	พนักงาน (ผู้ดูแลระบบทำได้เช่นกัน)
Actor รอง	—
คำอธิบาย	พนักงานบันทึกข้อมูลการดูแลสัตว์เลี้ยงในการจองเป็นรายวัน และแก้ไขรายงานที่บันทึกแล้วได้
เงื่อนไขก่อนเริ่ม (Pre-condition)	พนักงานเข้าสู่ระบบและบัญชียังใช้งานอยู่ มีสัตว์เลี้ยงในการจองที่ต้องการบันทึกรายงาน
เงื่อนไขหลังจบ (Post-condition)	มีรายงานการดูแลของสัตว์ในการจองสำหรับวันที่เลือก พร้อมข้อมูลผู้บันทึก โดยสัตว์หนึ่งรายการในการจองมีรายงานได้ไม่เกินหนึ่งฉบับต่อวัน
Include / Extend	—

ขั้นตอนหลัก (Main Flow)

1. พนักงานเปิดหน้ารายงานการดูแลและเลือกการจองกับสัตว์เลี้ยง
2. ระบบแสดงข้อมูลสัตว์เลี้ยงและรายงานที่เคยบันทึกไว้
3. พนักงานเลือกเขียนรายงานและระบุวันที่รายงาน
4. พนักงานกรอกข้อมูลการให้อาหาร การเดินเล่น การดูแลความสะอาด อารมณ์ สุขภาพ และหมายเหตุที่เกี่ยวข้อง
5. พนักงานกดบันทึก
6. ระบบตรวจสอบว่าผู้บันทึกเป็นพนักงานหรือผู้ดูแลระบบ และการจองอยู่ในสถานะ **CHECKED_IN** หรือ **CHECKED_OUT**
7. ระบบตรวจสอบว่าวันที่รายงานอยู่ในช่วงวันเช็กอินถึงวันเช็กเอาท์ และยังไม่มียางานของสัตว์ในการจองรายการนี้ในวันเดียวกัน

8. ระบบบันทึกรายงานพร้อมผู้บันทึก แล้วแสดงข้อความว่าบันทึกสำเร็จ

ทางเลือก (Alternative Flow)

- 3a. พนักงานเลือกรายงานที่บันทึกไว้แล้วและกดแก้ไข → ระบบแสดงข้อมูลเดิม
พนักงานแก้ไขข้อมูลแล้วบันทึก โดยระบบตรวจสอบวันที่ สัตว์ในการจอง
และรายงานที่อาจซ้ำอีกครั้ง
- 1a. พนักงานค้นหาการจองจากหมายเลขการจอง ชื่อหรืออีเมลเจ้าของ หรือชื่อสัตว์เลี้ยง
และกรองการจองที่มีรายงานตามวันที่ ก่อนเลือกรายการที่ต้องการ

กรณีผิดปกติ (Exception Flow)

- 6a. ผู้ใช้ไม่ใช่พนักงานหรือผู้ดูแลระบบ → ระบบปฏิเสธการบันทึกหรือแก้ไขรายงาน
- 6b. ไม่พบสัตว์ในการจอง หรือการจองไม่ได้อยู่ในสถานะ CHECKED_IN หรือ
CHECKED_OUT → ระบบไม่บันทึกรายงาน
- 7a. วันที่รายงานอยู่นอกช่วงเข้าพัก → ระบบแจ้งให้ตรวจสอบวันที่และไม่บันทึก
- 7b. มีรายงานของสัตว์ในการจองรายการนี้ในวันที่เลือกแล้ว →
ระบบแจ้งว่ามีรายงานอยู่แล้วและไม่สร้างรายงานซ้ำ
- 4a. ข้อมูลไม่ผ่านการตรวจสอบ เช่น ระยะเวลาเดินเล่นติดลบ →
ระบบแจ้งให้ตรวจสอบข้อมูลและไม่บันทึก

โค้ดที่เกี่ยวข้อง: AdminCareReportPageController,

DailyCareReportServiceImpl.createReport(),

DailyCareReportServiceImpl.updateReport(),

CreateDailyCareReportRequest, UpdateDailyCareReportRequest,

DailyCareReport

UC-11 ดูรายงานการดูแล

หัวข้อ	รายละเอียด
Actor หลัก	ลูกค้า
Actor รอง	—
คำอธิบาย	ลูกค้าดูรายงานการดูแลสัตว์เลี้ยงของตน โดยเลือกการจอง สัตว์เลี้ยง และวันที่ของรายงานที่ต้องการอ่าน
เงื่อนไขก่อนเริ่ม (Pre-condition)	ลูกค้าเข้าสู่ระบบ และบัญชียังใช้งานอยู่
เงื่อนไขหลังจบ (Post-condition)	ลูกค้าเห็นรายงานที่มีสิทธิ์อ่าน โดยไม่มีการเปลี่ยนข้อมูลรายงาน
Include / Extend	—

ขั้นตอนหลัก (Main Flow)

1. ลูกค้าเปิดหน้ารายงานการดูแล
2. ระบบแสดงเฉพาะการจองของลูกค้าที่มีรายงาน โดยแสดงหน้าละ 5 การจอง
3. ลูกค้าเลือกการจองและสัตว์เลี้ยงที่ต้องการดู
4. ระบบตรวจสอบว่าสัตว์เลี้ยงในการจองนั้นเป็นของลูกค้า
5. ระบบเปิดหน้ารายละเอียดและแสดงรายงานวันที่ล่าสุดก่อนหนึ่งฉบับ
6. ลูกค้าเลือกวันที่อื่นจากปฏิทินเพื่ออ่านรายงานของวันนั้น

ทางเลือก (Alternative Flow)

- 2a. ลูกค้ากรอกรายการการจองด้วยวันที่รายงาน →
ระบบแสดงเฉพาะการจองที่มีรายงานในวันที่เลือก
- 2b. รายการการจองมีมากกว่า 5 รายการ → ลูกค้าเปลี่ยนหน้าเพื่อดูรายการถัดไป

กรณีผิดปกติ (Exception Flow)

- 2c. ยังไม่มีรายงาน หรือไม่มีการจองที่ตรงกับวันที่กรอกร →
ระบบแสดงสถานะว่าไม่พบรายการ
- 4a. ไม่พบสัตว์ในการจอง หรือสัตว์ในการจองไม่ได้เป็นของลูกค้า →
ระบบไม่เปิดรายงานและตอบว่าไม่พบข้อมูล
- 6a. ลูกค้าเลือกวันที่รายงานที่ไม่อยู่ในรายการของสัตว์ในการจองนั้น →
ระบบตอบว่าไม่พบรายงาน

โค้ดที่เกี่ยวข้อง: CustomerCareReportPageController,

DailyCareReportServiceImpl.getReportBookingsForOwner(),

DailyCareReportServiceImpl.getReportBookingsForOwnerOnDate(),

DailyCareReportServiceImpl.getReportsByBookingPet()

UC-12 จัดการห้องพัก (Manage Rooms)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้ดูแลระบบ
คำอธิบาย	ผู้ดูแลระบบเพิ่ม แก้ไข เปลี่ยนสถานะ ปิดใช้งานห้อง และจัดการรูปห้อง
เงื่อนไขก่อนเริ่ม	เข้าสู่ระบบด้วยบทบาท ADMIN
เงื่อนไขหลังจบ	ข้อมูลห้องถูกบันทึก ห้อง ACTIVE แสดงให้ลูกค้าเห็นและจองได้

ขั้นตอนหลัก (เพิ่มห้อง)

1. ผู้ดูแลระบบเปิดหน้าจัดการห้องพัก แล้วกด "เพิ่มห้อง"

2. กรอกรหมายเลขห้อง ชื่อ รายละเอียด ความจุ ราคาต่อตัวต่อคืน และเลือกรูป (ไม่บังคับ)
3. ระบบตรวจสอบข้อมูลและหมายเลขห้องซ้ำ
4. ระบบบันทึกห้องสถานะ **ACTIVE**
5. ถ้ามีรูป ระบบตรวจไฟล์ บันทึกเป็นชื่อ UUID แล้วผูกกับห้อง
6. ระบบกลับไปหน้ารายการห้องพร้อมข้อความสำเร็จ

ทางเลือก

- A1. แก้ไขห้อง → แก้ข้อมูลแล้วบันทึก (ถ้าเปลี่ยนรูป ระบบลบไฟล์รูปเก่า)
- A2. เปลี่ยนสถานะเป็น **ACTIVE** / **MAINTENANCE** / **INACTIVE** → ห้องที่ไม่ใช่ **ACTIVE** จะหายจากหน้าลูกค้าและจองไม่ได้
- A3. ปิดใช้งานห้อง → เปลี่ยนเป็น **INACTIVE** โดยไม่ลบข้อมูล
- A4. ลบรูปห้อง → ล้าง `image_url` และลบไฟล์

กรณีผิดพลาด

- 3a. ความจุน้อยกว่า 1 หรือราคาติดลบ → แสดงข้อผิดพลาด
- 3b. หมายเลขห้องซ้ำ → แจ้ง "Room number already exists"
- 5a. ไฟล์ไม่ใช่ JPG/PNG, ใหญ่เกิน 5 MB หรือไม่ใช่รูปจริง → บันทึกข้อมูลห้องแล้ว แต่แจ้งว่าอัปโหลดรูปไม่สำเร็จ

โค้ดที่เกี่ยวข้อง: `RoomController`, `RoomRestController`,

`RoomServiceImpl`, `LocalFileStorageService`

UC-13 จัดการโปรโมชั่น (Manage Promotions)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้ดูแลระบบ
คำอธิบาย	ผู้ดูแลระบบดูรายการ เพิ่ม แก้ไข และเปิดหรือปิดใช้งานโปรโมชั่น
เงื่อนไขก่อนเริ่ม	เข้าสู่ระบบด้วยบทบาท ADMIN
เงื่อนไขหลังจบ	ข้อมูลโปรโมชั่นถูกบันทึกตามที่ระบุ โปรโมชั่นจะใช้งานได้เมื่อเปิดใช้งานและอยู่ในช่วงวันที่กำหนด

ขั้นตอนหลัก (เพิ่มโปรโมชั่น)

1. ผู้ดูแลระบบเปิดหน้าจัดการโปรโมชั่น `/admin/promotions`
2. ระบบแสดงรายการโปรโมชั่นทั้งหมด พร้อมส่วนลด วันหมดอายุ และสถานะเปิดใช้งาน
3. ผู้ดูแลระบบกด “เพิ่มโปรโมชั่น”

4. กรอกชื่อ เลือกประเภทส่วนลดเป็นเปอร์เซ็นต์หรือจำนวนเงิน ระบุมูลค่าส่วนลด
วันเริ่มต้น วันสิ้นสุด และสถานะเปิดใช้งาน แล้วกด “บันทึก”
5. ระบบตรวจสอบข้อมูล โดยชื่อห้ามว่าง มูลค่าส่วนลดต้องมากกว่า 0
หากเป็นเปอร์เซ็นต์ต้องไม่เกิน 100 และวันสิ้นสุดต้องไม่ก่อนวันเริ่มต้น
6. ระบบตัดช่องว่างหัวท้ายของชื่อและบันทึกโปรโมชั่น
7. ระบบกลับไปหน้ารายการพร้อมข้อความ “เพิ่มโปรโมชั่นสำเร็จ”

ทางเลือก

- A1. ค้นหารายการ → ผู้ดูแลระบบค้นหาตามชื่อและกรองสถานะเปิดใช้หรือปิดใช้
ระบบแสดงรายการที่ตรงกับตัวกรอง
- A2. แก้ไขโปรโมชั่น → กด “แก้ไข” ระบบแสดงข้อมูลเดิมในฟอร์ม
ผู้ดูแลระบบแก้ไขข้อมูลแล้วบันทึก ระบบตรวจสอบข้อมูลและแสดงข้อความ
“แก้ไขโปรโมชั่นสำเร็จ”
- A3. ปิดใช้งานโปรโมชั่น → เปิดฟอร์มแก้ไข เอาเครื่องหมาย “เปิดใช้โปรโมชั่น” ออก
แล้วบันทึก ระบบกำหนด **active = false** โดยไม่ลบข้อมูล
- A4. เปิดใช้งานอีกครั้ง → เปิดฟอร์มแก้ไข เลือก “เปิดใช้โปรโมชั่น” แล้วบันทึก
โปรโมชั่นจะใช้ได้เมื่ออยู่ในช่วงวันที่กำหนดด้วย
- A5. ยกเลิกการกรอก → กด “ยกเลิก” เพื่อปิดฟอร์มโดยไม่บันทึก

กรณีผิดพลาด

- 5a. ชื่อว่าง ไม่เลือกประเภทส่วนลด หรือไม่กรอกข้อมูลที่จำเป็น →
แสดงข้อผิดพลาดในฟอร์มและไม่บันทึก
- 5b. มูลค่าส่วนลดเป็น 0 หรือติดลบ → แสดงข้อผิดพลาดและไม่บันทึก
- 5c. ส่วนลดแบบเปอร์เซ็นต์มากกว่า 100 → แสดงข้อผิดพลาดและไม่บันทึก
- 5d. มูลค่าส่วนลดมีจำนวนเต็มเกิน 8 หลักหรือทศนิยมเกิน 2 ตำแหน่ง →
แสดงข้อผิดพลาดและไม่บันทึก
- 5e. ไม่ระบุวันเริ่มต้นหรือวันสิ้นสุด หรือวันสิ้นสุดก่อนวันเริ่มต้น →
แสดงข้อผิดพลาดและไม่บันทึก โดยอนุญาตให้วันเริ่มต้นและวันสิ้นสุดเป็นวันเดียวกันได้
- A2-1. ไม่พบโปรโมชั่นที่ต้องการแก้ไข →
ระบบกลับไปหน้ารายการพร้อมข้อความแจ้งข้อผิดพลาด

โค้ดที่เกี่ยวข้อง: AdminPromotionPageController,
PromotionServiceImpl, CreatePromotionRequest, PromotionMapper,
PromotionRepository, Promotion.isAvailableOn(), SecurityConfig
UC-14 จัดการบริการเสริม (Manage Extra Services)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้ดูแลระบบ
คำอธิบาย	ผู้ดูแลระบบดูรายการ เพิ่ม แก้ไข และเปิดหรือปิดใช้งานบริการเสริม
เงื่อนไขก่อนเริ่ม	เข้าสู่ระบบด้วยบทบาท ADMIN
เงื่อนไขหลังจบ	ข้อมูลบริการเสริมถูกบันทึกตามที่ระบุ บริการที่เปิดใช้งานสามารถเลือกในการจองใหม่ได้

ขั้นตอนหลัก (เพิ่มบริการเสริม)

1. ผู้ดูแลระบบเปิดหน้าจัดการบริการเสริม /admin/services
2. ระบบแสดงรายการบริการเสริมทั้งหมด พร้อมราคาและสถานะเปิดใช้งาน
3. ผู้ดูแลระบบกด “เพิ่มบริการ”
4. กรอกชื่อ ราคา รายละเอียด (ไม่บังคับ) และสถานะเปิดใช้งาน แล้วกด “บันทึก”
5. ระบบตรวจสอบข้อมูล โดยชื่อห้ามว่าง ราคาต้องไม่ติดลบ มีจำนวนเต็มไม่เกิน 8 หลัก และทศนิยมไม่เกิน 2 ตำแหน่ง โดยอนุญาตราคา 0 บาท
6. ระบบตัดช่องว่างหัวท้ายของชื่อและบันทึกบริการเสริม
7. ระบบกลับไปหน้ารายการพร้อมข้อความ “เพิ่มบริการเสริมสำเร็จ”

ทางเลือก

- A1. ค้นหารายการ → ผู้ดูแลระบบค้นหาตามชื่อและกรองสถานะเปิดใช้หรือปิดใช้ ระบบแสดงรายการที่ตรงกับตัวกรอง
- A2. แก้ไขบริการเสริม → กด “แก้ไข” ระบบแสดงข้อมูลเดิมในฟอร์ม ผู้ดูแลระบบแก้ไขข้อมูลแล้วบันทึก ระบบตรวจสอบและแสดงข้อความ “แก้ไขบริการเสริมสำเร็จ”
- A3. ปิดใช้งานบริการ → เปิดฟอร์มแก้ไข เอาเครื่องหมาย “เปิดให้บริการ” ออก แล้วบันทึก ระบบกำหนด `active = false` โดยไม่ลบข้อมูล และไม่อนุญาตให้เลือกบริการนั้นในการจองใหม่
- A4. เปิดใช้งานอีกครั้ง → เปิดฟอร์มแก้ไข เลือก “เปิดให้บริการ” แล้วบันทึก

- A5. ยกเลิกการกรอก → กด “ยกเลิก” เพื่อปิดฟอร์มโดยไม่บันทึก

กรณีผิดปกติ

- 5a. ชื่อว่างหรือไม่ระบุราคา → แสดงข้อผิดพลาดในฟอร์มและไม่บันทึก
- 5b. ราคาติดลบ → แสดงข้อผิดพลาดและไม่บันทึก
- 5c. ราคามีจำนวนเต็มเกิน 8 หลักหรือทศนิยมเกิน 2 ตำแหน่ง → แสดงข้อผิดพลาดและไม่บันทึก
- A2-1. ไม่พบบริการที่ต้องการแก้ไข → ระบบกลับไปหน้ารายการพร้อมข้อความแจ้งข้อผิดพลาด

กฎการทำงานเพิ่มเติม

- การแก้ราคาบริการไม่เปลี่ยน `unit_price` และ `total_price` ของรายการบริการที่บันทึกในการจองเดิมโดยอัตโนมัติ
- การปิดใช้งานบริการยังคงเก็บข้อมูลบริการและรายการที่เคยจองไว้

โค้ดที่เกี่ยวข้อง: `AdminExtraServicePageController`,

`ExtraServiceServiceImpl`, `ExtraServiceRequest`, `ExtraServiceMapper`, `ExtraServiceRepository`, `BookingServiceImpl`, `SecurityConfig`

UC-15 จัดการสมาชิก (Manage Users)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้ดูแลระบบ
คำอธิบาย	ผู้ดูแลระบบค้นหา เรียงลำดับ และแก้ไขข้อมูล บทบาท และสถานะของสมาชิก
เงื่อนไขก่อนเริ่ม	เข้าสู่ระบบด้วยบทบาท ADMIN
เงื่อนไขหลังจบ	ข้อมูลสมาชิกถูกบันทึก สมาชิกที่ถูกระงับเข้าสู่ระบบไม่ได้

ขั้นตอนหลัก

1. ผู้ดูแลระบบเปิดหน้าสมาชิก ระบบแสดงรายชื่อทั้งหมด
2. (ไม่บังคับ) ค้นหาด้วยชื่อ อีเมล หรือเบอร์โทร และเลือกการเรียงลำดับ
3. กด "แก้ไข" ที่สมาชิกที่ต้องการ
4. แก้ชื่อ นามสกุล อีเมล เบอร์โทร บทบาท หรือสถานะบัญชี
5. ระบบตรวจสอบข้อมูล แปลงอีเมลเป็นตัวพิมพ์เล็ก และตรวจอีเมลซ้ำ
6. ระบบบันทึกและกลับไปหน้ารายชื่อพร้อมข้อความสำเร็จ

กรณีผิดปกติ

- 5a. รูปแบบอีเมลหรือเบอร์โทรไม่ถูกต้อง → กลับไปหน้าแก้ไขพร้อมข้อผิดพลาด
- 5b. อีเมลถูกใช้โดยบัญชีอื่น → แจ้ง "อีเมลนี้ถูกใช้แล้ว"
- 5c. ผู้ดูแลระบบพยายามเปลี่ยนบทบาท ระบุ หรือเปลี่ยนอีเมลของตัวเอง → ปฏิเสธ
เพื่อป้องกันการล็อกตัวเองออกจากระบบ

โค้ดที่เกี่ยวข้อง: AdminUserController, UserServiceImpl, UpdateUserRequest

UC-16 ดู Dashboard รายได้ (View Revenue Dashboard)

หัวข้อ	รายละเอียด
Actor หลัก	ผู้ดูแลระบบ
คำอธิบาย	ผู้ดูแลระบบดูภาพรวมการจอง ยอดรับเงินรายเดือนและรายปี รวมถึงรายละเอียดรายการรับเงินและใบเสร็จ
เงื่อนไขก่อนเริ่ม	เข้าสู่ระบบด้วยบทบาท ADMIN
เงื่อนไขหลังจบ	ระบบแสดงข้อมูลภาพรวมและยอดรับเงินตามปีที่เลือก โดยไม่เปลี่ยนข้อมูลการจองหรือการชำระเงิน

ขั้นตอนหลัก

1. ผู้ดูแลระบบเปิดหน้า Dashboard /admin
2. ระบบใช้ปีปัจจุบันเป็นปีเริ่มต้น และแสดงยอดรับเงินเดือนนี้ จำนวนการจองเดือนนี้
ห้องว่างคืนนี้ และสัตว์ที่กำลังเข้าพัก
3. ระบบแสดงการจองล่าสุดไม่เกิน 5 รายการ พร้อมเลขที่การจอง ลูกค้า สัตว์เลี้ยง ห้อง
และสถานะ
4. ผู้ดูแลระบบระบุปี ค.ศ. แล้วกด “ดูยอดเงิน”
5. ระบบตรวจว่าปีอยู่ระหว่าง 1-9999
และรวมยอดจากใบเสร็จของการจองที่ยืนยันรับเงินแล้ว โดยจัดเดือนตามวันที่รับเงิน
6. ระบบแสดงยอดรับเงินครบ 12 เดือนและยอดรวมทั้งปีของปีที่เลือก
7. ผู้ดูแลระบบกดชื่อเดือน ระบบเปิดหน้าต่างรายการรับเงินของเดือนนั้น
พร้อมจำนวนรายการ ยอดรวม เลขที่การจอง ลูกค้า วันที่รับเงิน และยอดรับเงิน
8. ผู้ดูแลระบบกด “ดูรายละเอียด” ของรายการ ระบบแสดงสัตว์เลี้ยง ห้อง วันเข้าพัก
วันออก ค่าห้อง ค่าบริการเสริม ค่าใช้จ่ายเพิ่มเติม ส่วนลด และเลขที่ใบเสร็จ

ทางเลือก

- A1. ไม่เปลี่ยนปี → ดูยอดรายเดือนและรายปีของปีปัจจุบันได้ทันที

- A2. เปลี่ยนปี → ระบุปีใหม่แล้วกด “ดูยอดเงิน”
ระบบปรับส่วนรายงานรายเดือนและรายปีตามปีที่เลือก
- A3. ดูใบเสร็จ → กด “ดูใบเสร็จ” ในรายละเอียดรายการ ระบบเปิดใบเสร็จในแท็บใหม่
- A4. ย่อรายละเอียด → กด “ย่อรายละเอียด” เพื่อซ่อนรายละเอียดของรายการนั้น
- A5. ปิดรายการของเดือน → กด “ปิด” เพื่อกลับมาดูตารางยอดรายเดือน
- A6. เดือนที่ไม่มีรายการรับเงิน → แสดงยอด 0.00 บาท และเมื่อเปิดเดือนนั้น
แสดงข้อความ “เดือนนี้ยังไม่มีรายการที่ยืนยันรับเงินแล้ว”
- A7. ยังไม่มีการจอง → ตารางการจองล่าสุดแสดงข้อความ “ยังไม่มีรายการจอง”

กรณีผิดพลาด

- 5a. ส่งป้อนช่วง 1-9999 → ระบบปฏิเสธคำขอด้วย HTTP 400
โดยช่องกรอกปีบนหน้าจอกำหนดช่วงเดียวกันไว้ด้วย
- A2-1. โหลดรายงานของปีที่เลือกไม่สำเร็จ → แสดงข้อความ “โหลดรายงานไม่สำเร็จ
กรุณาลองอีกครั้ง หากหมดเวลาเข้าสู่ระบบให้เข้าสู่ระบบใหม่”

กฎการทำงานเพิ่มเติม

- ยอดรับเงินรวมจาก `Receipt.totalAmount` เฉพาะรายการที่มีการจองเกี่ยวข้อง มี
`paymentStatus = PAID` และมี `paidAt`
- การจัดยอดเข้าเดือนและปีใช้ `Booking.paidAt` ไม่ใช่วันสร้างการจอง วันเข้าพัก
หรือวันออกใบเสร็จ
- เดือนที่ไม่มีรายได้ยังแสดงในตาราง โดยมียอด 0.00 บาท ยอดรวมทั้งปีเป็นผลรวมของทั้ง
12 เดือน
- รายการรับเงินภายในแต่ละเดือนเรียงตาม `paidAt` จากเก่าไปใหม่
- การเลือกปีเปลี่ยนเฉพาะส่วนรายงานรายเดือนและรายปี ส่วนการ์ด “ยอดรับเงินเดือนนี้”
และ “การจองเดือนนี้” ยังคงอ้างอิงเดือนปัจจุบัน
- จำนวนการจองเดือนนี้นับตาม `createdAt` และไม่นับรายการสถานะ `CANCELLED`
- จำนวนสัตว์ที่กำลังเข้าพักนับสัตว์ในการจองสถานะ `CHECKED_IN`
- การจองล่าสุดเรียงตาม `createdAt` จากใหม่ไปเก่า และใช้ `id`
จากมากไปน้อยเมื่อวันเวลาสร้างเท่ากัน โดยไม่ได้ตัดรายการที่ยกเลิกออก

โค้ดที่เกี่ยวข้อง: `DashboardController`, `DashboardServiceImpl`,

`DashboardResponse`, `MonthlyRevenueResponse`,

`RevenueBookingResponse`, `RecentBookingMapper`,

RevenueBookingMapper, ReceiptRepository,
BookingReceiptController, SecurityConfig, admin/index.html,
dashboard-revenue.js

ภาคผนวก ข
พจนานุกรมข้อมูล (Data Dictionary)

ตาราง users

ตาราง **users** เก็บบัญชีผู้ใช้ทุกบทบาท ได้แก่ ลูกค้า พนักงาน และผู้ดูแลระบบ ใช้ทั้งสำหรับเข้าสู่ระบบและอ้างอิงเจ้าของข้อมูลในตารางอื่น

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสผู้ใช้
first_name	VARCHAR (255)	ไม่ได้	—	ชื่อ
last_name	VARCHAR (255)	ไม่ได้	—	นามสกุล
email	VARCHAR (255)	ไม่ได้	Unique (uk_users_email)	อีเมลสำหรับเข้าสู่ระบบ เก็บเป็นตัวพิมพ์เล็กเสมอ
password	VARCHAR (255)	ไม่ได้	เก็บเป็น BCrypt hash	รหัสผ่านที่เข้ารหัสแล้ว ไม่เก็บรหัสจริง
phone_number	VARCHAR (255)	ไม่ได้	—	เบอร์โทรศัพท์
role	VARCHAR (255)	ไม่ได้	เก็บค่า enum เป็นข้อความ, CHECK เฉพาะ 3 ค่า (ck_users_role)	บทบาท: CUSTOMER, STAFF หรือ ADMIN
active	BOOLEAN	ไม่ได้	DEFAULT TRUE	true = ใช้งานได้, false = ถูกระงับ เข้าสู่ระบบไม่ได้
created_at	TIMESTAMP	ไม่ได้	กำหนดอัตโนมัติตอนสร้าง	เวลาที่สร้างบัญชี

ความสัมพันธ์

- ผู้ใช้หนึ่งคนเป็นเจ้าของสัตว์เลี้ยงได้หลายตัว: `pets.owner_id → users.id`
- ผู้ใช้หนึ่งคนมีการจองได้หลายรายการ: `bookings.user_id → users.id`
- ผู้ใช้หนึ่งคน (พนักงานหรือผู้ดูแลระบบ) บันทึกรายงานการดูแลได้หลายฉบับ:
`daily_care_reports.recorded_by_id → users.id`
- ตาราง **users** ไม่มี Foreign Key ไปตารางอื่น

Index

- **email** มี index อัตโนมัติจาก Unique constraint ใช้ตอนเข้าสู่ระบบ (`findByEmail`)
- ตารางที่อ้างอิงถึง **users** มี index บน Foreign Key: `idx_pets_owner_id`, `idx_bookings_user_id`, `idx_care_reports_recorded_by`

กฎการทำงานที่เกี่ยวข้อง

- เมื่อสมัครสมาชิก `AuthServiceImpl.register()` แปลงอีเมลเป็นตัวพิมพ์เล็ก ตรวจสอบอีเมลซ้ำ เข้ารหัสรหัสผ่านด้วย `BCrypt` และกำหนด `role = CUSTOMER`, `active = true` ผู้สมัครเลือกบทบาทเองไม่ได้
- `User.prePersist()` กำหนด `created_at` และ `active = true` ถ้ายังไม่มีค่า
- ตอนเข้าสู่ระบบ `CustomUserDetailsService` ค้นหาด้วยอีเมลตัวพิมพ์เล็ก และปิดการใช้งานบัญชีที่ `active = false` ผู้ใช้ที่ถูกระงับจึงเข้าสู่ระบบไม่ได้
- ผู้ดูแลระบบแก้ไขชื่อ อีเมล เบอร์โทร บทบาท และสถานะได้ผ่าน `UserServiceImpl.updateUser()` แต่เปลี่ยนบทบาท ระงับบัญชี หรือเปลี่ยนอีเมลของตัวเองไม่ได้ เพื่อป้องกันการล็อกตัวเองออกจากระบบ
- `RegisterRequest` และ `UpdateUserRequest` ตรวจสอบรูปแบบอีเมล และเบอร์โทรต้องขึ้นต้นด้วย 0 ยาว 9–10 หลัก (`^0\d{8,9}$`) ข้อกำหนดนี้ตรวจที่ชั้นคำขอ ไม่ได้ประกาศเป็น `CHECK` constraint
- ไม่มีการลบแฉาผู้ใช้ ใช้การระงับ (`active = false`) แทน ข้อมูลการจองและรายงานที่อ้างอิงผู้ใช้จึงไม่หาย

ข้อมูลตัวอย่าง

`V2__insert_sample_data.sql` สร้างบัญชีเดโม 2 บัญชี คือ `admin@petshotel.test` (ADMIN) และ `staff@petshotel.test` (STAFF)

รหัสผ่านมาจาก Flyway placeholder `demopasswordhash`

- เครื่องนักพัฒนาและ Docker ใช้ค่าใน `application.properties`
- เว็บที่ deploy ตั้งค่าแยกผ่าน environment variable `SPRING_FLYWAY_PLACEHOLDERS_DEMOPASSWORDHASH` รหัสผ่านจริงจึงไม่อยู่ใน repository

แหล่งอ้างอิงในโค้ด

User, UserRole, UserRepository, AuthServiceImpl,
 CustomUserDetailsService, UserServiceImpl, RegisterRequest,
 UpdateUserRequest, db/migration/V1__create_tables.sql,
 db/migration/V2__insert_sample_data.sql

ตาราง pets

ตาราง **pets** เก็บข้อมูลสัตว์เลี้ยง โดยสัตว์เลี้ยงแต่ละตัวมีเจ้าของหนึ่งบัญชี

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่างค่า
id	BIGINT	ไม่ได้	Primary Key, IDENTITY BY DEFAULT	รหัสสัตว์เลี้ยง	2
name	VARCHAR(255)	ไม่ได้	—	ชื่อสัตว์เลี้ยง	โมจิ
type	VARCHAR(255)	ไม่ได้	pets_type_check จำกัดค่าเป็น DOG, CAT, OTHER	ประเภทสัตว์เลี้ยง	DOG
breed	VARCHAR(255)	ได้	—	สายพันธุ์	ชิบะอิ누
age	INTEGER	ได้	—	อายุเป็นเดือน	3
weight	DOUBLE PRECISION	ได้	—	น้ำหนักเป็นกิโลกรัม	7.5
medical_note	VARCHAR(255)	ได้	—	ข้อมูลสุขภาพหรือโรคประจำตัว	ไม่มีโรคประจำตัว
feeding_in	VARCHAR	ได้	—	คำแนะนำเรื่องอาหาร	อาหารเม็ดวันละ 2 มื้อ

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่างค่า
struc tion	AR(255)				
speci al_no te	VARCH AR(255)	ได้	—	ข้อควรดูแลเพิ่มเติม	กลัวเสียงดัง
image _url	VARCH AR(255)	ได้	—	ตำแหน่งรูปภาพสัตว์เลี้ยง	/uploads/pets/550e8400-e29b-41d4-a716-446655440000.jpg
gende r	VARCH AR(255)	ได้	—	เพศของสัตว์เลี้ยง	เพศผู้
owner _id	BIGINT	ไม่ได้	Foreign Key → users.id	บัญชีเจ้าของสัตว์เลี้ยง	1
activ e	BOOLE AN	ไม่ได้	—	true = ยังใช้งาน, false = นำออกจากรายการที่ใช้ งาน	true

ตัวอย่างค่าเป็นข้อมูลสมมติเพื่อแสดงรูปแบบ ไม่ใช่ข้อมูลจริงหรือค่าเริ่มต้นของฐานข้อมูล
คอลัมน์ที่อนุญาตให้เป็น NULL อาจไม่มีค่าได้

ความสัมพันธ์

- สัตว์เลี้ยงหลายตัวเป็นของผู้ใช้หนึ่งคน: `pets.owner_id` อ้างถึง `users.id`
- ใน Entity ความสัมพันธ์ `Pet.owner` เป็น `ManyToOne` และโหลดข้อมูลแบบ `LAZY`

กฎการทำงานที่เกี่ยวข้อง

- เมื่อเพิ่มสัตว์เลี้ยง `PetServiceImpl.createPet()` กำหนด `active = true`
โดยไม่ได้กำหนดค่า `DEFAULT` ไว้ในฐานข้อมูล
- เมื่อลูกค้าสัตว์เลี้ยงออก `PetServiceImpl.deactivatePet()` กำหนด
`active = false` โดยไม่ได้ลบแถวข้อมูล

- การค้นหารายการสัตว์เลี้ยงของเจ้าของแสดงเฉพาะแถวที่ **active = true**
- **CreatePetRequest** และ **UpdatePetRequest** ตรวจสอบให้อายุไม่ติดลบ และน้ำหนักมากกว่าศูนย์ ข้อกำหนดสองข้อนี้ตรวจสอบที่ชั้นคำขอ ไม่ได้ประกาศเป็น CHECK constraint ใน Entity

แหล่งอ้างอิงในโค้ด

Pet, PetType, PetServiceImpl, CreatePetRequest, UpdatePetRequest

ตาราง rooms

ตาราง **rooms** เก็บข้อมูลห้องพักสัตว์เลี้ยง ได้แก่ ความจุ ราคา สถานะการเปิดให้อยู่ และรูปห้อง

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสห้อง
room_number	VARCHAR(255)	ไม่ได้	Unique (uk_rooms_room_number)	หมายเลขห้อง เช่น 101
name	VARCHAR(255)	ไม่ได้	—	ชื่อหรือประเภทห้อง เช่น Deluxe
description	VARCHAR(255)	ได้	—	รายละเอียดห้อง
capacity	INTEGER	ไม่ได้	CHECK capacity >= 1 (ck_rooms_capacity)	จำนวนสัตว์เลี้ยงสูงสุดที่ห้องรองรับ
price_per_pet_per_night	NUMERIC(10,2)	ไม่ได้	CHECK >= 0 (ck_rooms_price)	ราคาต่อสัตว์ 1 ตัวต่อ 1 คืน (บาท)
status	VARCHAR(255)	ไม่ได้	เก็บค่า enum เป็นข้อความ, CHECK เฉพาะ 3 ค่า (ck_rooms_status)	สถานะห้อง: ACTIVE = เปิดให้อยู่, MAINTENANCE = ปิดซ่อม, INACTIVE = ปิดใช้งาน
image_url	VARCHAR(500)	ได้	—	path รูปห้อง เช่น /uploads/rooms/<uuid>.jpg ไม่มีรูปเป็น NULL

ความสัมพันธ์

- ห้องหนึ่งห้องมีการจองได้หลายรายการ: `bookings.room_id → rooms.id`
- ตาราง `rooms` ไม่มี Foreign Key ไปตารางอื่น

Index

- `room_number` มี index อัตโนมัติจาก Unique constraint
- ตาราง `bookings` มี index `idx_bookings_room_dates` บน (`room_id`, `check_in_date`, `check_out_date`)
ใช้ค้นหาการจองที่ช่วงวันซ้อนกันของห้องเดียวกันตอนตรวจห้องว่าง

กฎการทำงานที่เกี่ยวข้อง

- ห้องใหม่ที่สร้างผ่าน `RoomServiceImpl.createRoom()` มีสถานะ `ACTIVE` เสมอ
- `RoomServiceImpl` ตรวจสอบหมายเลขห้องซ้ำก่อนบันทึกทั้งตอนสร้างและแก้ไข
ซ้ำแล้วตอบ 409 และฐานข้อมูลมี Unique constraint กันอีกชั้น
- การปิดใช้งานห้อง (`deactivateRoom()`) เปลี่ยน `status` เป็น `INACTIVE`
ไม่ลบแถว การจองเก่าที่อ้างอิงห้องจึงยังอยู่
- เฉพาะห้อง `ACTIVE` เท่านั้นที่แสดงให้ลูกค้าเห็น (`getActiveRooms()`, `/api/rooms/search`), ค้นหาห้องว่างได้ และจองได้
(`BookingServiceImpl` ปฏิเสธการจองห้องที่ไม่ใช่ `ACTIVE`)
- ความจุและราคาถูกตรวจ 2 ชั้น: `CreateRoomRequest` / `UpdateRoomRequest`
ตรวจก่อน (ตอบ 400 พร้อมข้อความที่อ่านง่าย) และฐานข้อมูลมี CHECK constraint
(`ck_rooms_capacity`, `ck_rooms_price`) กันข้อมูลผิดที่ไม่ได้ผ่านแอป
- `image_url` กำหนดโดย `LocalFileStorageService` เท่านั้น รับเฉพาะไฟล์
JPG/PNG ขนาดไม่เกิน 5 MB ตั้งชื่อไฟล์ใหม่เป็น UUID
และลบไฟล์เก่าเมื่อเปลี่ยนหรือลบรูป
- ราคาในตารางนี้คือราคาปัจจุบัน เมื่อจองแล้ว ราคา ณ เวลาที่จองจะถูกบันทึกไว้ใน
`bookings.room_amount` ถ้าแก้ราคาห้องภายหลัง การจองเก่าจึงไม่เปลี่ยน

ข้อมูลตัวอย่าง

V2__insert_sample_data.sql เพิ่มห้องตัวอย่าง 4 ห้อง (101, 102 Standard · 201 Deluxe · 301 Family Suite) สถานะ **ACTIVE** ทั้งหมด

แหล่งอ้างอิงในโค้ด

Room, RoomStatus, RoomRepository, RoomServiceImpl, RoomMapper, CreateRoomRequest, UpdateRoomRequest, UpdateStatusRequest, LocalFileStorageService, db/migration/V1__create_tables.sql

ตาราง bookings

ตาราง **bookings** เก็บข้อมูลการจองห้องพักของ PetsHotel ได้แก่ ผู้จอง ห้อง ช่วงวันเข้าพัก สถานะการจอง โปรโมชั่น ราคา ณ เวลาจอง และข้อมูลการชำระเงิน

ชนิดข้อมูล PostgreSQL อ้างอิงจาก Entity **Booking** และ ER Diagram ปัจจุบันของโปรเจกต์

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสการจอง
user_id	BIGINT	ไม่ได้	Foreign Key → users.id	ผู้ใช้ที่เป็นเจ้าของการจอง
room_id	BIGINT	ไม่ได้	Foreign Key → rooms.id	ห้องพักที่จอง
promotion_id	BIGINT	ได้	Foreign Key → promotions.id	โปรโมชั่นที่เลือกใช้ ไม่มีโปรโมชั่นเป็น NULL
check_in_date	DATE	ไม่ได้	ตรวจสอบช่วงวันที่ใน Service	วันเริ่มเข้าพัก
check_out_date	DATE	ไม่ได้	ต้องหลัง check_in_date ตรวจสอบใน Service	วันสิ้นสุดการเข้าพัก
status	VARCHAR (255)	ไม่ได้	เก็บ BookingStatus เป็นข้อความ	สถานะการจอง
room_amount	NUMERIC (10,2)	ได้	กำหนด precision 10, scale 2	ค่าห้องพัก ณ เวลาจอง หน่วยเป็นบาท
service_amount	NUMERIC (10,2)	ได้	กำหนด precision 10, scale 2	ค่าบริการเสริมรวม ณ เวลาจอง หน่วยเป็นบาท

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย
surcharge_amount	NUMERIC (10,2)	ได้	กำหนด precision 10, scale 2	ค่าธรรมเนียมเพิ่มเติม เช่น ค่าบริการวันหยุด หน่วยเป็นบาท
discount_amount	NUMERIC (10,2)	ได้	กำหนด precision 10, scale 2	ส่วนลดที่เลือกใช้ในการจอง หน่วยเป็นบาท
total_price	NUMERIC (38,2)	ได้	ชนิดตาม ER Diagram; Entity ไม่กำหนด precision/scale เอง	ยอดสุทธิของการจอง หน่วยเป็นบาท
promotion_name	VARCHAR (255)	ได้	updatable = false ใน JPA mapping	ชื่อโปรโมชั่น ณ เวลาจอง ไม่มีโปรโมชั่นเป็น NULL
payment_status	VARCHAR (255)	ไม่ได้	เก็บ PaymentStatus เป็นข้อความ	สถานะการชำระเงิน
paid_amount	NUMERIC (10,2)	ได้	กำหนด precision 10, scale 2	ยอดเงินที่ยืนยันรับชำระ ยังไม่รับชำระเป็น NULL
paid_at	TIMESTAMP	ได้	กำหนดเมื่อยืนยันรับชำระเงิน	วันและเวลารับชำระเงิน ยังไม่รับชำระเป็น NULL
created_at	TIMESTAMP	ไม่ได้	กำหนดใน @PrePersist หากยังไม่มีค่า	วันและเวลาที่สร้างการจอง

ค่าสถานะ

สถานะการจอง: status

ค่า	ความหมาย
PENDING	รอพนักงานยืนยันการจอง
CONFIRMED	ยืนยันการจองแล้ว
CHECKED_IN	สัตว์เลี้ยงเข้าพักแล้ว
CHECKED_OUT	สิ้นสุดการเข้าพักแล้ว
CANCELLED	ยกเลิกการจองแล้ว

สถานะการชำระเงิน: payment_status

ค่า	ความหมาย
UNPAID	ยังไม่ได้ยืนยันรับชำระเงิน

ค่า	ความหมาย
PAID	ยืนยันรับชำระเงินแล้ว

status และ **payment_status** เป็นคนละข้อมูล

การเปลี่ยนสถานะการจองไม่ได้เปลี่ยนสถานะการชำระเงินโดยอัตโนมัติ

ความสัมพันธ์

- ผู้ใช้หนึ่งคนมีการจองได้หลายรายการ: **bookings.user_id** → **users.id**
- ห้องหนึ่งห้องมีการจองได้หลายรายการในประวัติ: **bookings.room_id** → **rooms.id** แต่ระบบไม่อนุญาตให้มีการจองที่ยังครองห้องทับช่วงวันกัน
- การจองหนึ่งรายการเลือกโปรโมชั่นได้ไม่เกินหนึ่งรายการ:
bookings.promotion_id → **promotions.id** โดยไม่บังคับเลือก
- การจองหนึ่งรายการมีสัตว์เลี้ยงผ่านตาราง **booking_pets**:
booking_pets.booking_id → **bookings.id**
- การจองหนึ่งรายการมีบริการเสริมได้หลายรายการ:
booking_extra_services.booking_id → **bookings.id**
- การจองหนึ่งรายการมีใบเสร็จได้ไม่เกินหนึ่งใบ: **receipts.booking_id** → **bookings.id** โดย **receipts.booking_id** มี Unique constraint
- ใน Entity ความสัมพันธ์ **user**, **room** และ **promotion** เป็น ManyToOne และโหลดแบบ LAZY
- **Booking.bookingPets** และ **Booking.extraServices** เป็น OneToMany โดยกำหนด **cascade = ALL** และ **orphanRemoval = true**
- **bookingPets** และ **extraServices** เป็นรายการความสัมพันธ์ใน Java ไม่ใช่คอลัมน์ของตาราง **bookings**

กฎการทำงานที่เกี่ยวข้อง

การสร้างการจอง

- Controller กำหนดผู้จองจากบัญชีที่ล็อกอิน ไม่ยอมให้ลูกค้าระบุผู้จองเป็นบัญชีอื่น
- ต้องระบุวันเข้าและวันออก โดย **check_out_date** ต้องหลัง **check_in_date**
- ต้องเลือกสัตว์เลี้ยงอย่างน้อยหนึ่งตัว รหัสสัตว์ต้องไม่ซ้ำ
สัตว์ต้องยังใช้งานอยู่และเป็นของผู้จอง

- ห้องต้องมีสถานะ **ACTIVE** และจำนวนสัตว์ต้องไม่เกินความจุห้อง
- ก่อนบันทึก ระบบตรวจว่าสัตว์และห้องไม่มีการจองซ้อนในช่วงวันที่เลือก
- สถานะที่นับว่าสงวนห้องและสัตว์คือ **PENDING**, **CONFIRMED** และ **CHECKED_IN**
- การจองใหม่ที่สร้างผ่าน **BookingServiceImpl** มีสถานะ **PENDING** และสถานะชำระเงิน **UNPAID**
- **Booking.prePersist()** กำหนด **status**, **paymentStatus** และ **createdAt** เมื่อค่านั้นยังเป็น **NULL** โดย **paymentStatus** มีค่าเริ่มต้น **UNPAID** ใน Entity ด้วย
- ค่าเริ่มต้นเหล่านี้กำหนดในโค้ด Java ไม่ได้ประกาศเป็น **SQL DEFAULT** ใน Entity

ราคาและโปรโมชั่น

- **BookingServiceImpl** เรียก **PricingService.calculate()** แล้วเก็บค่าห้อง บริการเสริม ค่าธรรมเนียม ส่วนลด และยอดสุทธิลงในการจอง
- ยอดสุทธิตำหนดจากค่าห้อง + ค่าบริการเสริม + ค่าธรรมเนียม - ส่วนลดที่เลือกใช้
- ราคาที่บันทึกเป็นราคา ณ เวลาจอง การแก้ราคาห้อง บริการ หรือโปรโมชั่นภายหลังไม่คำนวณราคาการจองเดิมใหม่โดยอัตโนมัติ
- หากเลือกโปรโมชั่น ระบบตรวจว่าโปรโมชั่นมีอยู่ เปิดใช้งาน และใช้ได้ในวันทำการจองตามเขตเวลา **Asia/Bangkok**
- ระบบเก็บทั้ง **promotion_id** และ **promotion_name** เพื่ออ้างอิงโปรโมชั่นและเก็บชื่อ ณ เวลาจอง
- **promotion_name** กำหนด **updatable = false** เพื่อไม่ให้ JPA อัปเดตคอลัมน์นี้หลังสร้างรายการ
- แม้อัลบั้มราคาใน Entity อนุญาตให้เป็น **NULL** แต่ขั้นตอนสร้างการจองปัจจุบันคำนวณและกำหนดราคาก่อนบันทึก

การเปลี่ยนสถานะและการชำระเงิน

- การยืนยัน เช็กอิน เช็กเอาท์ และยกเลิกใช้ **State Pattern** ผ่าน **BookingServiceImpl**
- การแก้สถานะโหลดการจองผ่าน **findByIdForUpdate()** ที่ใช้ **PESSIMISTIC_WRITE** lock ภายใน transaction
- ยกเลิกได้เฉพาะ **PENDING** หรือ **CONFIRMED** ที่ยังไม่ชำระเงิน

- การยกเลิกเปลี่ยนสถานะเป็น **CANCELLED**
โดยไม่ลบข้อมูลการจองและรายการที่เกี่ยวข้อง
- การยืนยันรับชำระเงินครั้งแรกทำได้เมื่อสถานะเป็น **CONFIRMED**, **CHECKED_IN** หรือ **CHECKED_OUT** และยอดสุทธิไม่เป็น **NULL** หรือค่าติดลบ
- **PaymentServiceImpl.confirmPayment()** กำหนด **payment_status** = **PAID**, **paid_amount** = **total_price** และ **paid_at** เป็นเวลาปัจจุบันในเขตเวลา **Asia/Bangkok** พร้อมประสานการสร้างใบเสร็จ
- **paid_at** และ **created_at** ใช้ **LocalDateTime** จึงไม่ได้เก็บชื่อเขตเวลาหรือ **UTC offset** ไว้ในตัวค่า โดย **created_at** กำหนดด้วย **LocalDateTime.now()** ตามเวลาของระบบ
- กฎช่วงวันที่ การตรวจห้อง/สัตว์ซ่อน และเงื่อนไขการทำรายการข้างต้นตรวจในโค้ด ไม่ได้ประกาศเป็น **CHECK** constraint ของกฎเหล่านี้ใน Entity

แหล่งอ้างอิงในโค้ด

Booking, BookingStatus, PaymentStatus, BookingPet, Receipt, BookingController, BookingRestController, BookingServiceImpl, AvailabilityServiceImpl, PaymentServiceImpl, BookingRepository, PricingServiceImpl, BookingMapper

ชนิดคอลัมน์ประกอบการอ้างอิงจาก `doc/diagrams/er-diagram.puml`

ตาราง booking_pets

ตาราง **booking_pets** เชื่อมการจองกับสัตว์เลี้ยงใน **PetsHotel** โดยหนึ่งแถวหมายถึงสัตว์เลี้ยงหนึ่งตัวที่เข้าพักในการจองหนึ่งรายการ

ตารางนี้รองรับความสัมพันธ์ Many-to-Many ระหว่าง **bookings** และ **pets** และมีรหัสของตนเองเพื่อให้รายงานการดูแลและบริการเสริมอ้างอิงถึงสัตว์ในการเข้าพักครั้งนั้นได้

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสรายการสัตว์เลี้ยงในการจอง
booking_id	BIGINT	ไม่ได้	Foreign Key → bookings.id	การจองที่สัตว์เลี้ยงเข้าพัก
pet_id	BIGINT	ไม่ได้	Foreign Key → pets.id	สัตว์เลี้ยงที่เข้าพักในการจองนี้

ความสัมพันธ์

- การจองหนึ่งรายการมีรายการสัตว์เลี้ยงได้หลายรายการ:
`booking_pets.booking_id → bookings.id`
- สัตว์เลี้ยงหนึ่งตัวมีประวัติการเข้าพักหลายครั้งได้: `booking_pets.pet_id → pets.id`
- รายการสัตว์เลี้ยงในการจองหนึ่งรายการมีรายงานการดูแลได้หลายวัน:
`daily_care_reports.booking_pet_id → booking_pets.id`
- รายการสัตว์เลี้ยงในการจองหนึ่งรายการรับบริการเสริมได้หลายรายการ:
`booking_extra_services.booking_pet_id → booking_pets.id`
- ใน Entity ความสัมพันธ์ **booking** และ **pet** เป็น **ManyToOne** และโหลดข้อมูลแบบ **LAZY**
- **Booking.bookingPets** เป็น **OneToMany** โดยกำหนด **cascade = ALL** และ **orphanRemoval = true**
- วันที่เข้าพักและสถานะเก็บในตาราง **bookings** ส่วนชื่อ ประเภท และเจ้าของสัตว์เก็บในตาราง **pets** ไม่ได้เก็บซ้ำใน **booking_pets**

กฎการทำงานที่เกี่ยวข้อง

การเพิ่มสัตว์ในการจอง

- การสร้างการจองต้องเลือกสัตว์เลี้ยงอย่างน้อยหนึ่งตัว
- รหัสสัตว์ในคำขอ **petIds** ต้องไม่เป็น **NULL** และต้องไม่ซ้ำกัน
- สัตว์ทุกตัวต้องมีอยู่ในระบบ ยังใช้งานอยู่ และเป็นของผู้จอง
- จำนวนสัตว์ต้องไม่เกินความจุห้องที่เลือก
- **BookingServiceImpl.prepareBooking()** เรียก **Booking.addPet(pet)** สำหรับสัตว์แต่ละตัว
- **Booking.addPet()** สร้าง **BookingPet** โดยกำหนดทั้ง **booking** และ **pet** แล้วเพิ่มลงใน **Booking.bookingPets**
- เมื่อบันทึก Booking รายการ BookingPet ถูกบันทึกผ่าน cascade ของ JPA
- Entity ไม่ได้ประกาศ Unique constraint สำหรับคู่ **booking_id** กับ **pet_id**
 การป้องกันสัตว์ซ้ำในการจองปัจจุบันตรวจในคำขอที่ Service

การตรวจสอบสัตว์ที่มีการจองซ้อน

- `BookingPetRepository.findOverlappingPetIds()`
ค้นหาลิสต์สัตว์ที่มีการจองทับช่วงวันที่ต้องการ
- การสร้างการจองนับเฉพาะสถานะ `PENDING`, `CONFIRMED` และ `CHECKED_IN`
เป็นสถานะที่ยังสงวนสัตว์เลี้ยง
- เงื่อนไขช่วงวันที่ซ้อนกันคือ `existing.checkInDate < requested.checkOutDate` และ `existing.checkOutDate > requested.checkInDate`
- ช่วงเข้าพักนับรวมวันเช็กอิน แต่ไม่รวมวันเช็กเอาท์
การจองใหม่ที่เริ่มในวันเช็กเอาท์ของการจองเดิมจึงไม่ถือว่าซ้อนกัน
- หากพบสัตว์ที่จองซ้อน ระบบปฏิเสธการสร้างการจองด้วย `PetNotAvailableException`
- `BookingServiceImpl.getPetAvailability()`
ใช้ข้อมูลนี้แจ้งหน้าฟอร์มให้ปิดตัวเลือกสัตว์ที่ติดจองในช่วงวันที่เลือก
- การป้องกันการจองซ้อนตรวจสอบในโค้ด ไม่ได้ประกาศเป็น `constraint` ของตาราง `booking_pets`

รายงานการดูแลและบริการเสริม

- รายงานการดูแลอ้างอิงถึง `booking_pets.id`
เพื่อระบุทั้งสัตว์และการเข้าพักครั้งที่เกี่ยวข้อง
- ตาราง `daily_care_reports` มี Unique constraint สำหรับคู่ `booking_pet_id` กับ `report_date`
เพื่อไม่ให้มีรายงานซ้ำของสัตว์ในการเข้าพักเดียวกันในวันเดียวกัน
- บริการเสริมอ้างอิงถึง `booking_pets.id`
เพื่อระบุว่าสัตว์ตัวไหนในการจองเป็นผู้รับบริการ
- `BookingServiceImpl` สร้าง `bookingPetsByPetId` เพื่อจับคู่รหัสสัตว์กับ `BookingPet` แล้วนำไปผูกกับรายการบริการเสริม
- ผู้รับบริการต้องเป็นสัตว์ในการจองนั้น
และไม่สามารถเลือกสัตว์ตัวเดิมซ้ำสำหรับบริการเดียวกันในคำขอได้

การเก็บประวัติ

- สัตว์ตัวเดิมสามารถมี `BookingPet` คนละรายการสำหรับการเข้าพักคนละครั้งได้

- การยกเลิกการจองเปลี่ยนสถานะใน Booking โดยไม่ลบ BookingPet
- เมื่อการจองเป็น **CANCELLED** หรือ **CHECKED_OUT** รายการสัตว์ยังอยู่เป็นประวัติ แต่ไม่นับเป็นรายการที่สงวนสัตว์ในการตรวจช่วงวันที่ซ้อน
- Repository มีเมธอด **deleteByBookingId()** แต่เส้นทางยกเลิกการจองปัจจุบันไม่ได้เรียกเมธอดนี้

ตัวอย่างข้อมูล

สมมติว่าการจองรหัส 101 มีสัตว์เลี้ยงรหัส 3 และ 4 ส่วนสัตว์รหัส 3 กลับมาเข้าพักอีกครั้งในการจองรหัส 102 โดยช่วงวันที่ไม่ซ้อนกัน

id	booking_id	pet_id
201	101	3
202	101	4
203	102	3

- รหัส 201 และ 203 อ้างถึงสัตว์ตัวเดียวกัน แต่เป็นการเข้าพักคนละครั้ง
- รายงานการดูแลของสัตว์รหัส 3 ในการจอง 101 ต้องอ้าง **booking_pet_id = 201**
- บริการเสริมของสัตว์รหัส 3 ในการจอง 102 ต้องอ้าง **booking_pet_id = 203**
- รหัสในตัวอย่างเป็นข้อมูลสมมติ ไม่ใช่ค่าเริ่มต้นของฐานข้อมูล

เมธอด Repository ที่เกี่ยวข้อง

เมธอด	หน้าที่
findByBookingId(bookingId)	ค้นรายการสัตว์ทั้งหมดในการจองที่ระบุ
deleteByBookingId(bookingId)	ลบรายการสัตว์ของการจองที่ระบุ
findOverlappingPetIds(...)	ค้นรหัสสัตว์ที่ติดจองในช่วงวันที่และสถานะที่ระบุ
findReportableBookingPets(statuses)	ค้นรายการสัตว์ในการจองตามสถานะที่ผู้เรียกกำหนด พร้อมโหลดข้อมูลการจอง ผู้จอง และสัตว์

แหล่งอ้างอิงในโค้ด

BookingPet, Booking, Pet, BookingPetRepository, BookingServiceImpl, DailyCareReport, BookingExtraService, PetNotAvailableException, BookingPetRepositoryTest

ตาราง extra_services

ตาราง **extra_services** เก็บข้อมูลบริการเสริมของ petshotel เช่น อาบน้ำและตัดเล็บ โดยกำหนดชื่อ รายละเอียด ราคา และสถานะเปิดใช้งาน บริการหนึ่งรายการสามารถถูกเลือกใช้ในการจองหลายรายการได้

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่าง
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสบริการเสริม	1
name	VARCHAR(255)	ไม่ได้	—	ชื่อบริการเสริม	อาบน้ำ
description	TEXT	ได้	—	รายละเอียดบริการเสริม	บริการอาบน้ำและเป่าขนสัตว์เลี้ยง
price	NUMERIC(10,2)	ไม่ได้	ทศนิยม 2 ตำแหน่ง	ราคาบริการเสริมต่อหน่วย หน่วยเป็นบาท	250.00
active	BOOLEAN	ไม่ได้	—	true = เปิดใช้งาน, false = ปิดใช้งาน	true

ความสัมพันธ์

- บริการเสริมหนึ่งรายการสามารถถูกอ้างอิงโดยรายการบริการที่จองหลายรายการ: `booking_extra_services.extra_service_id` อ้างถึง `extra_services.id`
- ใน Entity ความสัมพันธ์ `BookingExtraService.extraService` เป็น `ManyToOne` และโหลดข้อมูลแบบ `LAZY`
- `booking_extra_services` เก็บบริการที่เลือกในการจอง พร้อมสัตว์เลี้ยงผู้รับบริการ จำนวน และราคา ณ เวลาจอง

กฎการทำงานที่เกี่ยวข้อง

- `ExtraServiceServiceImpl.createExtraService()` และ `updateExtraService()` ตรวจสอบว่าชื่อไม่ว่าง ราคาต้องไม่เป็น NULL และต้องไม่ติดลบ โดยอนุญาตราคา 0 บาท

- ก่อนบันทึก Service ตัดช่องว่างหัวท้ายของชื่อด้วย `trim()`
- `ExtraServiceRequest` ตรวจสอบให้ราคามีตัวเลขส่วนจำนวนเต็มไม่เกิน 8 หลัก และทศนิยมไม่เกิน 2 ตำแหน่ง
- การสร้างบริการใหม่ หากไม่ได้ระบุ `active` หรือระบุเป็น `NULL` Service จะกำหนดเป็น `true`
- การแก้ไขบริการ หาก `active` เป็น `NULL` จะคงสถานะเดิมไว้ แต่หากส่ง `true` หรือ `false` จะเปลี่ยนตามค่าที่ส่งมา
- `description` ไม่จำเป็นต้องระบุ หากส่งเป็น `NULL` ตอนแก้ไข รายละเอียดเดิมจะถูกเปลี่ยนเป็น `NULL`
- `ExtraServiceServiceImpl.getAllExtraServices()`
แสดงบริการทั้งหมด ส่วน `getActiveExtraServices()` แสดงเฉพาะบริการที่ `active = true`
- `ExtraServiceServiceImpl.deleteExtraService()` กำหนด `active = false` โดยไม่ได้ลบแถวข้อมูล
- เมื่อสร้างการจอง `BookingServiceImpl` ตรวจสอบว่าบริการที่เลือกเปิดใช้งาน มีราคา และราคาไม่ติดลบ
- สัตว์เลี้ยงผู้รับบริการต้องอยู่ในการจองนั้น และไม่สามารถเลือกสัตว์ตัวเดิมซ้ำสำหรับบริการเดียวกันได้
- โค้ดสร้างการจองปัจจุบันสร้าง `BookingExtraService` แยกตามสัตว์เลี้ยงผู้รับบริการ โดยกำหนด `quantity = 1` และคัดลอก `ExtraService.price` ไปเป็น `unit_price` และ `total_price`
- ราคาที่คัดลอกไว้ทำให้การแก้ราคาใน `extra_services` ภายหลังไม่เปลี่ยนราคาบริการที่บันทึกในการจองเดิมโดยอัตโนมัติ
- `ExtraServicePricingStrategy` รวมค่าบริการจาก `unit_price × quantity` ของแต่ละรายการ
- กฎตรวจสอบชื่อและราคาที่ไม่ติดลบตรวจสอบที่ชั้นคำขอและ Service ไม่ได้ประกาศเป็น `CHECK constraint` ใน Entity

ตัวอย่างการคำนวณค่าบริการ

สมมติว่าบริการอาบน้ำราคา 250.00 บาท เปิดใช้งาน และลูกค้าเลือกให้สัตว์เลี้ยง 2

ตัวในการจองเดียวกัน ระบบจะสร้างรายการบริการแยกตามสัตว์เลี้ยง

รายการ	ราคาต่อหน่วย	จำนวน	ค่าบริการ
--------	--------------	-------	-----------

รายการ	ราคาต่อหน่วย	จำนวน	ค่าบริการ
อาบน้ำให้สัตว์เลี้ยงตัวที่ 1	250.00 บาท	1	250.00 บาท
อาบน้ำให้สัตว์เลี้ยงตัวที่ 2	250.00 บาท	1	250.00 บาท
รวม	—	—	500.00 บาท

- หากภายหลังแก้ราคาบริการอาบน้ำเป็น 300.00 บาท

รายการบริการที่บันทึกในการจองเดิมยังคงใช้ `unit_price = 250.00`

- หากปิดบริการด้วย `active = false`

ระบบจะไม่อนุญาตให้เลือกบริการนั้นในการสร้างการจองใหม่

แต่ยังเก็บข้อมูลบริการและรายการที่เคยจองไว้

แหล่งอ้างอิงในโค้ด

ExtraService, BookingExtraService, ExtraServiceRequest,
ExtraServiceMapper, ExtraServiceServiceImpl,
ExtraServiceRepository, BookingServiceImpl,
ExtraServicePricingStrategy

ตาราง `booking_extra_services`

ตาราง `booking_extra_services` เก็บรายการบริการเสริมที่เลือกในการจองของ petshotel โดยเชื่อมการจอง บริการเสริม และสัตว์เลี้ยงผู้รับบริการ พร้อมเก็บจำนวนและราคา ณ เวลาจอง

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่าง
<code>id</code>	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสรายการบริการเสริมที่จอง	1
<code>booking_id</code>	BIGINT	ไม่ได้	Foreign Key → <code>bookings.id</code>	การจองที่รายการบริการนี้เป็นส่วนหนึ่ง	101
<code>extra_service_id</code>	BIGINT	ไม่ได้	Foreign Key → <code>extra_services.id</code>	บริการเสริมที่เลือก	1 หมายถึงบริการอาบน้ำ
<code>booking_pet_id</code>	BIGINT	ได้	Foreign Key → <code>booking_pets.id</code>	รายการสัตว์เลี้ยงในการจองที่เป็นผู้รับบริการ	201

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่าง
quantity	INTEGER	ไม่ได้	—	จำนวนหน่วยบริการในรายการนี้	1
unit_price	NUMERIC(10,2)	ไม่ได้	ทศนิยม 2 ตำแหน่ง	ราคาต่อหน่วย ณ เวลาจอง หน่วยเป็นบาท	250.00
total_price	NUMERIC(10,2)	ไม่ได้	ทศนิยม 2 ตำแหน่ง	ราคารวมของรายการบริการ หน่วยเป็นบาท	250.00

ความสัมพันธ์

- รายการบริการเสริมหลายรายการเป็นส่วนหนึ่งของการจองหนึ่งรายการ:
`booking_extra_services.booking_id` อ้างถึง `bookings.id`
- รายการบริการเสริมหลายรายการสามารถเลือกบริการเดียวกันได้:
`booking_extra_services.extra_service_id` อ้างถึง `extra_services.id`
- รายการบริการเสริมหลายรายการสามารถมีสัตว์เลี้ยงผู้รับบริการตัวเดียวกันได้:
`booking_extra_services.booking_pet_id` อ้างถึง `booking_pets.id`
- `booking_pet_id` อ้างถึงรายการสัตว์เลี้ยงในการจอง ไม่ได้อ้างถึง `pets.id` โดยตรง
- ใน Entity ความสัมพันธ์ `booking`, `extraService` และ `bookingPet` เป็น `ManyToOne` และโหลดข้อมูลแบบ `LAZY`
- `Booking.extraServices` เป็น `OneToMany` โดยกำหนด `cascade = ALL` และ `orphanRemoval = true`

กฎการทำงานที่เกี่ยวข้อง

- `BookingServiceImpl.addSelectedServices()`
สร้างรายการบริการแยกตามสัตว์เลี้ยงผู้รับบริการ
- สัตว์เลี้ยงผู้รับบริการต้องอยู่ในการจองนั้น
และไม่สามารถเลือกสัตว์ตัวเดิมซ้ำสำหรับบริการเดียวกันในคำขอสร้างการจองได้

- แม้ Entity อนุญาตให้ `booking_pet_id` เป็น NULL
แต่ขั้นตอนสร้างการจองปัจจุบันกำหนดผู้ให้บริการให้ทุกรายการที่สร้าง
- บริการที่เลือกต้องมีอยู่ เปิดใช้งาน มีราคา และราคาไม่ติดลบ
- ขั้นตอนสร้างการจองปัจจุบันกำหนด `quantity = 1` และคัดลอก
`ExtraService.price` ไปเป็น `unit_price` และ `total_price`
- `Booking.addExtraService()` เพิ่มรายการลงใน `extraServices`
และกำหนดการจองให้รายการนั้นด้วย `service.setBooking(this)`
- ราคาที่บันทึกเป็นราคา ณ เวลาจอง การแก้ราคาบริการใน `extra_services`
ภายหลังจึงไม่เปลี่ยนราคาของรายการที่จองไว้โดยอัตโนมัติ
- `ExtraServicePricingStrategy` ตรวจสอบว่า `unit_price` ไม่เป็น NULL
และไม่ติดลบ ส่วน `quantity` ต้องไม่เป็น NULL และมากกว่า 0
- Strategy คำนวณราคาแต่ละรายการด้วย `unit_price × quantity`
หากรายการมี `total_price` อยู่แล้ว คำนวณต้องเท่ากับผลคำนวณ
- Strategy รวมผลคำนวณของทุกรายการเป็นค่าบริการเสริมของการจอง
- `BookingExtraServiceRepository.findByBookingId()`
ใช้ค้นหารายการบริการเสริมของการจองที่ระบุ
- กฎตรวจจำนวน ราคา และผู้ให้บริการข้างต้นตรวจในโค้ด ไม่ได้ประกาศเป็น CHECK
constraint ใน Entity และไม่ได้ประกาศ UNIQUE constraint
สำหรับคู่บริการกับผู้ให้บริการ

ตัวอย่างรายการบริการที่จอง

สมมติว่าการจองรหัส 101 มีสัตว์เลี้ยง 2 ตัว โดยรายการใน `booking_pets` มีรหัส 201 และ 202 ทั้งสองตัวเลือกบริการอาบน้ำรหัส 1 ราคา 250.00 บาท ระบบจะสร้างรายการดังนี้

id	booking_id	extra_service_id	booking_pet_id	quantity	unit_price	total_price
1	101	1	201	1	250.00	250.00
2	101	1	202	1	250.00	250.00

- ค่าบริการเสริมรวม: $(250.00 \times 1) + (250.00 \times 1) = 500.00$ บาท
- หากแก้ราคาบริการอาบน้ำเป็น 300.00 บาทภายหลัง สองรายการเดิมยังคงเก็บ
`unit_price = 250.00` และ `total_price = 250.00`

- หากส่งสัตว์เลี้ยงที่ไม่ได้อยู่ในการจองรหัส 101 มาเป็นผู้รับบริการ ระบบจะปฏิเสธคำขอสร้างการจองนั้น

แหล่งอ้างอิงในโค้ด

BookingExtraService, Booking, BookingPet, ExtraService, BookingServiceImpl, ExtraServicePricingStrategy, BookingExtraServiceRepository

ตาราง promotions

ตาราง **promotions** เก็บข้อมูลโปรโมชั่นส่วนลด โดยกำหนดประเภทส่วนลด มูลค่า ช่วงวันที่ และสถานะเปิดใช้งาน โปรโมชั่นหนึ่งรายการสามารถใช้กับการจองหลายรายการได้

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่าง
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสโปรโมชั่น	1
name	VARCHAR(255)	ไม่ได้	—	ชื่อโปรโมชั่น	ลดเดือนตุลาคม 10%
description	TEXT	ได้	—	รายละเอียดโปรโมชั่น	NULL
type	VARCHAR(255)	ไม่ได้	เก็บค่า enum เป็นข้อความ	ประเภทส่วนลด: PERCENTAGE หรือ FIXED_AMOUNT	PERCENTAGE
value	NUMERIC(10,2)	ไม่ได้	ทศนิยม 2 ตำแหน่ง	มูลค่าส่วนลด: เปอร์เซนต์เมื่อเป็น PERCENTAGE หรือบาทเมื่อเป็น FIXED_AMOUNT	10.00 หมายถึงลด 10%
start_date	DATE	ได้	—	วันเริ่มต้นโปรโมชั่น	2026-10-01
end_date	DATE	ได้	—	วันสิ้นสุดโปรโมชั่น	2026-10-31
minimum_nights	INTEGER	ได้	—	จำนวนคืนขั้นต่ำที่ต้องเข้าพักเพื่อรับส่วนลด; NULL คือไม่กำหนดขั้นต่ำ	3 หมายถึงพักอย่างน้อย 3 คืน
active	BOOLEAN	ไม่ได้	—	true = เปิดใช้งาน, false = ปิดใช้งาน	true

ความสัมพันธ์

- การจองหลายรายการสามารถใช้โปรโมชั่นเดียวกันได้: `bookings.promotion_id` อ้างถึง `promotions.id`
- การจองไม่จำเป็นต้องใช้โปรโมชั่น จึงอนุญาตให้ `bookings.promotion_id` เป็น NULL
- ใน Entity ความสัมพันธ์ `Booking.promotion` เป็น ManyToOne และโหลดข้อมูลแบบ LAZY
- `Booking` เก็บชื่อโปรโมชั่นขณะจองแยกไว้ใน `promotion_name` และกำหนด `updatable = false`

กฎการทำงานที่เกี่ยวข้อง

- `PromotionServiceImpl.createPromotion()` และ `updatePromotion()` ตรวจสอบข้อมูลก่อนบันทึก โดยชื่อโปรโมชั่นต้องไม่ว่าง และตัดช่องว่างหัวท้ายของชื่อด้วย `trim()`
- มูลค่าส่วนลดต้องมากกว่า 0 หากเป็น PERCENTAGE ต้องไม่เกิน 100
- `CreatePromotionRequest` ตรวจสอบให้มูลค่าส่วนลดมีตัวเลขส่วนจำนวนเต็มไม่เกิน 8 หลัก และทศนิยมไม่เกิน 2 ตำแหน่ง
- แม้ Entity อนุญาตให้ `start_date` และ `end_date` เป็น NULL แต่การเพิ่มหรือแก้ไขผ่านระบบต้องระบุทั้งสองวัน และวันสิ้นสุดต้องไม่ก่อนวันเริ่มต้น โดยเป็นวันเดียวกันได้
- `Promotion.isAvailableOn(date)` คืนค่า true เมื่อโปรโมชั่นเปิดใช้งาน มีวันเริ่มต้นและสิ้นสุดครบ และวันที่ตรวจสอบอยู่ในช่วงดังกล่าว โดยรวมวันเริ่มต้นและวันสิ้นสุดด้วย
- `PromotionServiceImpl.getActivePromotions()` ตรวจสอบวันที่ปัจจุบันในเขตเวลา Asia/Bangkok และแสดงเฉพาะโปรโมชั่นที่ใช้ได้ในวันนี้
- เมื่อสร้างการจอง `BookingServiceImpl` ตรวจสอบว่าโปรโมชั่นเปิดใช้งานและอยู่ในช่วงวันที่กำหนด โดยใช้วันที่สร้างการจองเป็นวันที่ตรวจสอบ

- **PromotionDiscountStrategy** คำนวณส่วนลดแบบ **PERCENTAGE**
จากยอดก่อนหักส่วนลด ส่วน **FIXED_AMOUNT** ให้ส่วนลดได้ไม่เกินยอดดังกล่าว
- หาก **minimum_nights** มีค่า แต่จำนวนคืนที่เข้าพักน้อยกว่าค่านั้น
PromotionDiscountStrategy คืนส่วนลดเป็น 0
- หากไม่ได้เลือกโปรโมชั่น หรือโปรโมชั่นปิดใช้งาน **PromotionDiscountStrategy**
คืนส่วนลดเป็น 0
- **PromotionServiceImpl.deactivatePromotion()** กำหนด **active = false** โดยไม่ได้ลบแถวข้อมูล
- ค่าขอ **CreatePromotionRequest** ปัจจุบันไม่มีช่อง **description** และ **minimum_nights** ดังนั้นการสร้างผ่าน Service นี้ไม่ได้กำหนดสองฟิลด์ดังกล่าว
- กฎตรวจสอบข้อมูลค่าส่วนลด และลำดับวันที่ข้างต้นตรวจสอบที่ชั้นคำขอและ Service
ไม่ได้ประกาศเป็น **CHECK** constraint ใน Entity

ตัวอย่างการคำนวณส่วนลด

สมมติว่าโปรโมชั่นผ่านการตรวจวันที่ เปิดใช้งาน และผ่านเงื่อนไขจำนวนคืนแล้ว

ตัวอย่างนี้แสดงเฉพาะผลของ **PromotionDiscountStrategy**

ประเภท	มูลค่าส่วนลด	ยอดก่อนหักส่วนลด	ส่วนลดที่คำนวณได้
PERCENTAGE	10.00	1,500.00 บาท	150.00 บาท
FIXED_AMOUNT	200.00	1,500.00 บาท	200.00 บาท
FIXED_AMOUNT	200.00	150.00 บาท	150.00 บาท

- เปอร์เซ็นต์: $1,500 \times 10 \div 100 = 150$ บาท
- จำนวนเงิน: เลือกค่าที่น้อยกว่าระหว่างมูลค่าส่วนลดกับยอดก่อนหักส่วนลด
- หากข้อมูลโปรโมชั่นกำหนด **minimum_nights = 3** แต่เข้าพักเพียง 2 คืน
ส่วนลดจะเป็น 0
- หากโปรโมชั่นเริ่มวันที่ 2026-10-01 และสิ้นสุดวันที่ 2026-10-31
จะใช้ได้ทั้งสองวันดังกล่าวเมื่อ **active = true**

แหล่งอ้างอิงในโค้ด

Promotion, PromotionType, Booking, CreatePromotionRequest,
PromotionServiceImpl, BookingServiceImpl,
PromotionDiscountStrategy, PromotionRepository

ตาราง receipts

ตาราง **receipts** เก็บข้อมูลใบเสร็จของ PetsHotel หลังยืนยันรับชำระเงิน โดยบันทึกเลขที่ใบเสร็จ วันเวลาที่ออก รายละเอียดจำนวนเงิน และการจองที่เกี่ยวข้อง การจองหนึ่งรายการมีใบเสร็จได้ไม่เกินหนึ่งใบ

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่าง
id	BIGINT	ไม่ได้	Primary Key, สร้างค่าแบบ IDENTITY	รหัสใบเสร็จ	1
receipt_number	VARCHAR(255)	ไม่ได้	UNIQUE	เลขที่ใบเสร็จ สร้างจาก REC- ตามด้วย UUID	REC-550e8400-e29b-41d4-a716-446655440000
customer_name	VARCHAR(255)	ได้	—	ชื่อและนามสกุลลูกค้า ณ วันที่ออกใบเสร็จ เก็บเป็น snapshot เพื่อไม่ให้เปลี่ยนตามข้อมูลโปรไฟล์ภายหลัง	สมชาย ใจดี
issued_at	TIMESTAMP WITHOUT TIME ZONE	ไม่ได้	—	วันที่เวลาที่ออกใบเสร็จ	2026-10-04 15:30:00
room_amount	NUMERIC(10,2)	ได้	ทศนิยม 2 ตำแหน่ง	ค่าห้องพักที่บันทึกไว้ในการจอง หน่วยเป็นบาท	1500.00
service_amount	NUMERIC(10,2)	ได้	ทศนิยม 2 ตำแหน่ง	ค่าบริการเสริมที่บันทึกไว้ในการจอง หน่วยเป็นบาท	500.00
surcharge_amount	NUMERIC(10,2)	ได้	ทศนิยม 2 ตำแหน่ง	ค่าธรรมเนียมเพิ่มเติมที่บันทึกไว้ในการจอง หน่วยเป็นบาท	0.00
discount_amount	NUMERIC(10,2)	ได้	ทศนิยม 2 ตำแหน่ง	ส่วนลดที่บันทึกไว้ในการจอง หน่วยเป็นบาท	200.00
total_amount	NUMERIC(10,2)	ไม่ได้	ทศนิยม 2 ตำแหน่ง	ยอดรับชำระที่แสดงในใบเสร็จ หน่วยเป็นบาท	1800.00
booking_id	BIGINT	ไม่ได้	Foreign Key → bookings.i	การจองที่ออกใบเสร็จให้	101

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่าง
			d, UNIQUE		

ความสัมพันธ์

- ใบเสร็จแต่ละใบอ้างอิงการจองหนึ่งรายการ: `receipts.booking_id` อ้างถึง `bookings.id`
- การจองอาจยังไม่มีใบเสร็จ แต่เมื่อออกแล้วจะมีได้ไม่เกินหนึ่งใบ เนื่องจาก `booking_id` มีข้อกำหนด `UNIQUE`
- ใน Entity ความสัมพันธ์ `Receipt.booking` เป็น `OneToOne` และโหลดข้อมูลแบบ `LAZY`

กฎการทำงานที่เกี่ยวข้อง

- `PaymentServiceImpl.confirmPayment()` ยืนยันการรับเงินและเรียก `ReceiptService.createReceipt()` เพื่อออกใบเสร็จ
- `ReceiptServiceImpl.createReceipt()` อ่านการจองผ่าน `BookingRepository.findByIdForUpdate()` หากไม่พบการจองจะเกิด `ResourceNotFoundException`
- ก่อนออกใบเสร็จ การจองต้องมี `paymentStatus = PAID`
- การจองต้องมี `paidAt`, `paidAmount` และ `totalPrice` ครบ โดย `paidAmount` ต้องไม่ติดลบและเท่ากับ `totalPrice`
- หากพบใบเสร็จของการจองอยู่แล้ว Service จะคืนใบเสร็จเดิม โดยไม่สร้างใบใหม่และไม่เปลี่ยนเลขที่หรือวันเวลาที่ออกใบเสร็จ
- หากต้องสร้างใบเสร็จใหม่ การจองต้องมี `roomAmount`, `serviceAmount`, `surchargeAmount` และ `discountAmount` ครบ
- แม้ Entity อนุญาตให้จำนวนเงินทั้งสี่ช่องข้างต้นเป็น `NULL` แต่ขั้นตอนการสร้างใบเสร็จผ่าน Service ต้องมีข้อมูลครบ โดยค่าที่ไม่มีค่าใช้จ่ายสามารถเป็น 0 ได้
- เลขที่ใบเสร็จสร้างด้วย `"REC-" + UUID.randomUUID()` และวันเวลาที่ออกกำหนดด้วย `LocalDateTime.now(ZoneId.of("Asia/Bangkok"))`

- เมื่อสร้างใบเสร็จใหม่ Service นำชื่อและนามสกุล ของเจ้าของการจอง มาบันทึกใน `customer_name` เป็นชื่อ ณ วันที่ออกใบเสร็จ การแก้ไขในโปรไฟล์ภายหลัง ไม่เปลี่ยนชื่อนี้ เพราะ `Receipt.customerName` กำหนด `updatable = false`
- คอลัมน์ `customer_name` เพิ่มผ่าน Flyway
`V3__add_receipt_customer_name.sql` และอนุญาตให้เป็น NULL เพราะ migration ไม่ได้เดิมชื่อย้อนหลัง ให้ใบเสร็จที่ออกก่อนหน้านี้
- Service คัดลอกค่าห้อง บริการเสริม ค่าธรรมเนียมเพิ่มเติม และส่วนลดจากข้อมูลราคาที่บันทึกไว้ในการจอง
- `total_amount` กำหนดจาก `Booking.paidAmount` โดย Service ไม่คำนวณราคาการจองใหม่ขณะออกใบเสร็จ
- การแก้ราคาห้อง บริการเสริม หรือโปรโมชั่นภายหลังไม่เปลี่ยนจำนวนเงินที่คัดลอกไว้ในใบเสร็จโดยอัตโนมัติ
- `ReceiptRepository.findByBookingId()` ใช้ค้นหาใบเสร็จตามรหัสการจอง และ `findByReceiptNumber()` ใช้ค้นหาตามเลขที่ใบเสร็จ
- กฎตรวจสอบสถานะการชำระเงินและความสอดคล้องของยอดเงินตรวจใน Service ไม่ได้ประกาศเป็น CHECK constraint ใน Entity

ตัวอย่างจำนวนเงินในใบเสร็จ

สมมติว่าการจองรหัส 101 ยืนยันรับชำระแล้ว และมีข้อมูลราคาดังนี้

รายการ	จำนวนเงิน
ค่าห้องพัก	1,500.00 บาท
ค่าบริการเสริม	500.00 บาท
ค่าธรรมเนียมเพิ่มเติม	0.00 บาท
ส่วนลด	200.00 บาท
ยอดรับชำระ	1,800.00 บาท

- ยอดของตัวอย่าง: $1,500.00 + 500.00 + 0.00 - 200.00 = 1,800.00$ บาท
- เมื่อ `paidAmount` และ `totalPrice` เท่ากับ 1,800.00 บาท Service จะคัดลอก `paidAmount` ไปเป็น `receipts.total_amount`
- หากเรียกออกใบเสร็จซ้ำและข้อมูลการชำระเงินยังผ่านการตรวจ ระบบจะคืนใบเสร็จเดิมของการจองรหัส 101

- หากการจองยังมี `paymentStatus = UNPAID` ระบบจะปฏิเสธการออกใบเสร็จ

แหล่งอ้างอิงในโค้ด

Receipt, Booking, PaymentStatus, ReceiptServiceImpl,
PaymentServiceImpl, ReceiptRepository, BookingRepository

ตาราง `daily_care_reports`

ตาราง `daily_care_reports`

เก็บรายงานการดูแลสัตว์เลี้ยงรายวันของสัตว์ในการจอง แต่ละรายงานระบุวันที่และผู้บันทึก

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อจำกัด	ความหมาย	ตัวอย่างค า
<code>id</code>	BIGINT	ไม่ได้	Primary Key, IDENTITY BY DEFAULT	รหัสรายงาน	101
<code>booki ng_pe t_id</code>	BIGINT	ไม่ได้	Foreign Key → <code>booking_pets.id</code> ; เป็นส่วนหนึ่งของ Unique Constraint	สัตว์เลี้ยงในการจ องที่ได้รับการดูแล	12
<code>reco rded_b y_id</code>	BIGINT	ไม่ได้	Foreign Key → <code>users.id</code>	ผู้บันทึกรายงาน	3
<code>repo rt_dat e</code>	DATE	ไม่ได้	เป็นส่วนหนึ่งของ Unique Constraint	วันที่รายงานการดู แล	2026- 10-03
<code>feedi ng_mo rning</code>	VARCHAR(25 5)	ได้	—	อาหารที่ให้ช่วงเช้า	อาหารเม็ด 100 กรัม
<code>feedi ng_ev ening</code>	VARCHAR(25 5)	ได้	—	อาหารที่ให้ช่วงเย็น	อาหารเปียก 1 ของ
<code>walki ng_mi nutes</code>	INTEGER	ได้	—	ระยะเวลาเดินเล่น เป็นนาที	15
<code>groom ing</code>	VARCHAR(25 5)	ได้	—	การดูแลความสะอาด	แปรงขน
<code>mood</code>	VARCHAR(25 5)	ได้	—	อารมณ์ของสัตว์เลี้ยง	ร่าเริง
<code>healt h_not e</code>	VARCHAR(25 5)	ได้	—	ข้อมูลสุขภาพประจำวัน	สุขภาพปกติ
<code>gener al_no te</code>	VARCHAR(25 5)	ได้	—	หมายเหตุเพิ่มเติม	กินอาหารครบทั้งสองมือ

คอลัมน์	ชนิดข้อมูล PostgreSQL	NULL ได้	คีย์ / ข้อกำหนด	ความหมาย	ตัวอย่างค่า
	5)				
created_at	TIMESTAMP(6) WITHOUT TIME ZONE	ไม่ได้	ไม่อัปเดตหลังสร้างผ่าน Entity	เวลาที่สร้างรายงาน	2026-10-03 18:30:00

ตัวอย่างค่าเป็นข้อมูลสมมติเพื่อแสดงรูปแบบ ไม่ใช่ข้อมูลจริงหรือค่าเริ่มต้นของฐานข้อมูล
คอลัมน์ที่อนุญาตให้เป็น NULL อาจไม่มีค่าได้

ความสัมพันธ์

- รายงานหลายฉบับอ้างอิงสัตว์ในการจองหนึ่งรายการ: `booking_pet_id` → `booking_pets.id`
- ผู้ใช้หนึ่งคนบันทึกรายงานได้หลายฉบับ: `recorded_by_id` → `users.id`
- ความสัมพันธ์ทั้งสองใน Entity เป็น ManyToOne แบบ LAZY และต้องมีข้อมูลอ้างอิง

ข้อกำหนดไม่ให้รายงานซ้ำ

Entity กำหนด Unique Constraint ชื่อ `uk_care_report_booking_pet_date` บนคอลัมน์ (`booking_pet_id`, `report_date`)

จึงมีรายงานได้ไม่เกินหนึ่งฉบับต่อสัตว์ในการจองหนึ่งรายการต่อหนึ่งวัน

กฎการทำงานที่เกี่ยวข้อง

- `DailyCareReportServiceImpl.createReport()` อนุญาตให้ STAFF หรือ ADMIN บันทึกรายงาน
- การจองต้องอยู่ในสถานะ `CHECKED_IN` หรือ `CHECKED_OUT`
- `report_date` ต้องอยู่ระหว่างวันเช็กอินและวันเช็กเอาท์ของการจอง
- `CreateDailyCareReportRequest` และ `UpdateDailyCareReportRequest` ตรวจสอบให้ `walking_minutes` ไม่ติดลบ กฎนี้ไม่ได้ประกาศเป็น CHECK constraint ใน Entity
- เมื่อตารางได้รับรายงานใหม่ เมธอด `prePersist()` กำหนด `created_at` หากยังไม่มีค่า ค่าเวลานี้กำหนดจากแอปพลิเคชัน ไม่ใช่ค่าเริ่มต้นที่ประกาศในฐานข้อมูล

แหล่งอ้างอิงในโค้ด

DailyCareReport, BookingPet, DailyCareReportServiceImpl,
DailyCareReportRepository, CreateDailyCareReportRequest,
UpdateDailyCareReportRequest

ภาคผนวก ค
คู่มือการติดตั้งและบัญชีทดสอบ

Installation / How to Run

วิธีที่ 1: Docker (แนะนำ)

ต้องมี Docker Desktop

```
cd code
docker compose up --build
```

URL	ใช้ทำอะไร
http://localhost:8080	เว็บ PetsHotel
http://localhost:8080/swagger-ui.html	Swagger UI
http://localhost:8025	Mailpit (ดูอีเมลที่ระบบส่ง)

หยุดระบบ: `docker compose down` (ถ้าต้องการลบข้อมูลใน database ด้วย ใช้ `docker compose down -v`)

วิธีที่ 2: รันบนเครื่อง

ต้องมี JDK 26 และ PostgreSQL

- สร้าง database ชื่อ **pethotel**
- ตั้งรหัสผ่าน database ผ่าน environment variable

Windows (cmd)

```
set "SPRING_DATASOURCE_PASSWORD=รหัสผ่าน PostgreSQL ของคุณ"
```

macOS / Linux

```
export SPRING_DATASOURCE_PASSWORD=รหัสผ่าน PostgreSQL ของคุณ
```

- รันโปรแกรม (Flyway จะสร้างตารางและใส่ข้อมูลตัวอย่างให้อัตโนมัติ)

```
cd code
./mvnw spring-boot:run
```

Windows ใช้ `mvnw.cmd spring-boot:run`

- เปิด `http://localhost:8080`

บัญชีทดสอบ (local / Docker)

Role	Email	Password
ADMIN	admin@petshotel.test	PetsHotel@2026

Role	Email	Password
STAFF	staff@petshotel.test	PetsHotel@2026

บัญชีลูกค้า (CUSTOMER) สมัครเองได้ที่หน้า Register
บน Live demo ใช้รหัสผ่านคนละชุดกับข้างบน

ภาคผนวก ง
ประวัติผู้จัดทำ

นายกิตติธัช คำโท

รหัสนักศึกษา 673380029-0 Section 1

หน้าที่ในโครงการ: Room & Admin, User, Payment, Notification (Observer), Security, Flyway, Docker / Deploy

นายธัช พนมเรียงไชย

รหัสนักศึกษา 673380041-0 Section 1

หน้าที่ในโครงการ: Availability & Room Search, REST API Error Handling, Login / Profile, CI

นางสาวรติมา สวัสดิ์นที

รหัสนักศึกษา 673380055-9 Section 1

หน้าที่ในโครงการ: Pet & Daily Care Report, Test Report

นางสาวกิตติญาดา กองคำ

รหัสนักศึกษา 673380509-6 Section 1

หน้าที่ในโครงการ: Booking & State Pattern

นางสาวนาเดีย คิดอ่าน

รหัสนักศึกษา 673380513-5 Section 1

หน้าที่ในโครงการ: Pricing (Strategy), Promotion, Extra Service, Dashboard